

ADVANCED COMPUTER ARCHITECTURE

Emanuele Torti

Tel. 0382 985371

e-mail: emanuele.torti@unipv.it

Time table: Friday 9 – 11

B4 room

Reception: Wednesday 14 – 16

Teaching material

- Book:
John L. Hennessy, David A. Patterson
Computer Architecture. A Quantitative Approach
- Lectures Slides + handouts
- Documents from the course websites:
Kiro (available soon)

Exam Rules

- **Exam calls**
written test with theoretical questions and exercises
- **Individual project on GPU programming (optional)**
developed on personal equipment, but must run on UNIPV machines and discussed with an oral presentation
- **Maximum grade without project is 26**

Course Program

Requirements: assembly language, fundamentals of computer architecture, operating systems

- Memory hierarchy design
- Cache and coherence
- Virtual memory
- Performance modeling
- Storage systems
- I/O architecture

FOUNDATIONS OF MEMORY HIERARCHY DESIGN

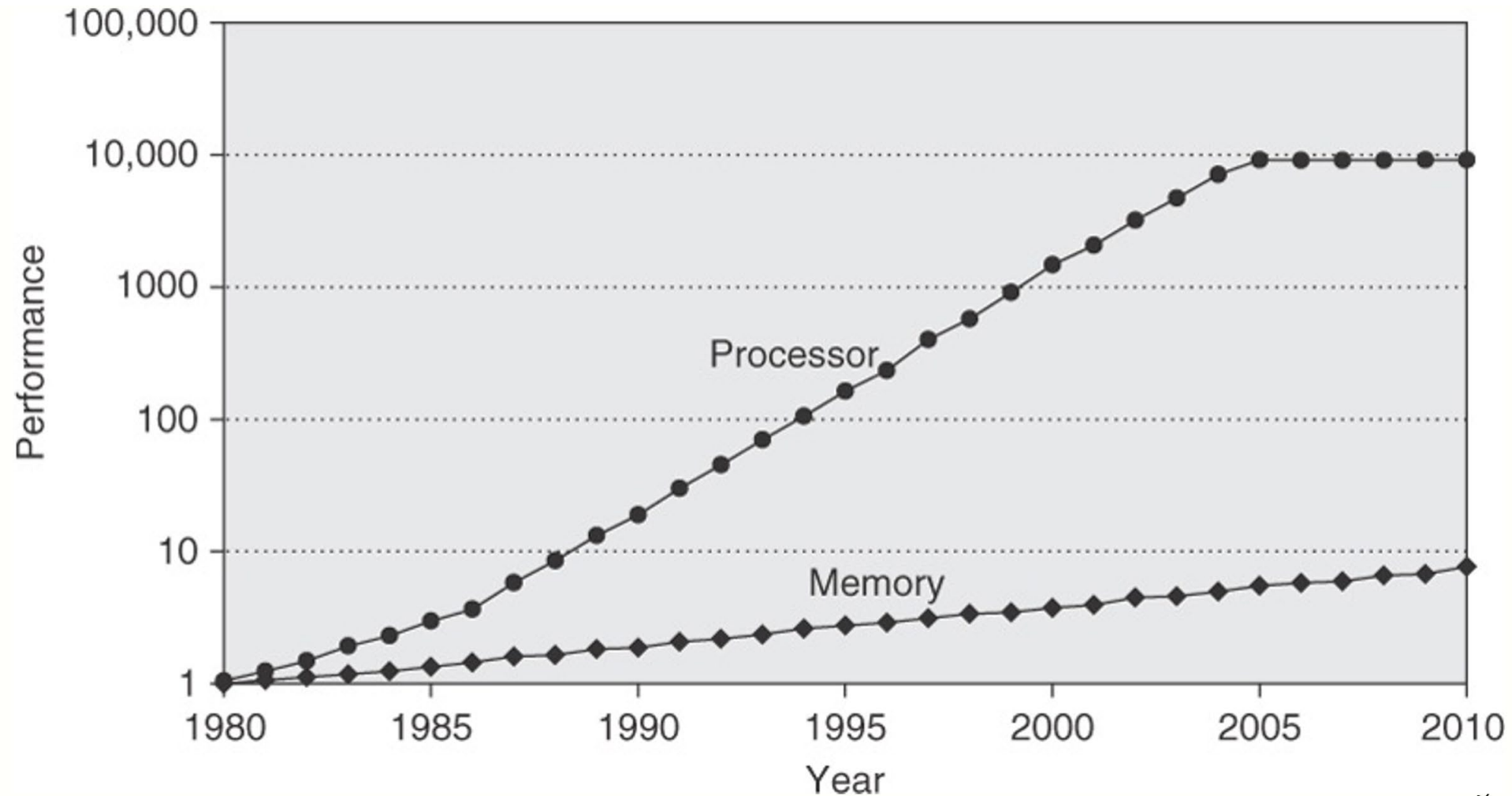
Why memory matters today

The Real Bottleneck:

- CPU frequency scaling slowed
- Core counts increased
- Memory latency improved very slowly
- Data-intensive workloads dominate

Performance today is often limited by memory behavior, not raw compute.

Why memory matters today



Copyright © 2011, Elsevier Inc. All rights Reserved.

Why memory dominates memory systems

The time to execute a program is given by

$$CPU_{time} = Instruction\ Count \times CPI \times Clock\ Cycle\ Time$$

CPI can be expanded in

$$CPI = CPI_{base} + CPI_{memory} + CPI_{branch} + \dots$$

Why memory dominates memory systems

Let us focus on the CPI

$$CPI = CPI_{base} + CPI_{memory} + CPI_{branch} + \dots$$

In early processor the CPI_{base} was dominant, but today CPI_{memory} dominates

Processor performance improved dramatically over decade, but DRAM latency did not scale at the same rate

Why memory dominates memory systems

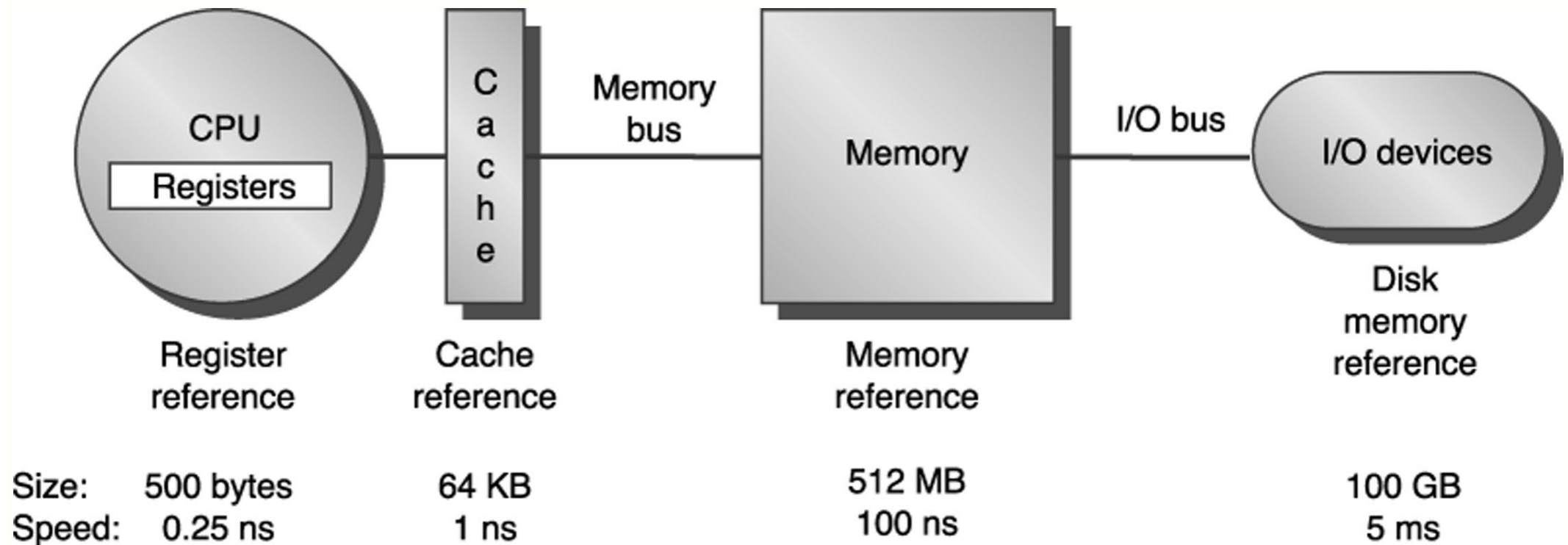
This phenom is known as Memory Wall



We cannot simply build faster CPUs

We must design smarter memory systems

Why memory dominates memory systems



Latency vs Bandwidth

Latency -> time to complete a single memory access

Bandwidth -> data transferred per unit time

Examples:

GPUs have enormous bandwidth, yet the individual memory access may still have significant latency

SSDs provide high throughput, but random access latency is orders of magnitude higher than DRAM

Locality Refresher

The memory hierarchy exists because of one fundamental property: locality

Temporal locality (if you accessed data recently, you will likely to access it again soon)

Spatial locality (if you access address X, you will likely access nearby addresses)

Locality Refresher

Locality exists because programs are structured:

- Loops
- Contiguous data structures
- Stack disciplines
- Cache-friendly compilation

If programs had no locality, caches would be useless!

Working Set Concepts

The working set is the set of memory locations actively used during a time window

Working set fits in cache -> low miss rate

Working set exceeds cache -> thrashing

Cache size decisions depend on

- Target workload
- Expected working set distribution
- Cost and energy constraints

Memory Hierarchy Structure

Single-level memory design is infeasible

Fast memory:

- Expensive
- Large area
- High leakage

Large memory:

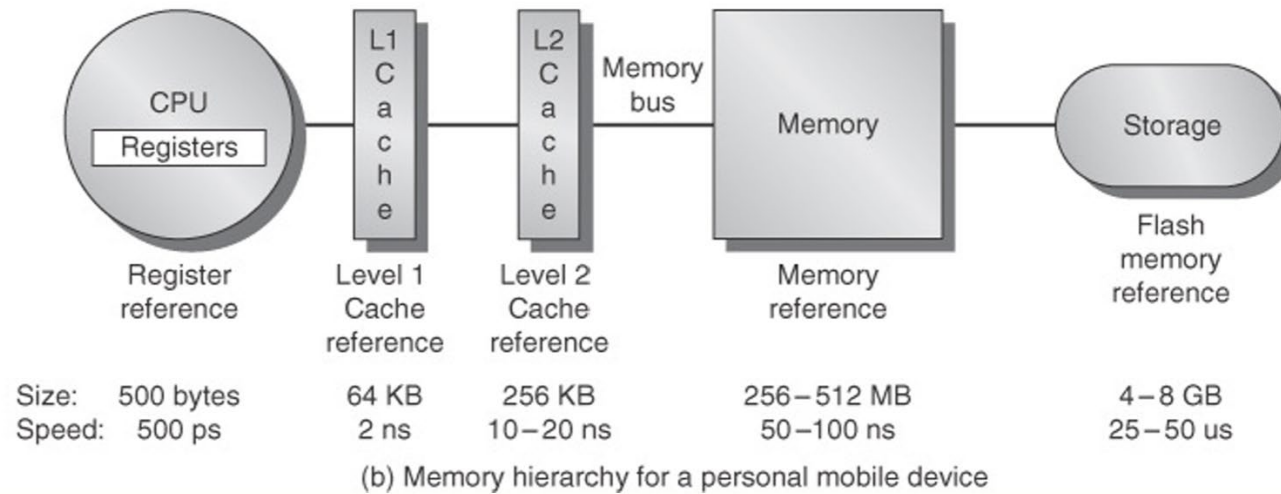
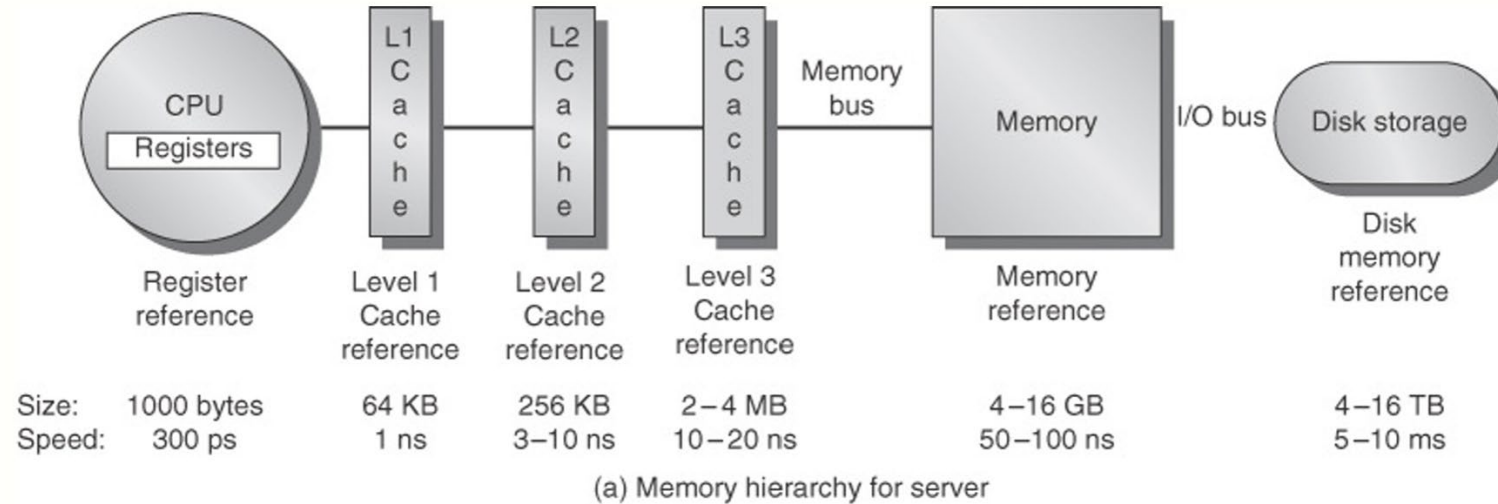
- Slow
- High latency

Memory Hierarchy Structure

Typical structure

- Registers (~1 cycle)
- L1 cache (~1-4 cycles)
- L2 cache (~10 cycles)
- LLC – Last Level Cache (~30-50 cycles)
- DRAM (~80-150+ ns)
- Storage (microseconds to milliseconds)

Memory Hierarchy Structure



Copyright © 2011, Elsevier Inc. All rights Reserved.

Average Memory Access Time

A first fundamental model can be introduced

$$AMAT = HitTime + MissRate \times MissPenalty$$

Examples

Hit time = 1 cycle

Miss rate = 5%

Miss penalty = 50 cycles

$$AMAT = 1 + 0.05 \times 50 = 3.5$$

Miss penalty = 200 cycles

$$AMAT = 1 + 0.05 \times 200 = 11$$

Average Memory Access Time

As hierarchies deepen miss penalty dominates

What to do?

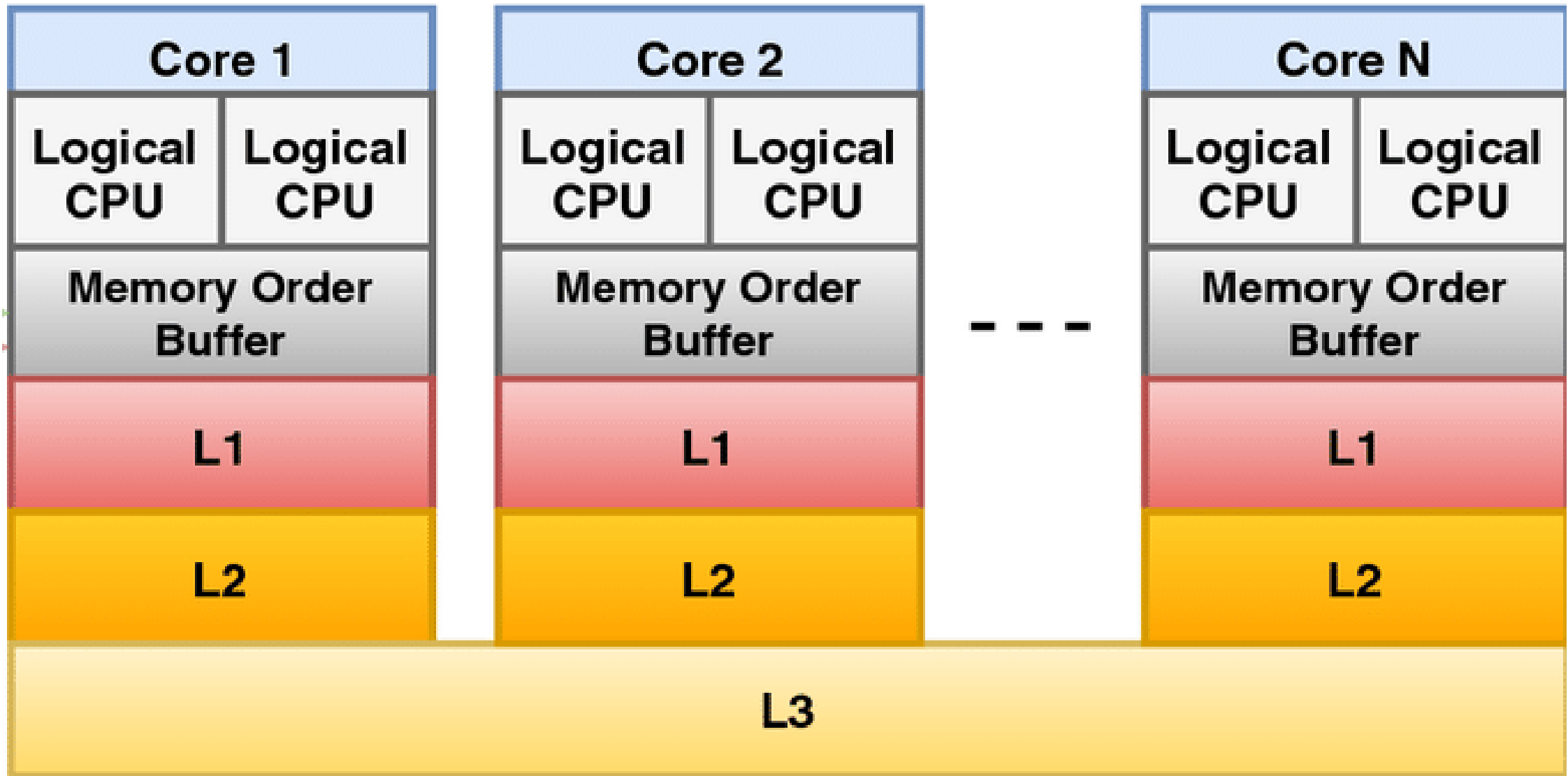
- Reducing miss penalty
- Overlapping misses
- Exploiting memory-level parallelism

Modern Multicore System View

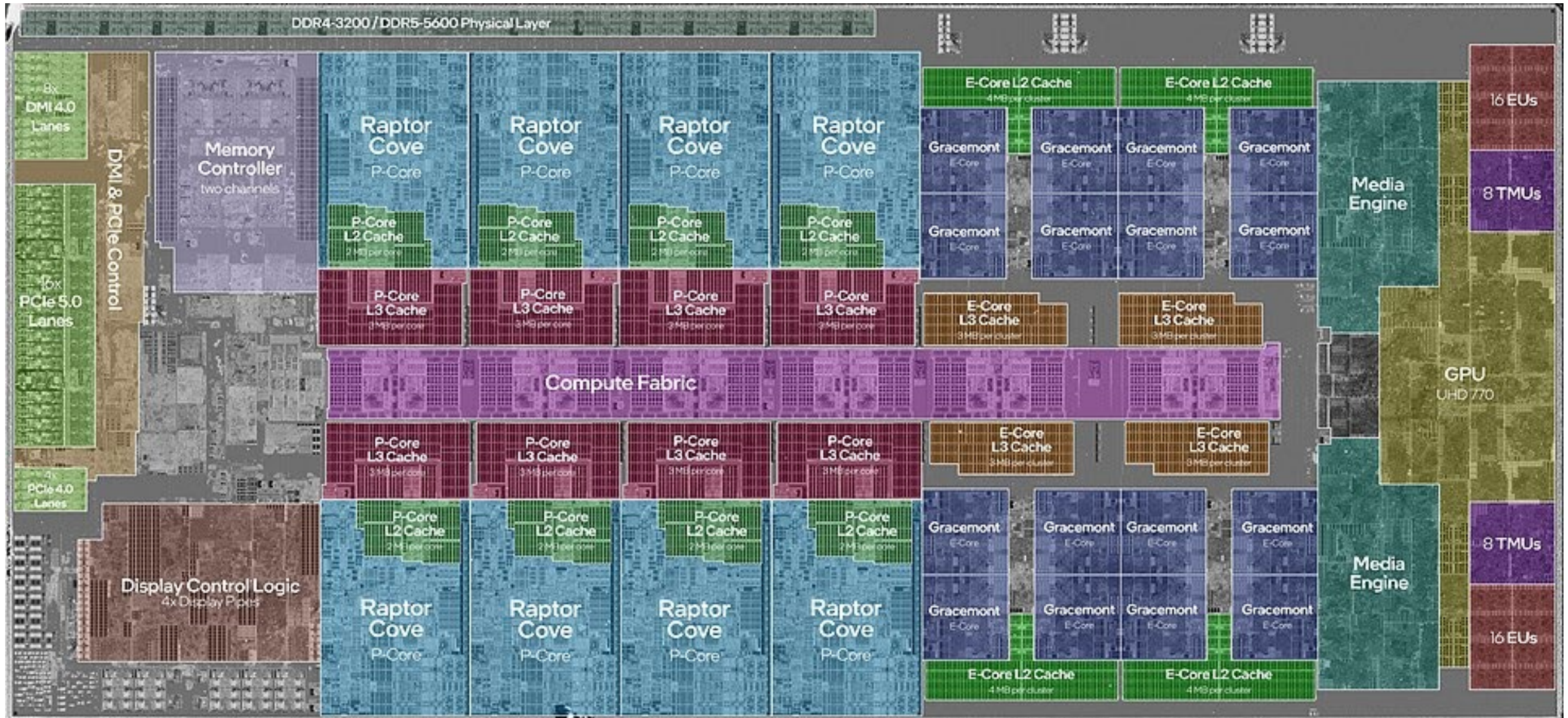
In a modern multicore processor you can observe

- multiple cores
- Private L1 and L2 caches
- Shared L3 cache
- Integrated memory controller
- On-chip interconnect

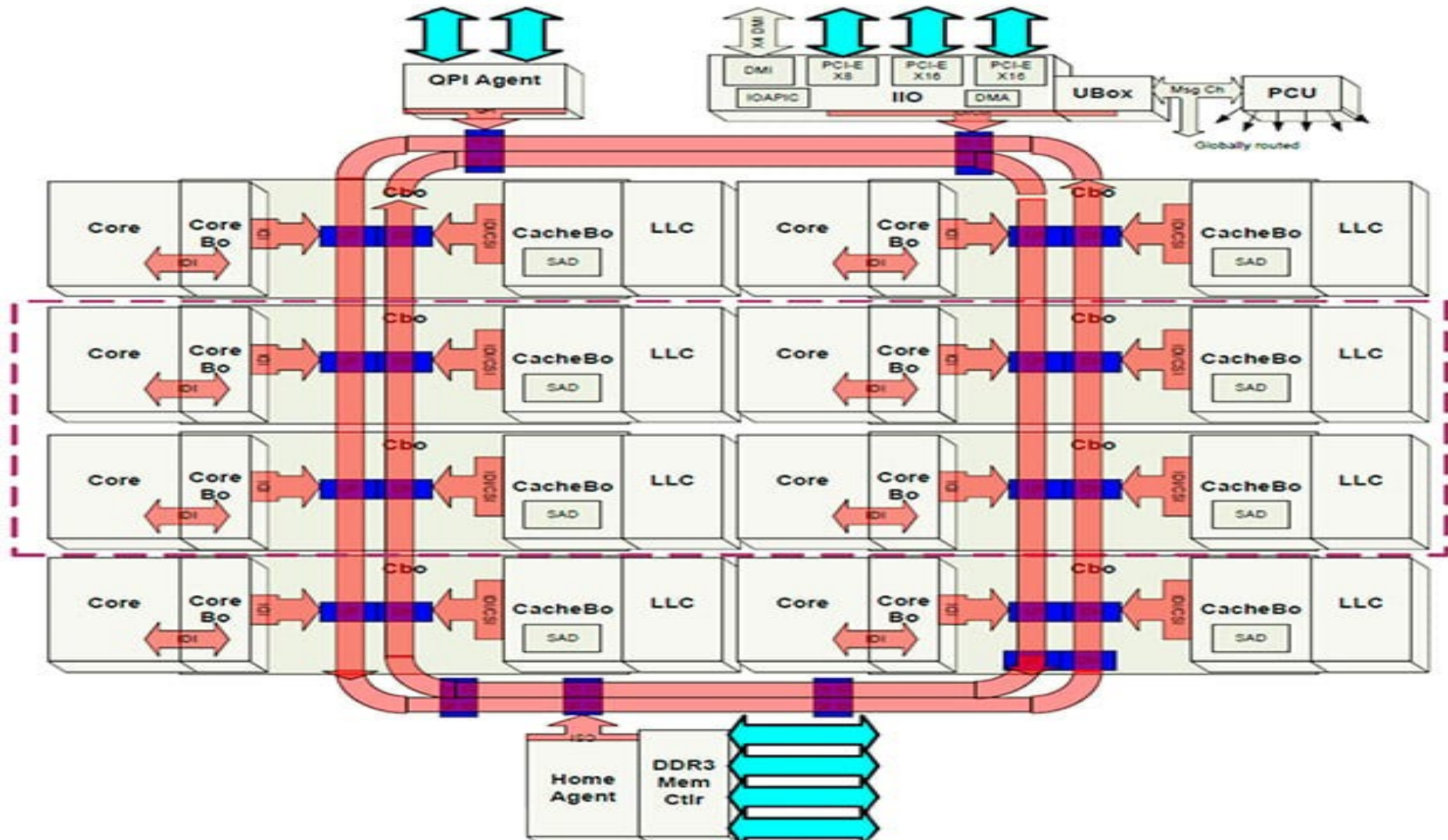
Modern Multicore System View



Modern Multicore System View – i9 13900K



Modern Multicore System View – Intel Xeon



DRAM as a Structured System

DRAM is not a flat array!

It contains:

- Banks
- Rows
- Row buffer

Row activation is expensive and access pattern matters!

Closing

Take home messages:

- Memory dominates modern performance
- Locality enables hierarchy
- Working set matter
- AMAT provides modelling framework
- Memory design is a multi-objective optimization

Next episode: we begin analyzing real cache organization

PROBLEM SET 1

MEMORY FUNDAMENTALS AND AMAT

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

1. Compute AMAT.
2. If miss rate is reduced to 3% compute new AMAT.
3. If miss penalty is reduced by 60 cycles instead, compute AMAT.
4. Which improvements provides larger benefit? Explain.

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

1. Compute AMAT.

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.04 \times 100 = 5 \end{aligned}$$

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

2. If miss rate is reduced to 3% compute new AMAT.

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.03 \times 100 = 4 \end{aligned}$$

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

3. If miss penalty is reduced by 60 cycles instead, compute AMAT.

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.04 \times 60 = 3.4 \end{aligned}$$

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

4. Which improvements provides larger benefit? Explain.

Reducing miss rate -> improvement of 1 cycle

Reducing miss penalty -> improvement of 1.6 cycles

When miss penalty is large even a moderate miss rate causes many stalls cycles (deep hierarchy, DRAM access).

Cutting penalty directly reduces the cost per miss which often dominates total memory time.

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

1. Compute memory stall cycles per instruction.
2. Compute total CPI.
3. If miss penalty doubles, what happens to total CPI?

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

1. Compute memory stall cycles per instruction.

First compute miss frequency per instruction:

$$P(\text{miss per instruction}) = 0.30 \times 0.10 = 0.03$$

(3% of the instructions incur a miss)

$$\text{stall cycles per instruction} = 0.03 \times 50 = 1.5$$

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

2. Compute total CPI.

$$CPI = CPI_{base} + CPI_{memory} = 1 + 1.5 = 2.5$$

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

3. If miss penalty doubles, what happens to total CPI?

$$CPI_{memory} = 0.03 \times 100 = 3$$

$$CPI = CPI_{base} + CPI_{memory} = 1 + 3 = 4$$

CACHE ARCHITECTURE IN MODERN PROCESSORS

Why we need caches

DRAM latency is too high relative to CPU speed

Component	Latency
Registers	~ 1 cycle
L1 cache	~ 4 cycle
L2 cahce	~ 10 cycle
L3 cache	~ 30-50 cycle
DRAM	~ 100-200 cycle

Without caches CPU would stall most of the time

Caches act as small fast memories storing recently used data

Caching

Modern CPUs have separate caches for data and for instructions.

In a pipelined datapath it is possible in the same clock cycle:

- fetch an instruction from the Instruction cache (I-cache)
- load/store data (for a different instruction) from/to the data cache (D-cache)

I-cache and D-cache are together the first-level cache (L1)

Most processors have a second-level cache (L2) and there are processors with a third-level cache (L3)

The Cache Block Concept

Caches do not store single bytes or words, they store block (cache lines)

Typical block size:

- 32 B
- 64 B
- 128 B

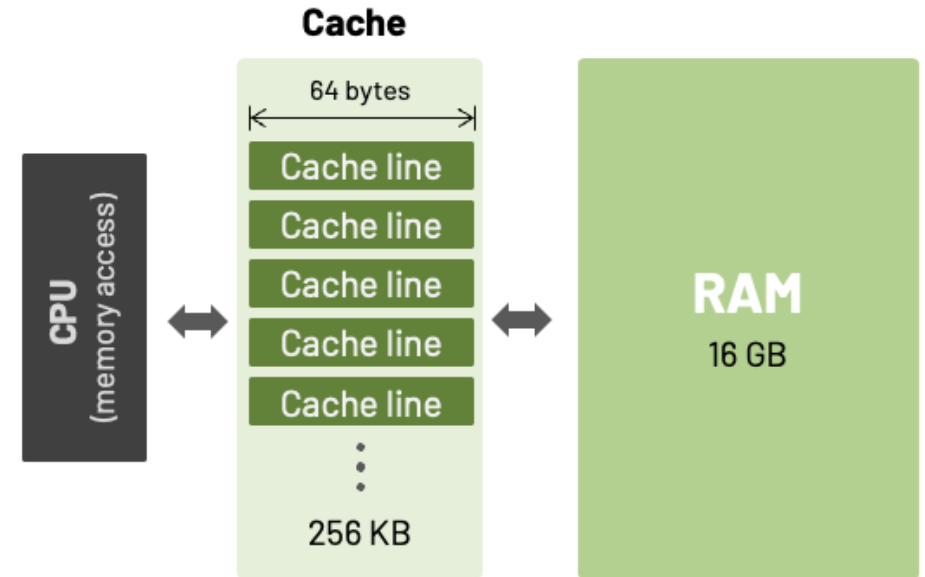
Why ?

To exploit spatial locality

Basic Cache Operation

When CPU requests and address

1. Cache checks if block is present
2. If present -> cache hit
3. If absent -> cache miss
4. Block fetched from next memory level



Miss penalty = time to fetch a block from lower level

Cache Organization Problem

Where should a memory block be placed in cache?

Three main design exists:

1. Direct-mapped
2. Set-associative
3. Fully associative

Each represents a different trade-off

Direct-Mapped Cache

The most simply caches.

A direct-mapped cache consists of 2^n entries (sets)

Each entry stores a block $\rightarrow 2^m$ consecutives bytes (typically 32 or 64) of RAM

Each entry has:

1. A validity bit for the entry
2. A tag that uniquely identifies the block stored in the cache with respect to the RAM

Direct-Mapped Cache

Each memory block maps to exactly one location.

To find the cache location containing the RAM block holding the addressed data or instruction, the computation is

$$\text{Cache index} = (\text{Block address}) \bmod (\text{Number of sets})$$

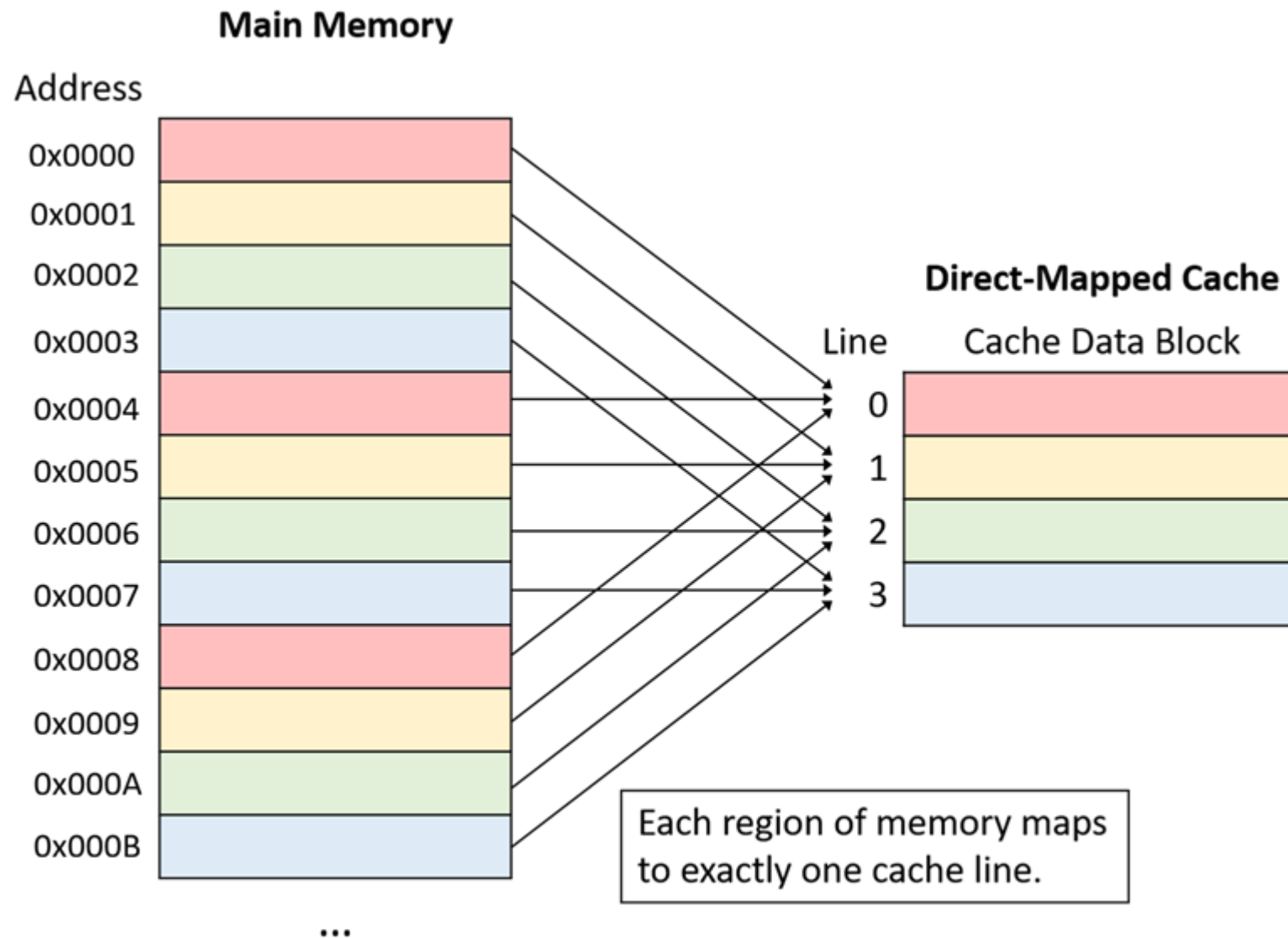
Advantages:

- Very simple
- Very fast
- Low energy

Disadvantages:

- High conflict misses

Direct-Mapped Cache



Direct-Mapped Cache

To locate the data in a cache, the address must be divided in:

- Tag -> verifies correct block
- Index -> selects cache set
- Offset -> selects byte inside block

Example: 32 bits address, cache size of 64 KB and block size of 64 B

Block size = 64 B -> offset 6 bits

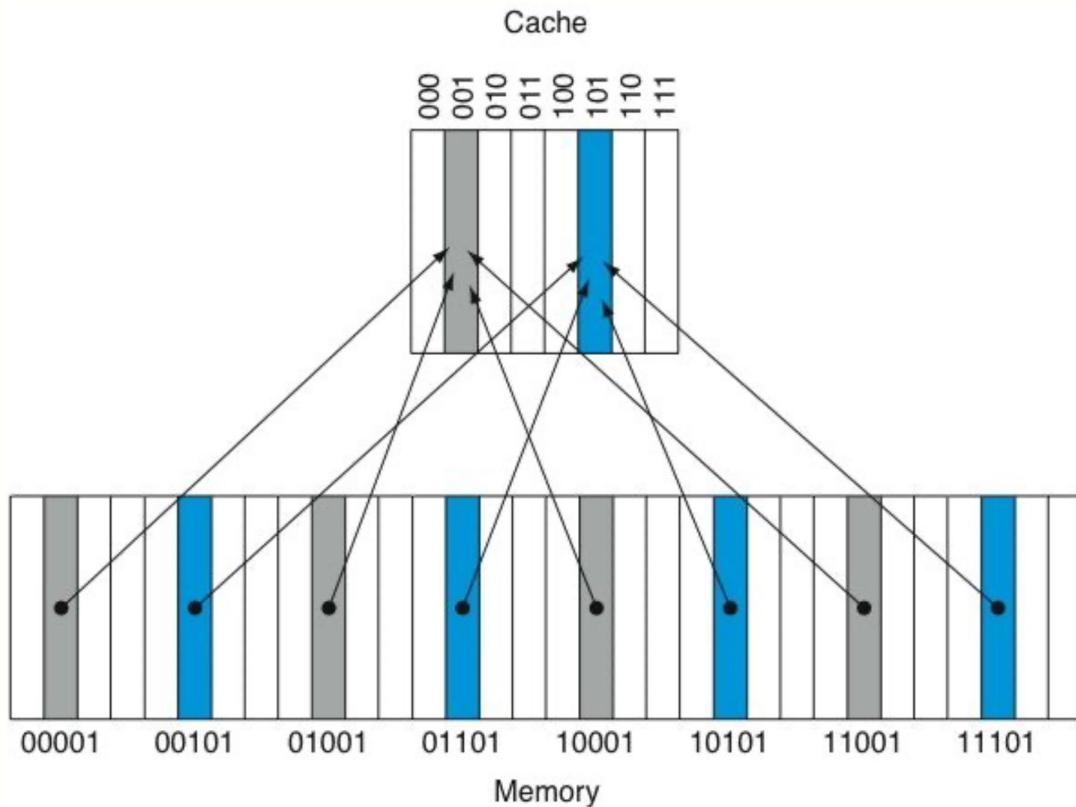
Number of blocks -> $64 \text{ KB} / 64 = 1024$

Index bits -> $\log_2(1024) = 10$

Tag bits -> $32 - 10 - 6 = 16$

Direct-Mapped Cache

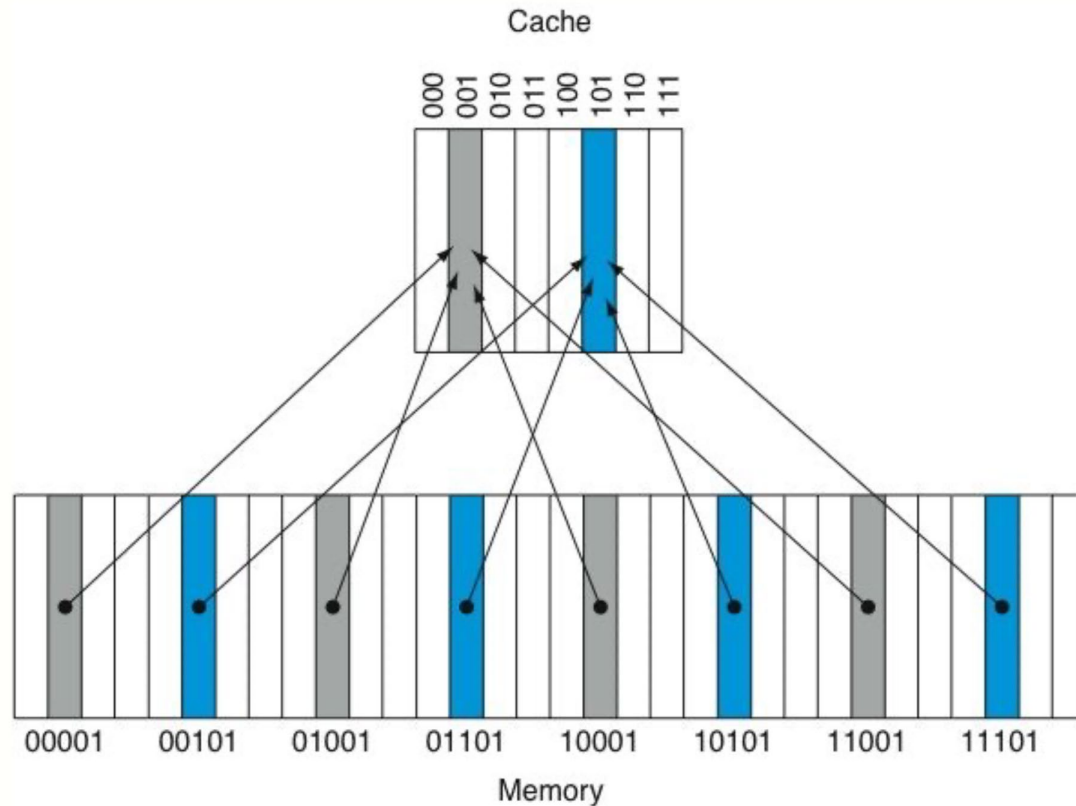
To better understand the tag meaning, consider a RAM split into 32 blocks and an 8-entry cache



The address 00001, 01001, 10001 and 11001 will all match the cache entry 001!

The blocks stored in each entry are discriminated by the tag associated to each cache line.

Direct-Mapped Cache



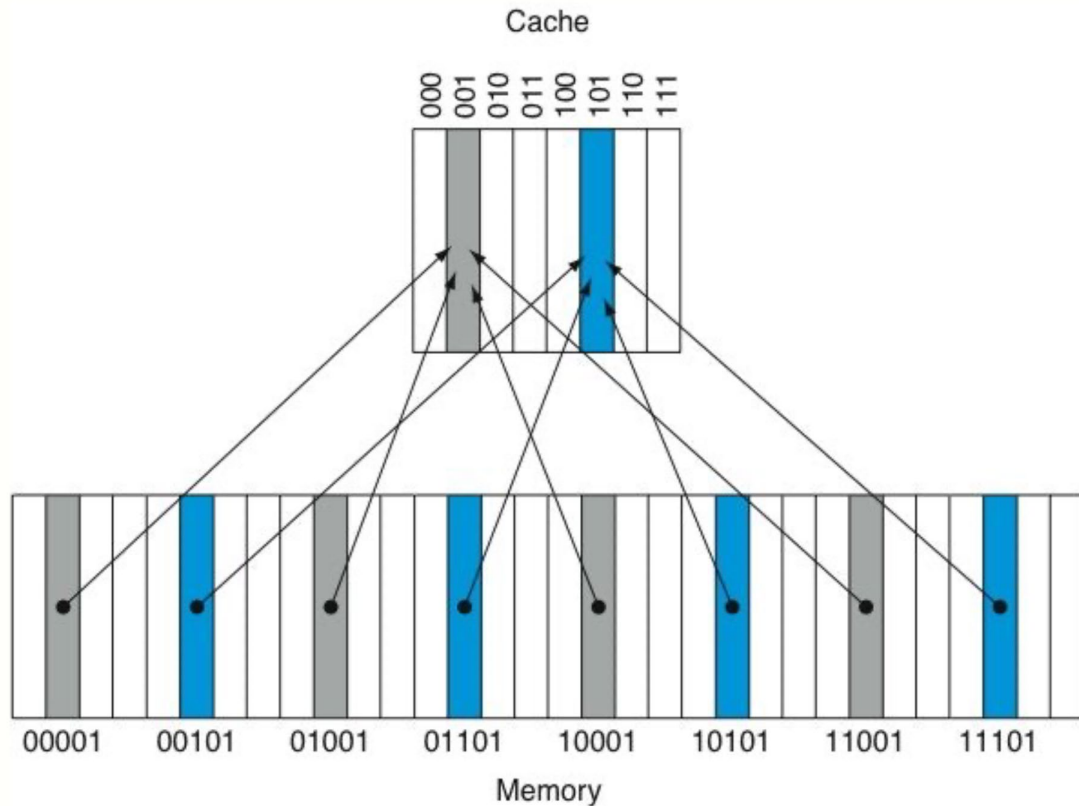
5 bits address

8 cache entries -> 3 index bits

No offset since we have that each block stores a single byte

Tag bits -> $5 - 3 = 2$

Direct-Mapped Cache

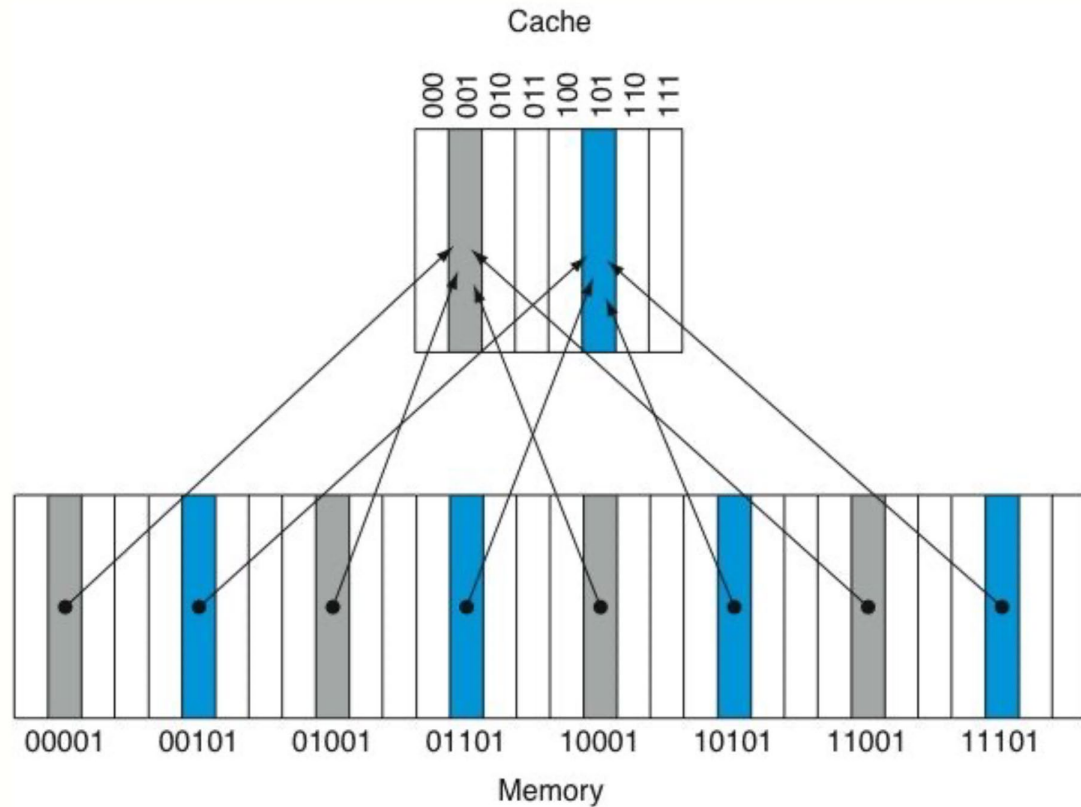


The CPU requests block 01001. It checks if cache entry 001 has its validity bit set

If the bit is set, entry 001 tag is compared with the two most significant bits of the address

If there is a match, cache entry 001 holds the requested line, otherwise there is a cache miss

Direct-Mapped Cache



If the entry is not valid or the tags do not match, there is a cache miss

The missing block is fetched from RAM and it replaces the one present

Direct-Mapped Cache

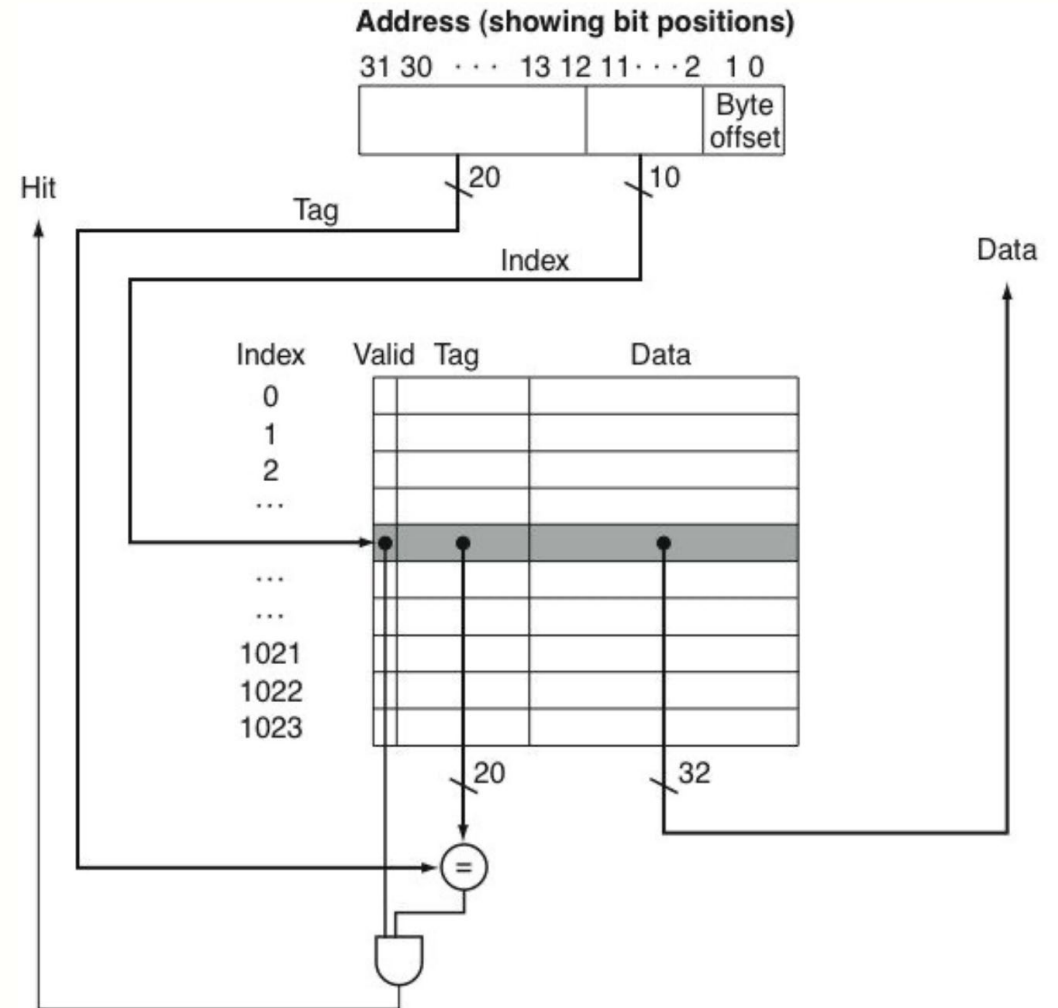
Consider a direct-mapped cache with 1024 4 B entries.

The addresses are 32 bits.

Block size = 4 B $\rightarrow$ offset 2 bits

Index bits $\rightarrow \log_2(1024) = 10$

Tag bits $\rightarrow 32 - 10 - 2 = 20$



Direct-Mapped Cache

The effective size of a cache is larger than the nominal one since it must store all the information to correctly identify the RAM block stored

In the previous case, each cache line stores:

- 1 validity bit

- 20 bits for the tag

- 32 bits for the data

for a total of 53 bits.

Nominal size $\rightarrow 1024 \text{ lines} \times 4 \text{ B} = 4096 \text{ B} = 4 \text{ KB}$

Effective size $\rightarrow 1024 \text{ lines} \times 53 \text{ bits} = 54272 \text{ bits} \cong 6.6 \text{ KB}$

Set-Associative Cache

Born as a solution to conflict misses

Allow multiple blocks per index

Each index corresponds to a set

Block can occupy any way inside the set.

Example:

- 2-way
- 4-way
- 8-way

Set-Associative Cache

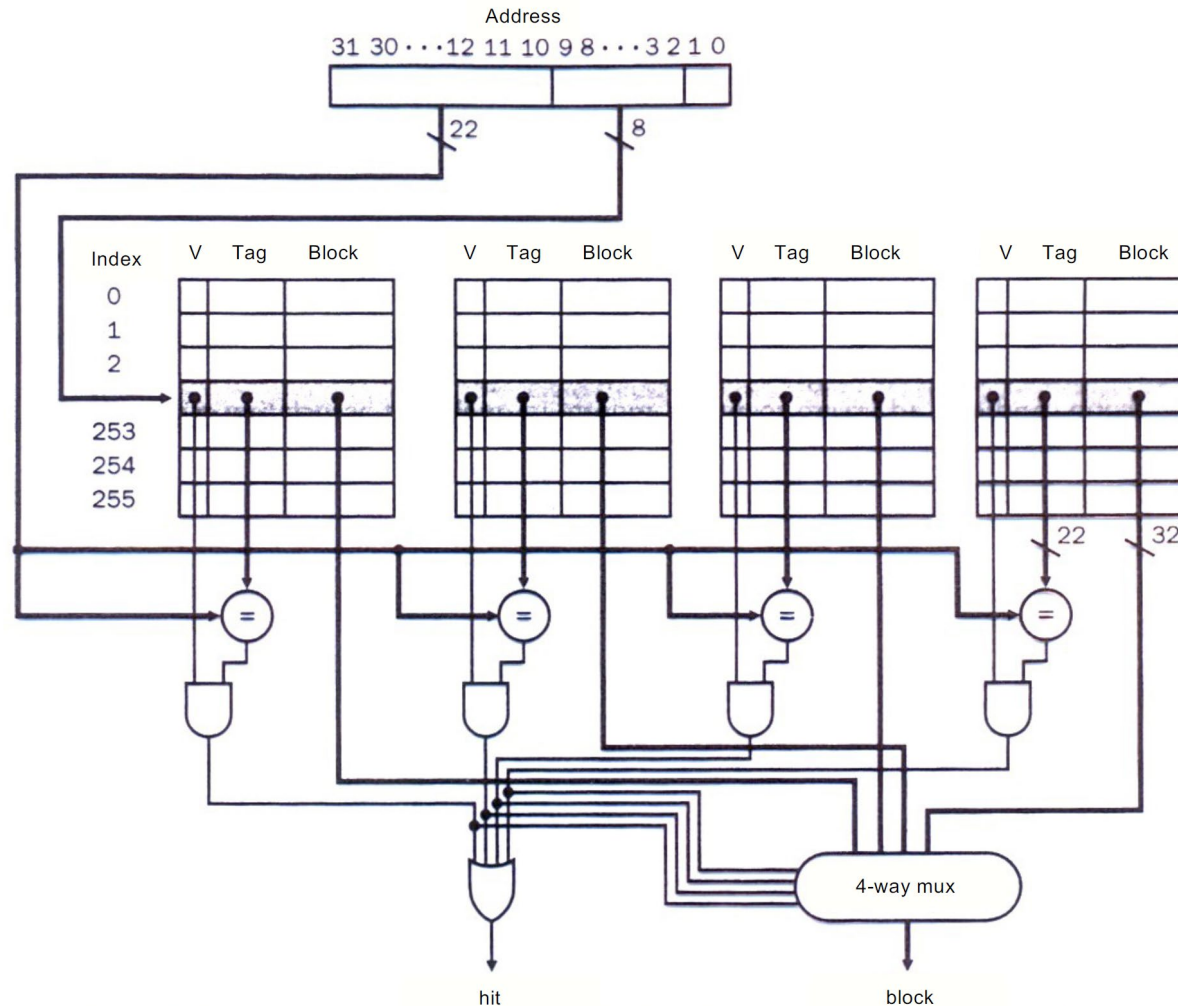
Operations:

1. Index selects a set
2. All tags in set compared in parallel
3. Matching tag indicate hits

Trade-off is more hardware complexity than directly-mapped cache

Set-Associative Cache

Structure of a 4-way set-associative cache with 256 sets (8 bit index)



Fully Associative Cache

Most flexible design

Block can be placed anywhere in cache

No index

All tags are compared simultaneously

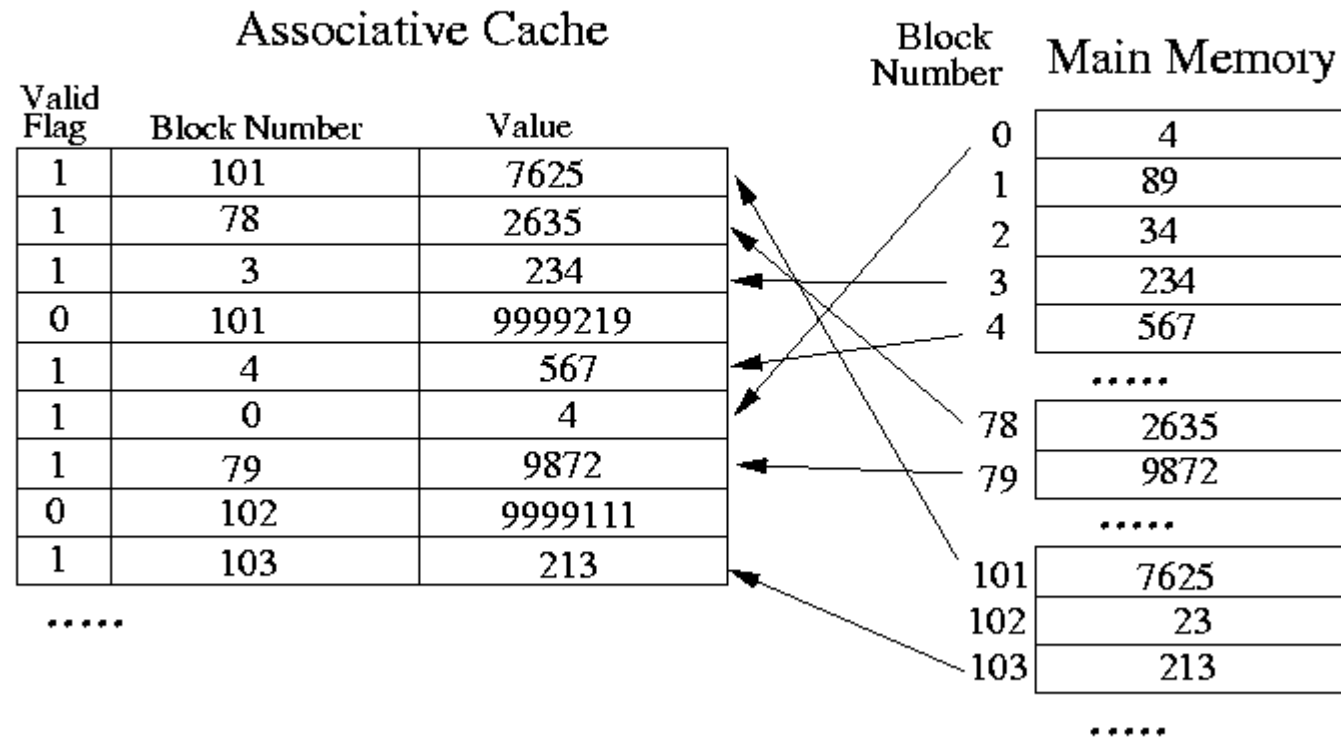
Advantages:

- Eliminates conflict misses

Disadvantages:

- Expensive
- Slow for large caches

Fully Associative Cache



Associativity Trade-Off

Advantages of increasing associativity

- Fewer conflict misses
- Better cache utilization

Disadvantages of increasing associativity:

- Longer hit time
- Higher energy
- More complex hardware

Associativity Trade-Off

Engineering reality

Level	Associativity
L1	4-8 ways
L2	8 ways
L3	12-16 ways

Block Size Trade-Off

Block size (typical 64 B in modern CPUs) influences:

- Spatial locality
- Miss penalty
- Bandwidth usage

Small blocks:

- Lower miss penalty
- Less bandwidth waste

Large blocks:

- Better spatial locality
- Higher miss penalty

Interaction with DRAM

Block size must match DRAM behavior

DRAM accesses entire rows efficiently

Fetching blocks aligns with

- Memory burst transfers
- Bus width

Cache block size is also a memory system design choice

Real Processor Example

Intel Core architecture typical hierarchy:

- L1: 32 KB, 8 way
- L2: 256-512 KB, 8 way
- L3: several MB, 12-16 way

Design goals:

L1 -> minimal latency

L3 -> maximize capacity

Different levels optimize different metrics

Key Engineering Trade-Offs

Cache design balances:

- Hit time
- Miss rate
- Power
- Area
- Scalability

These trade-offs drive real processor design

Closing

Summary of key concepts:

- Cache block
- Address decomposition
- Cache mapping strategies
- Associative trade-offs
- Block size design

Next episode: cache performance optimization (miss types, replacement policies, write policies, non-blocking caches)

CACHE PERFORMANCE AND MISS BEHAVIOR

Cache Performance Optimization

Topics:

- Why cache miss
- The 3C model of cache misses
- Conflict behavior
- Replacement policies
- Techniques used in real processors

What is a cache miss?

A cache miss occurs when requested data is not present in cache

Miss consequences:

1. Access lower level memory
2. Fetch block
3. Replace existing block

Cost = miss penalty

In modern system this can exceed 200 cycles

Why do misses occur?

Misses occur for three fundamental reasons

They were formalized in the 3C model:

1. Compulsory misses
2. Capacity misses
3. Conflict misses

Understanding these helps guide architectural design

Compulsory misses

Also called cold misses

Occur when data is accessed for the first time

Characteristics:

- Cannot be avoided by cache size
- Only reduced by prefetching

Example: first iteration of a loop accessing an array

Capacity misses

Occurs when the working set exceeds cache size

Even with perfect mapping, cache cannot hold all blocks

Solution:

- Larger caches
- Multi-level caches

Example: cache is 64 KB but the program uses 2 MB of active data

Conflict misses

Occurs when multiple blocks map to same cache set

Common in direct-mapped caches and low associative designs

Solution:

- Increase associativity
- Better indexing schemes

Example: two arrays mapping to same index: they repeatedly evict each other

Cache misses - example

A processor has 32 bit addresses and a 256 lines direct-mapped cache, with a 16 B block. Compute the total misses of the following assembly code.

```
    la $t0, 0xC0ABD030
    la $t1, 0xD8EFC030
    li $t2, 32
    li $t3, 0
loop: lbu $t4, 0($t0)
      lbu $t5, 0($t1)
      addiu $t0, $t0, 1
      addiu $t1, $t1, 1
      addu $t3, $t4, $t5
      addiu $t3, $t3, 1
      bne $t3, $t2, loop
```


Cache misses - example

```
        la $t0, 0xC0ABD030
        la $t1, 0xD8EFC030
        li $t2, 32
        li $t3, 0
loop:   lbu $t4, 0($t0)
        lbu $t5, 0($t1)
        addiu $t0, $t0, 1
        addiu $t1, $t1, 1
        addu $t3, $t4, $t5
        addiu $t3, $t3, 1
        bne $t3, $t2, loop
```

Address decomposition:

16 B block -> 4 bit offset

256 lines -> 8 bit cache index

$32 - 4 - 8 = 20$ bit tag

Memory content:

0xC0ABD030 -> 32 B vector A

0xD8EFC030 -> 32 B vector B

Cache misses - example

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		
4	0		
.			
.			
.			
254	0		
255	0		

```
la $t0, 0xC0ABD030
la $t1, 0xD8EFC030
li $t2, 32
li $t3, 0
loop: lbu $t4, 0($t0)
      lbu $t5, 0($t1)
      addiu $t0, $t0, 1
      addiu $t1, $t1, 1
      addu $t3, $t4, $t1
      addiu $t3, $t3, 1
      bne $t3, $t2, loop
```

Cache misses - example

First load: `lbu $t4,0($t0)`

`$t0` contains the address `0xC0ABD030` -> offset = 0

cache index = 03

tag = C0ABD

Compulsory miss and caching of the first 16 B of vector A

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of vector A
4	0		
.			
.			
254	0		
255	0		

Cache misses - example

Second load: `lbu $t5,0($t1)`

`$t1` contains the address `0xD8EFC030` -> offset = 0

cache index = 03

tag = D8EFC

Conflict miss and caching of the first 16 B of vector B

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	D8EFC	16 B of vector B
4	0		
.			
.			
254	0		
255	0		

Cache misses - example

The loop iterates over 32 iterations

At each iteration there are 2 misses

These two misses are conflict misses (only in the first and sixteenth iteration one is compulsory misses)

The total number of misses is then 64!

More in detail 63 conflict misses and 1 compulsory misses

Access pattern A B A B A B -> miss rate = 100%

Cache misses - example

Higher associativity reduces conflict misses

In this example, if we assume a 2-way cache we have 2 blocks per set

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of vector A
4	0		

.

.

254	0		
255	0		

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	D8EFC	16 B of vector B
4	0		

.

.

254	0		
255	0		

Cache misses - example

In the first iteration we have 2 compulsory misses

At the 16th iteration we have 2 compulsory misses (block size is 16 B, array A and B are 32 B each)

No conflict misses!

Miss rate = $4/64 = 6.25\%$

Replacement Problem

When cache is full, we must choose a block to evict

This is the replacement policy

Goal: evict block least likely to be used

Optimal policy: evict block whose next use is farthest in the future

Impossible in practice

It provides upper bound on performance

Last Recently Used (LRU) Replacement

Block not used recently unlikely to be used soon

Works well when temporal locality is strong

Used historically in many caches

LRU requires tracking usage order

For N-way caches LRU state complexity grows

Pseudo LRU

Approximates LRU with lower cost

Trade-off:

- Slightly worse miss rate
- Much lower hardware cost

Popular techniques:

- Tree-based PLRU
- Not Recently Used (NRU)

Random Replacement

Simple strategy: choose a random block

Surprisingly effective in large caches

Advantages:

- Very low hardware complexity
- Avoids pathological patterns

Used in some GPUs

The write problem

So far we focused on reads

Writes introduce additional complexity

When the CPU writes to a cache block, what should happen?

Two fundamentals write policies:

- Write-Through (WT) -> write goes to cache and to memory immediately
- Write-Back (WB) -> write goes only to cache and the memory is updated later, on eviction

Write-Through

Characteristics:

- ✓ Simple
- ✓ Memory always consistent
- ✓ Easy for I/O coherence
- ✗ High memory traffic
- ✗ Lower performance

Often used in L1 caches (historically) and simple systems

Write-Back

Characteristics:

- ✓ Lower memory traffic
- ✓ Better performance
- ✓ Exploits temporal locality
- ✗ More complex
- ✗ Requires dirty bit
- ✗ More complex coherence handling

Used in most modern CPUs (L2/L3 always WB)

Write-Back – Dirty bit concept

Each cache block has:

- Validity bit
- Tag
- Dirty bit

Dirty = modified in cache but not in memory

On eviction:

if dirty -> write to memory
if clean -> discard block

Write Miss Policies

What happens on a write miss?

Two options:

Write Allocate (fetch on write)

- Load block into cache and then write

No Write Allocate (Write-around)

- Write directly on memory and do not load block into cache

Policy combination

Common combinations

Policy	Miss Policy
Write-through	No Write Allocate
Write-back	Write Allocate

Why?

Write-back benefits from locality -> keep data in cache

Write-through avoids cache pollution

Impact on Performance

Write policies affect:

- Memory bandwidth usage
- Energy consumption
- Latency
- Coherence traffic

In multicore systems:

Write-back Increases coherence complexity

Write-through increases interconnect traffic

Real System Design Choices

Typical modern CPUs:

- L1:
 - Often write-back
 - Sometimes write-through (older/simpler designs)
- L2/L3:
 - Always write-back

Reason:

Reduce memory bandwidth pressure and improve energy efficiency

Engineering Trade-Off Summary

Aspect	Write-Through	Write-Back
Memory traffic	High	Low
Complexity	Low	High
Performance	Lower	Higher
Coherence	Easier	Harder

Techniques to Reduce Misses

There are several strategies

1. Increase cache size
Reduce capacity misses
2. Increase associativity
Reduce conflict misses
3. Larger blocks
Exploits spatial locality
4. Hardware prefetching
Reduce compulsory misses

Hardware Prefetching

Prefetching anticipates future access

Benefits:

- Hides memory latency
- Reduce compulsory misses

Risk:

- Useless prefetches increase bandwidth pressure

Example: sequential array traversal. Prefetch next block before it is requested.

Engineering Perspective

Cache performance depends on:

- Workload access pattern
- Data structure layout
- Compiler optimizations
- Hardware prefetching
- Cache hierarchy

Memory performance is software-hardware co-design!

Closing

Summary of key concepts:

- Why caches miss
- The 3 C miss model
- Replacement policies
- Techniques to reduce misses

Next episode: multi-level cache hierarchies (L1/L2/L3 interactions, inclusive vs exclusive caches and modern CPU design)

MULTILEVEL CACHE HIERARCHIES AND SYSTEM DESIGN

Cache Performance Optimization

Topics:

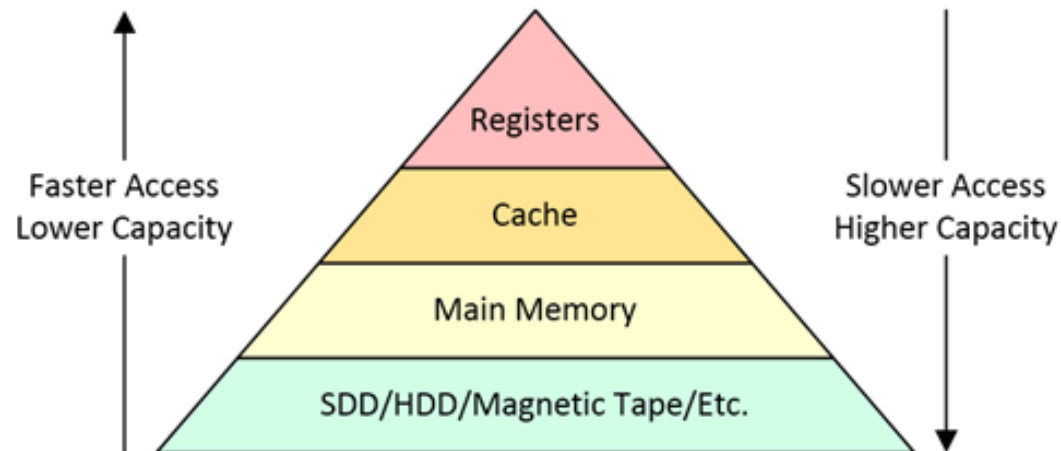
- Why multiple cache levels exist
- How AMAT extends to multiple levels
- Roles of L1, L2 and L3
- Inclusive vs exclusive hierarchies
- Private vs shared caches

Why Single Level is not Enough

Trade-off:

- Small cache -> fast but many misses
- Large cache -> slow but fewer misses

We cannot optimize both simultaneously!



Multilevel Hierarchy

Introduce:

- L1 -> very fast, small
- L2 -> medium
- L3 (LLC) -> large, slower

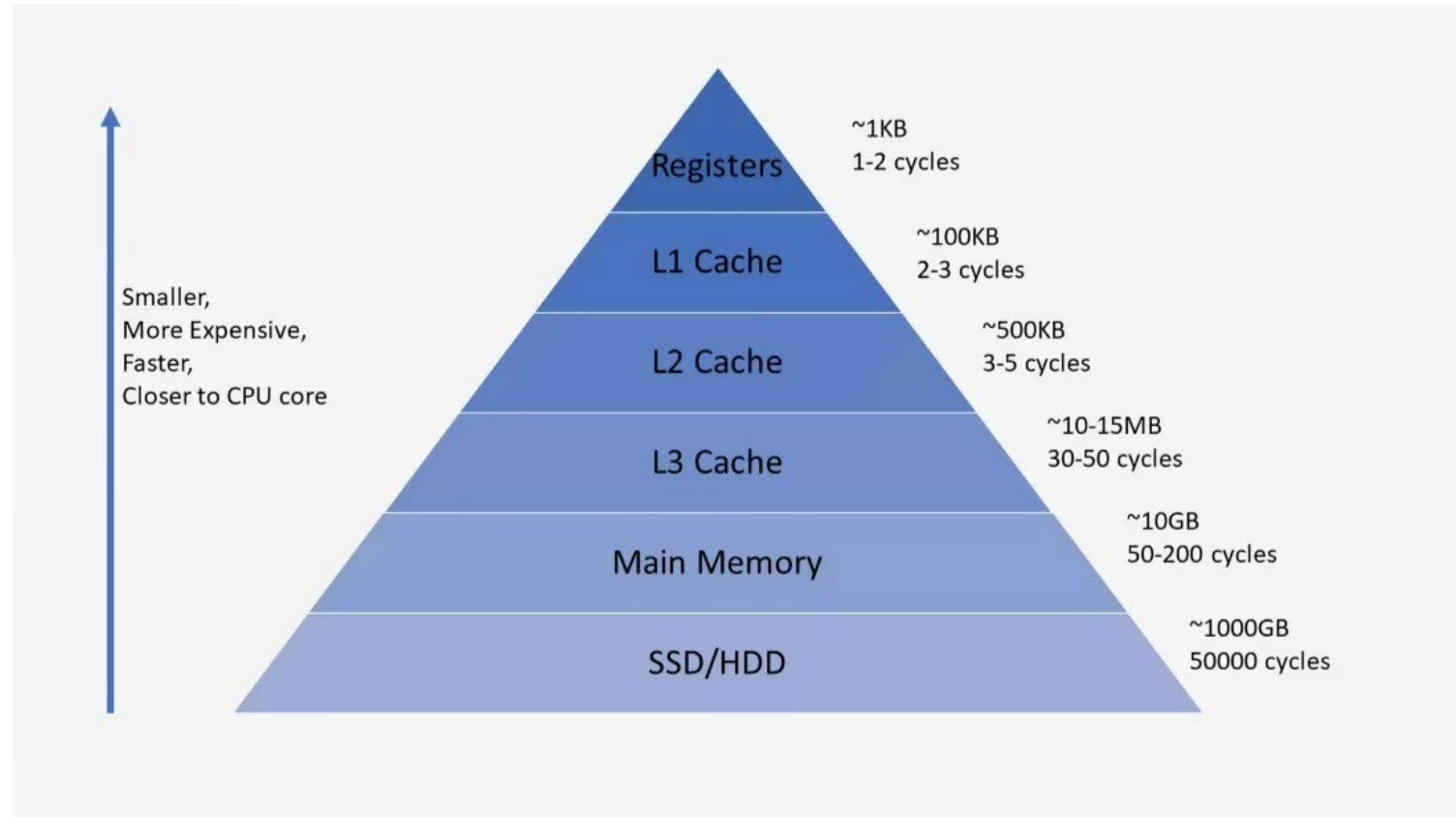
The idea is that each level reduces the effective miss penalty

Multilevel Access Flow

Access flow:

1. Check L1
2. Miss -> check L2
3. Miss -> check L3
4. Miss -> Dram

Each level adds latency



AMAT for Two Levels

The generic AMAT equation is:

$$AMAT = HitTime + MissRate \times MissPenalty$$

This equation can be specified for the two levels scenario:

$$AMAT = HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2} \times MP_{L2})$$

Interpretation: L2 reduces effective penalty of L1 misses

AMAT for Two Levels

Example:

L1 HT = 1, MR = 5%

L2 HT = 10, MR = 20%

Dram penalty = 100

$$\begin{aligned} AMAT &= HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2} \times MP_{L2}) = \\ &= 1 + 0.05 \times (10 + 0.20 \times 100) = 2.5 \end{aligned}$$

Without L2 we have:

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.05 \times 100 = 6 \end{aligned}$$

Role of Each Level

L1 Cache

- Extremely fast, small, on critical path

L2 Cache

- Reduces L1 miss penalty, moderately fast

L3 (LLC)

- Reduces DRAM accesses, large capacity

Level	Priority
L1	Latency
L2	Balance
L3	Capacity

The Problem: Conflict Misses in L1

Even with good cache size and low latency L1 (often low associativity e.g. 4-8 ways) still suffers from conflict misses

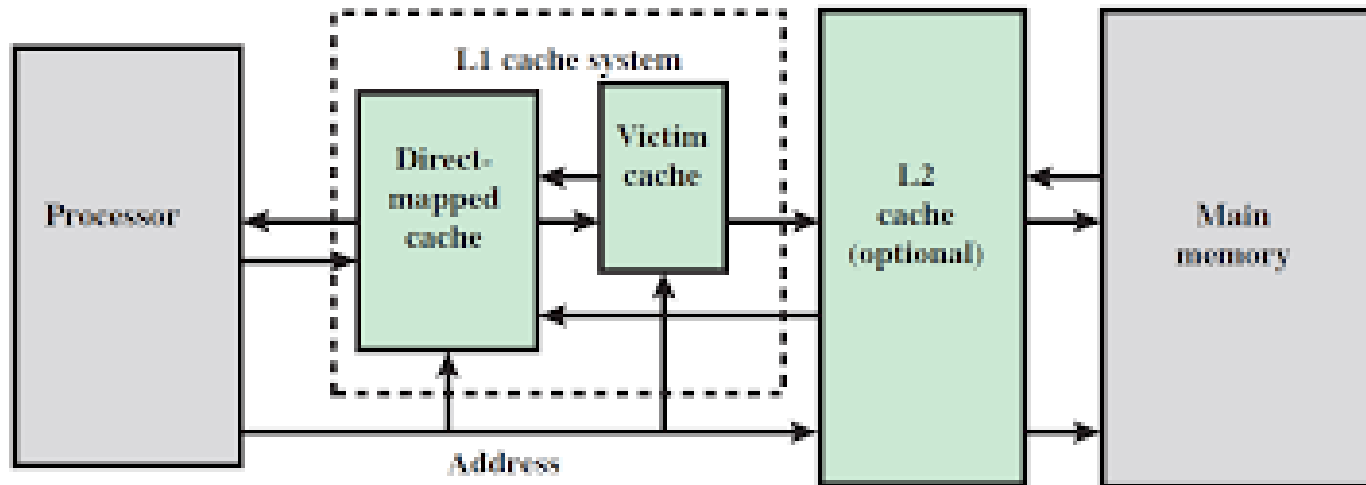
Solution -> Victim cache, a small fully-associative buffer placed between L1 and L2

Purpose -> store recently evicted cache lines from L1

Victim Cache

Flow:

1. L1 miss
2. Check victim cache
3. If hit -> swap with L1
4. If miss -> go to L2

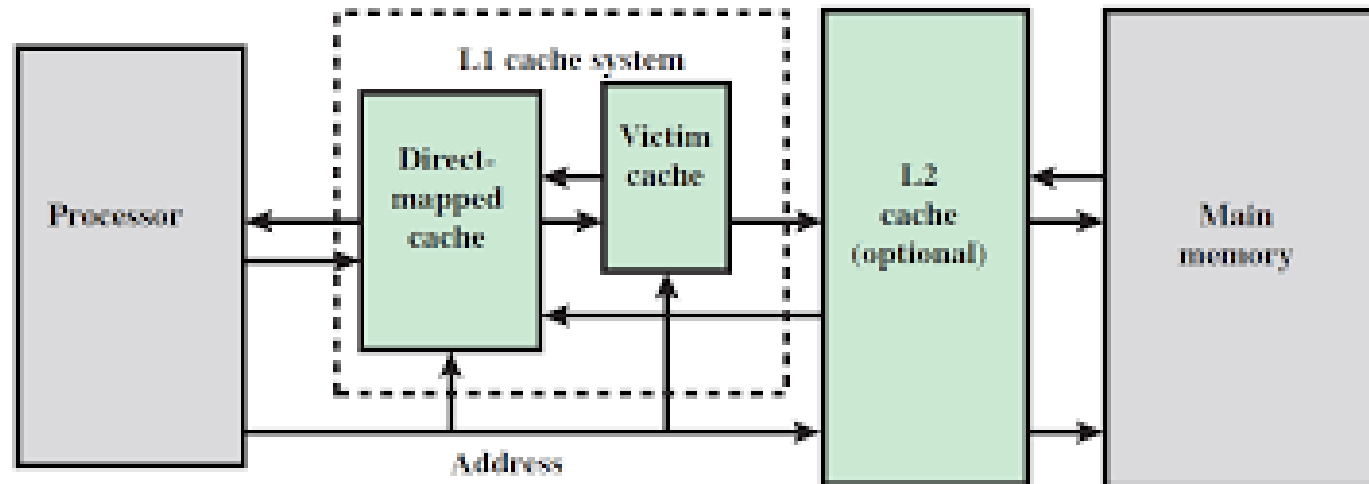


Key Mechanism: Swap

Victim cache uses swap, not copy!

If hit in victim cache:

- Bring block back to L1
- Move evicted L1 block into victim cache



Victim Cache - Example

A processor has 32 bit addresses and a 256 lines direct-mapped cache a 4 lines victim cache, each with a 16 B block. Compute the total misses of the following assembly code.

```
    la $t0, 0xC0ABD030
    la $t1, 0xD8EFC030
    li $t2, 32
    li $t3, 0
loop: lbu $t4, 0($t0)
      lbu $t5, 0($t1)
      addiu $t0, $t0, 1
      addiu $t1, $t1, 1
      addu $t3, $t4, $t5
      addiu $t3, $t3, 1
      bne $t3, $t2, loop
```

Victim Cache - Example

L1 cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		
4	0		
		.	
		.	
		.	
54	0		
55	0		

Victim cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		

Victim Cache - Example

First load: `lbu $t4,0($t0)`

`$t0` contains the address `0xC0ABD030` -> offset = 0

cache index = 03

tag = C0ABD

Compulsory miss and caching of the first 16 B of vector A

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of array A
4	0		

.

254	0		
255	0		

L1 cache

Victim cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		

Victim Cache - Example

Second load: `lbu $t5,0($t1)`

`$t1` contains the address `0xD8EFC030` -> offset = 0

cache index = 03

tag = D8EFC

Conflict miss and caching of the first 16 B of vector B, evicted block moved to victim cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	D8EFC	16 B of array B
4	0		

L1 cache

.

254	0		
255	0		

Victim cache

	validity bit	tag	data
0	1	C0ABD	16 B of array A
1	0		
2	0		
3	0		

Victim Cache - Example

third load: `lbu $t4,0($t0)`

`$t1` contains the address `0xC0ABD031` -> offset = 1

cache index = 03

tag = C0ABD

Conflict miss but hit in victim cache -> swap L1 and victim

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of array A
4	0		

.

254	0		
255	0		

L1 cache

Victim cache

	validity bit	tag	data
0	1	D8EFC	16 B of array B
1	0		
2	0		
3	0		

Victim Cache - Example

The loop iterates over 32 iterations

At the first iteration there are 2 misses

The victim cache avoids conflict misses at the next iterations

At the 16th iteration we have 2 compulsory misses (block size is 16 B, array A and B are 32 B each)

No conflict misses!

Miss rate = $4/64 = 6.25 \%$

What type of Misses Does it Fix?

Victim cache targets conflict misses

It does not help with

- Compulsory misses
- Capacity misses

Why fully associative?

Victim cache is small (4-16 entries) and fully associative to maximize probability of hit and to feature a low overhead due to small size

Performance Impact

Benefits:

- ✓ reduces conflict misses
- ✓ small hardware cost
- ✓ no impact on L1 hit time

Costs:

- ✗ extra lookup
- ✗ slightly higher complexity

Most effective when:

- L1 is low associativity (direct mapped or few ways)
- Access patterns cause index conflicts
- Working set fits in L1 + victim cache

Historical and Practical Use

Used in:

- Early high-performance CPUs
- Some embedded systems
- Specialized architectures

Less common today because L1 associativity increased (4-8 ways)

Still very relevant conceptually.

Inclusive Cache Hierarchy

All data in L1 is also present in L2 (and L3 and the other levels)

- ✓ Easy coherence
- ✓ Simple eviction tracking
- ✗ Redundant data
- ✗ Wasted capacity

Exclusive Cache Hierarchy

Data exists only in one level

✓ Maximizes total capacity

✓ No duplication

✗ More complex

✗ Harder coherence

Aspect	Inclusive	Exclusive
Capacity	Lower	Higher
Complexity	Lower	Higher
Coherence	Easier	Harder

Private and Shared Caches

In multicore systems we can have private and shared caches.

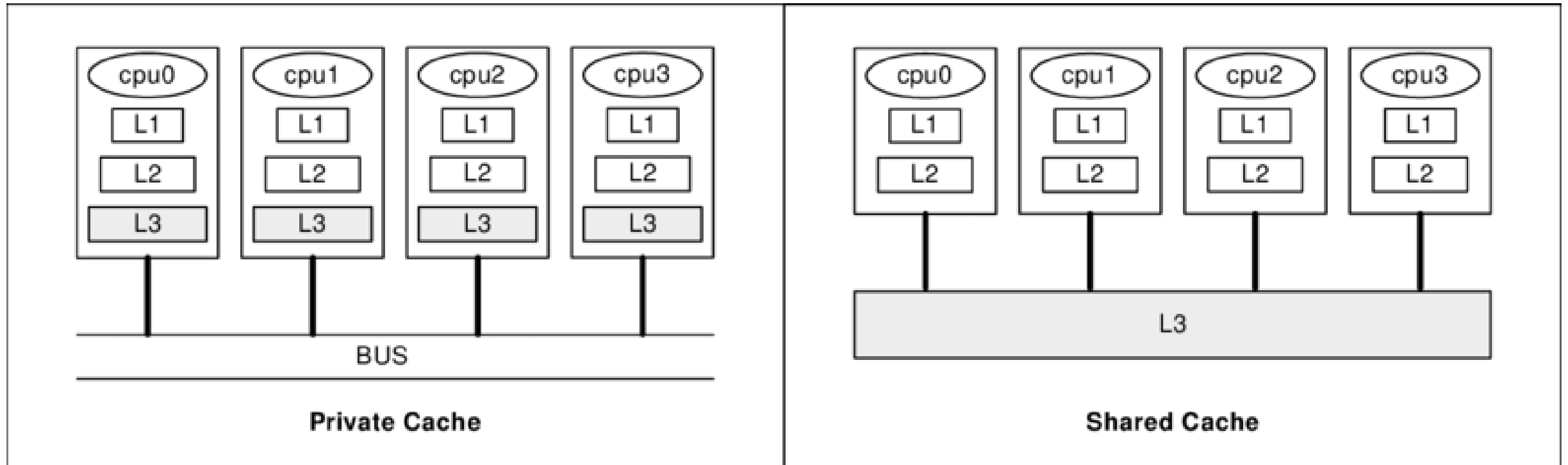
Private cache -> each core had its own cache

- ✓ low latency
- ✓ no contention
- ✗ duplication

Shared cache (LLC) -> each core accesses the same cache

- ✓ better utilization
- ✓ easier communication
- ✗ contention
- ✗ higher latency

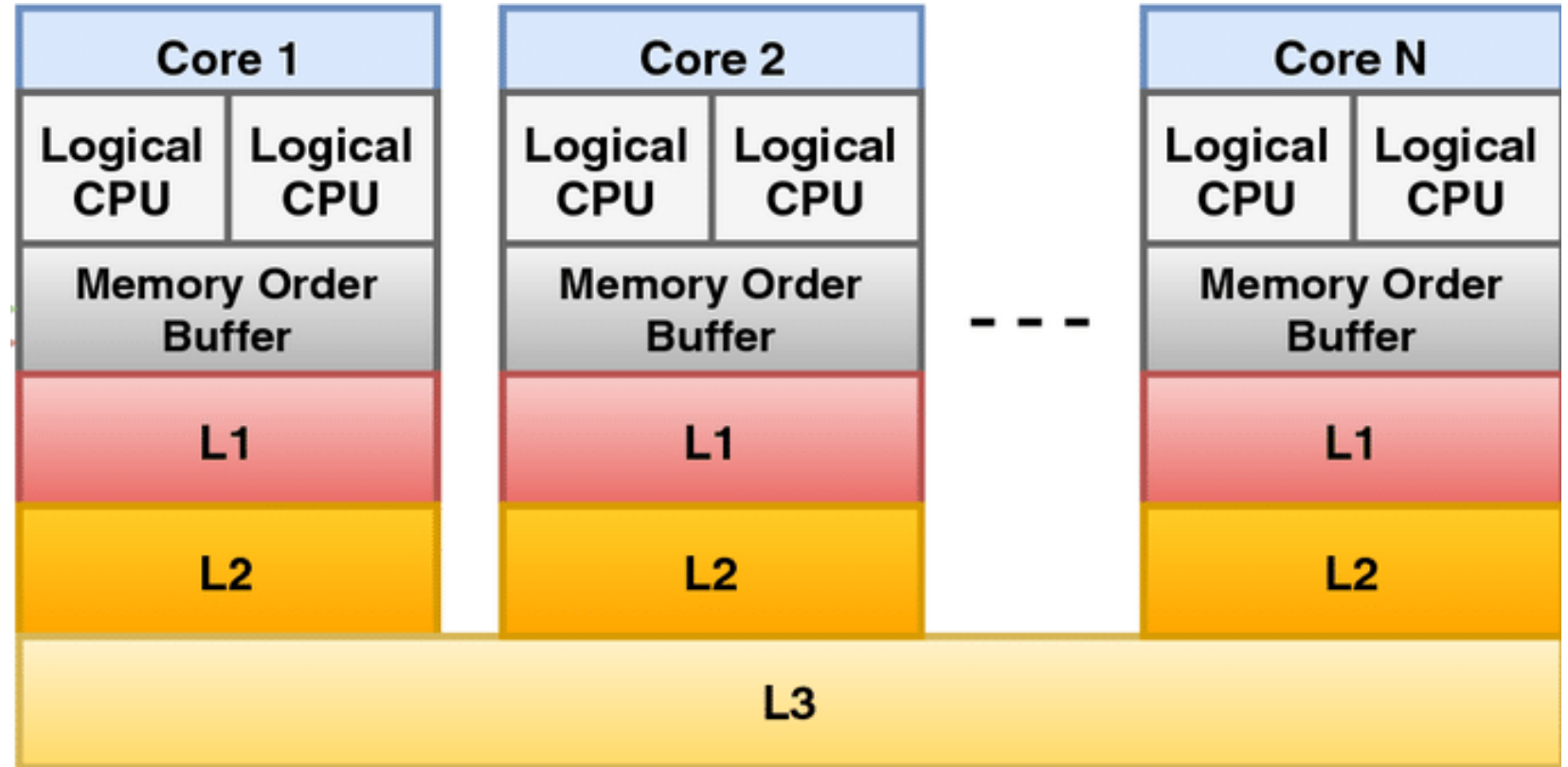
Private and Shared Caches



Modern CPU Design

Typical:

- L1 private
- L2 private
- L3 shared



Interconnection Matters

Caches are connected via:

- Ring bus
- Mesh
- Crossbar

Latency depends on physical distance

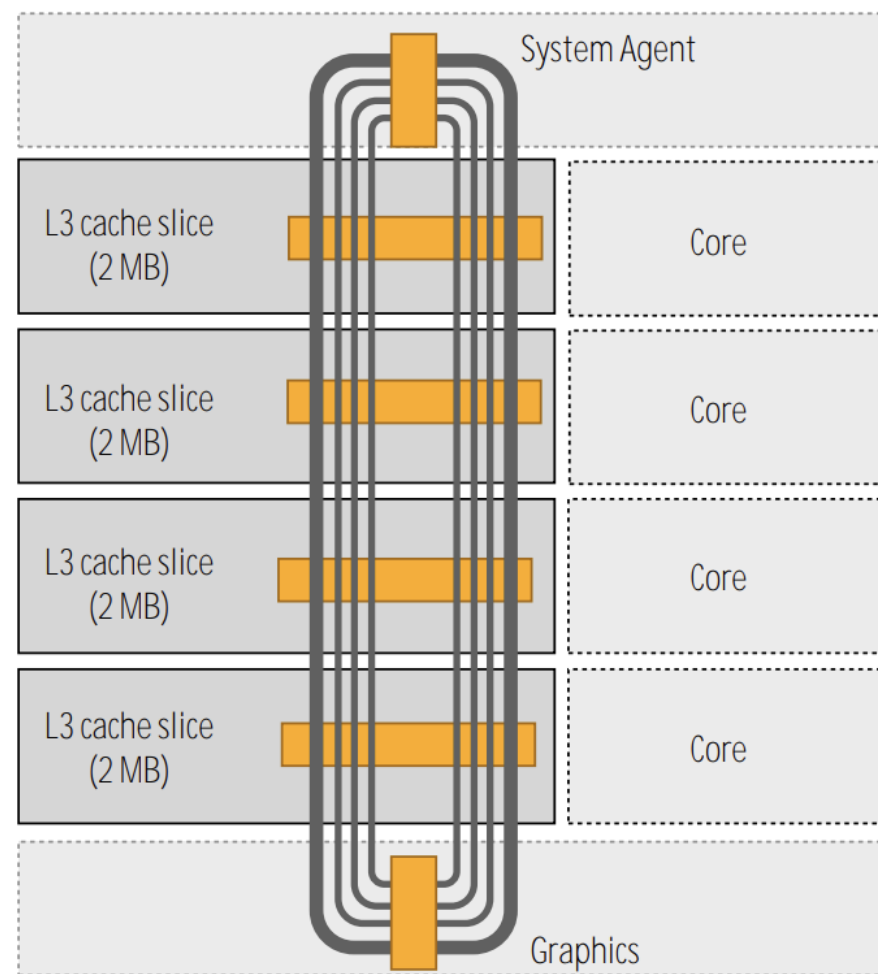
As cores increase:

- Coherence traffic increases
- LLC contention increases
- Latency variability increases

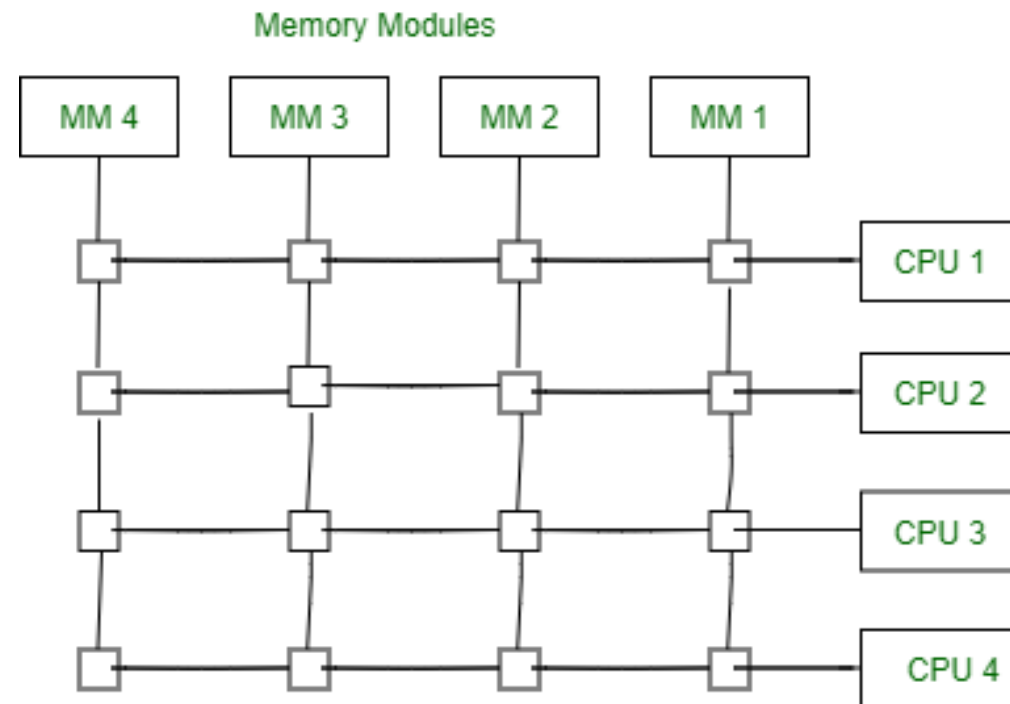
Memory becomes a shared bottleneck!

Interconnection Matters

Intel's Ring Interconnect

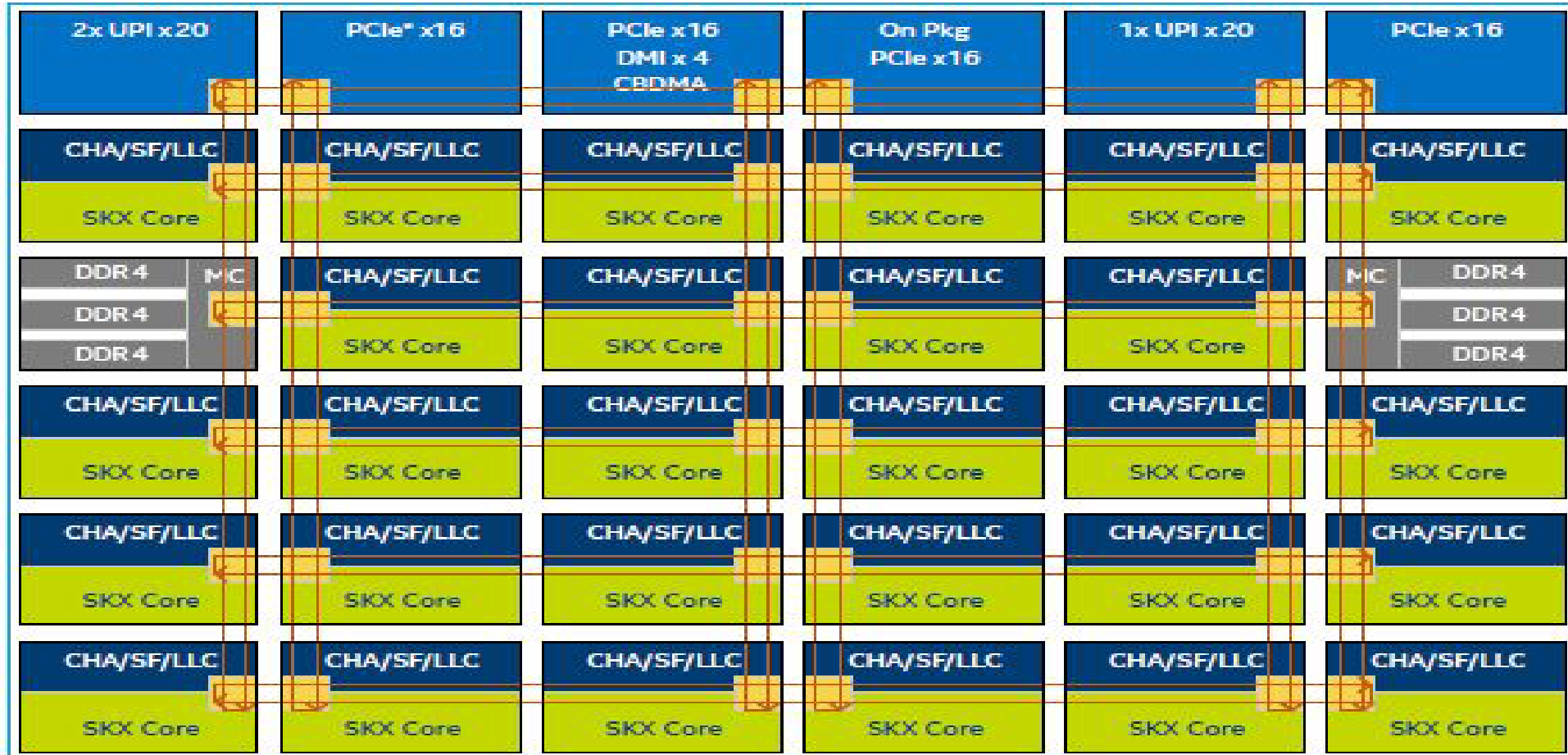


Crossbar Interconnect



Interconnection Matters

Intel's Mesh Interconnect



Engineering Trade-Offs

Design decisions:

- Cache sizes
- Associativity
- Number of levels
- Inclusive vs exclusive
- Shared vs private

No single optimal solution! It depends on the workload.

Datacenter CPU

- Large LLC
- Many cores
- High bandwidth

Mobile CPU

- Small caches
- Energy optimized

Closing

Key concepts:

- Multilevel AMAT
- Role of each cache level
- Inclusive vs exclusive
- Private vs shared caches
- System-level trade-offs

Next episode: cache coherence (why it is needed, MESI protocol and multicore behavior)

CACHE COHERENCE

Cache Coherence

Topics:

- Why coherence is required
- What goes wrong without it
- Hardware solutions
- MESI protocol
- Impact on system performance

From Single Core to Multicore

Previously:

- Single core -> one cache -> no problem

Now:

- Multiple cores each with private caches

Problem:

- Multiple copies of the same data

The Core Problem

Example:

Core 1: $X = 10$ (in cache)

Core 2: $X = 10$ (in cache)

Core 1 updates $X = 20$

What does Core 2 see?

Without coherence Core 1 sees $X = 20$ and Core 2 sees $X = 10$

Inconsistent system state that breaks program correctness

Definition of Cache Coherence

A system is coherent if:

1. A read returns the most recent write
2. Writes to the same location are seen in the same order by all the processors

Concept	Meaning
Coherence	Same value per location
Consistency	Ordering across locations

Two Possible Solutions

To maintain coherence::

1. Write-invalidate:
 - Invalidate other copies
2. Write-update:
 - Propagate updates

Modern systems use invalidate protocols

Snooping Protocols

Basic idea: all caches monitor a shared bus and observe other cores' memory operations

When a core writes the others react accordingly

Each cache line has:

- State (coherence state)
- Tag
- Data

Transition occur based on local actions (read/write) and bus actions (other cores)

Introducing MESI Protocol

The MESI invalidation protocol defines four states:

- M -> Modified
 - Only copy
 - Dirty (different from memory)
- E -> Exclusive
 - Only copy
 - Clean
- S -> Shared
 - Multiple Copies
 - Clean
- I -> Invalid
 - Not present

Each cache line is always in one of these states

State Transitions

Operations:

- Read hit
- Read miss
- Write hit
- Write miss

Plus:

- Bus read
- Bus write

Each triggers state changes

Example

Core 1 reads X

- Not in cache -> miss
- Fetch from memory

If no other core has it -> state = Exclusive

Core 2 reads X but Core 1 already has X

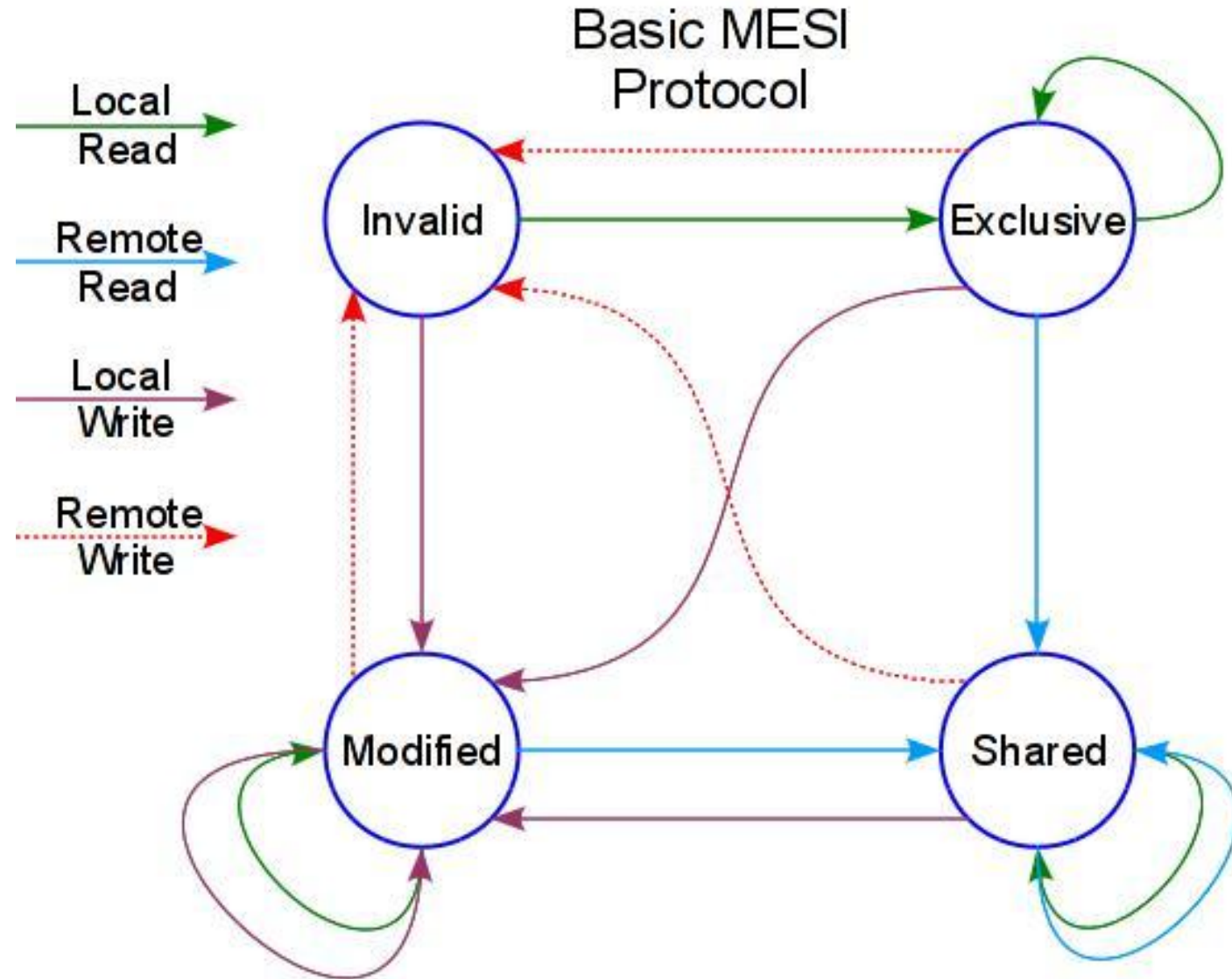
Result -> both cores state = Shared

Core 1 writes X (must invalidate other copies!)

Result -> Core 1 state = Modified and Core 2 state = Invalid

Core 1 writes X again -> No bus traffic needed, only local update

MESI State Diagram



Why invalidate Protocols Win

Write-updates have high bandwidth cost

Write invalidate:

- Fewer messages
- Scalable
- Simpler

Almost all modern CPUs use invalidate protocols

With respect to single core CPUs, coherence introduces extra traffic, invalidations, stalls and latency variability

False Sharing

Multiple cores access different variables that happen to reside in the same cache line and at least one core writes

Data are not logically shared, but the hardware sees them as shared

Reason -> cache works at cache line level, not at single variable level

Example:

- cache with 4 B blocks

- Variable A at address 0xABCD0000

- Variable B at address 0xABCD0001

The two variables are independent but mapped on the same cache line

False Sharing

Core 1 caches A

Core 2 caches B

both cores load a block X of 4 B containing both A and B
the state of the block X is shared

Core 1 writes A -> write hit in cache A

X is Modified in cache A

X is Invalid in cache B

Core 2 writes B -> since the block is invalid it caches the new data

X in is Invalid in cache A

X is Modified in cache B

False Sharing

What happens if Core 1 writes A again?

The result is that the operations of the previous slide are repeated

There is a ping-pong of the cache line between the two cores
continuous invalidation, high traffic and huge latency

It is defined “false” because there are not shared variables between the cores, but the hardware is not aware of this

False Sharing - Performance

False sharing impacts on performance with

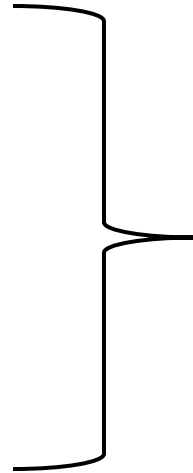
- Continuous invalidations
 - Each writes Invalidates the other cores
- Cache lines bouncing
 - A line continuously switches between the cores
- High coherence traffic
 - The communication bus could be saturated
- Bad scalability
 - Adding cores degrades performance

False Sharing – Example

Int counter[2];

Thread 1:
counter[0]++;

Thread 2:
counter[1]++;



counter1 and counter2 are in the same cache line

False Sharing - Performance

How to solve?

- Padding -> separate variables to use different cache lines

```
struct {  
    int value;  
    char padding[60];  
} counter1, counter2;
```

- Align to cache line
 __attribute__((aligned(64)))
- Data layout redesign

Engineering Impact

Coherence affects:

- Scalability
- Latency
- Bandwidth
- Power

In large systems coherence becomes dominant

Closing

Key concepts:

- Why coherence is necessary
- MESI protocol
- Invalidate mechanism
- False sharing

Next episode: memory consistency models (ordering constraints, TSO, weak models)

MEMORY CONSISTENCY MODELS

Memory Consistency Models

Topics:

- What consistency means
- Sequential consistency
- Reordering effects
- Weak models (TSO)
- Memory fences
- Programming implications

Recap: Coherence

Coherence ensures

- one value per memory location
- All cores eventually see the same value

Coherence guarantees correct value per variable, but not order of operations across variables!

Property	Guarantees
Coherence	Single variable correctness
Consistency	Global ordering

Consistency is about time/order

Example problem

Consider two variables $X = 0$ and $Y = 0$ and two cores

Core1:

$X = 1$

$r1 = Y$

Core 2:

$Y = 1$

$r2 = X$

Can we have $r1 = 0$ and $r2 = 0$?

Example problem

Sequential reasoning says it is impossible

However, it is possible in modern hardware because

- Processors reorder operations
- Writes are buffered
- Memory is not instantaneous

Sequential Consistency

Definition (Lamport): “The result is as if all operations were executed in some sequential order, and each processor’s operations appear in program order”.

The interpretation is that there are two constraints:

1. Program order preserved
2. Global interleave exists

Sequential Consistency Example

Program: C1: $X = 1 \rightarrow r1 = Y$
 C2: $Y = 1 \rightarrow r2 = X$

Under SC there are different allowed interleavings:

- C1 fully first
- C2 fully first
- Interleaved but respecting order

Sequential Consistency Example

Program: C1: $X = 1 \rightarrow r1 = Y$
 C2: $Y = 1 \rightarrow r2 = X$

Under SC there are different allowed interleavings:

- C1 fully first
- C2 fully first
- Interleaved but respecting order

Possible results:

- $r1 = 0, r2 = 1$
- $r2 = 0, r1 = 1$
- $r1 = 1, r2 = 1$

Why SC is Expensive

SC forbids:

- Instruction reordering
- Store buffering
- Many hardware optimizations

SC Kills performance!

Modern CPUs:

- Reorder instructions
- Use store buffers
- Execute out-of-order

Violates SC!

Stored Buffered Example

C1: $X = 1$ (goes to buffer)
 $r1 = Y$ (reads Y before write is visible)

C2: $Y = 1$ (goes to buffer)
 $r2 = X$ (reads X before write is visible)

Both cores read the old values

The result $r1 = 0$ and $r2 = 0$ is allowed under weak models

Hardware sees local writes first and global visibility later

Types of Reordering

Hardware may reorder:

- Load → Load
Two reads $r1 = X; r2 = Y;$
- Load → Store
Load and then write $r1 = X; Y = 1;$
- Store → Load
Store and the load $X = 1; r1 = Y;$
- Store → Store
Two stores $X = 1; Y = 1;$

Some models restrict some of these

Total Store Order (TSO)

Used in x86 architecture

Rules:

- Loads not reordered with other loads
- Stores not reordered with other stores
- Store → Load reordering allowed

Behavior:

- Writes go to store buffer
- Loads may bypass buffer

Total Store Order (TSO) - Example

Consider two cores, each one with a cache and a store buffer. At the beginning X and Y are initialized to 0

// Core 1

X = 1;

r1 = Y;

// Core 2

Y = 1;

r2 = X;

Total Store Order (TSO) - Example

Step 0:

X = 0

Y = 0

C1 buffer: empty

C2 buffer: empty

// Core 1

X = 1;

r1 = Y;

// Core 2

Y = 1;

r2 = X;

Step 1: C1 executes X = 1

C1 buffer: X = 1 (not visible to the other core)

Total Store Order (TSO) - Example

Step 2: C2 executes $Y = 1$

C1 buffer: $X = 1$

C2 buffer: $Y = 1$

```
// Core 1  
X = 1;  
r1 = Y;
```

Step 3: C1 executes $r1 = Y$

C1 reads Y from the memory

$Y = 0$ since C2 has only written in the buffer

```
// Core 2  
Y = 1;  
r2 = X;
```

Step 4: C2 executes $r2 = X$

C2 reads X from the memory

$X = 0$ since C1 has only written in the buffer

Memory Fences

To enforce order we use fence / barrier

```
X=1;  
mfence;  
r1=Y;
```

Fence ensures:

- All previous writes are visible
- No reordering across fence

Types of Fences

Full fence -> a type of CPU instruction that ensures all memory read and write operations initiated before the barrier are completed and visible to other processors before any memory operations initiated after the barrier are executed

Load fence -> a hardware instruction that ensures all load operations preceding the fence are completed and visible to the processor before any load operations following the fence

Store fence -> a hardware instruction that ensures all store operations preceding the fence are completed and visible to the processor before any store operations following the fence

Software Implications

Fences ensure correctness but reduce performance

Programmer must:

- Understand memory model
- Use synchronization primitives
- Avoid data races (two threads access same variable, at least one write and no synchronization)

Languages such as C++11 and Java define memory models which map to hardware model

Closing

Key concepts:

- Coherence $\neq$ consistency
- SC model
- Why SC is too strict
- TSO and reordering
- Memory fences

Next episode -> virtual memory

VIRTUAL MEMORY SYSTEMS

Virtual Memory Systems

Topics:

- Motivation for virtual memory
- Virtual vs physical addressing
- Paging
- Address translation
- Page tables
- TLB (Translation Lookaside Buffer)
- Page faults
- Replacement policies
- Thrashing
- NUMA and virtualization

Fundamental Problems

Modern systems execute:

- Multiple processes simultaneously
- Applications requiring large memory spaces
- Programs that must be isolated from each other

However

- Physical memory is limited
- Memory is fragmented
- Processes cannot directly access arbitrary physical addresses

Goals of Virtual Memory

Virtual memory provides

- Abstraction
- Protection
- Relocation
- Controlled sharing
- Larger logical address spaces

Key Idea of Virtual Memory

The main abstraction is that each process sees:

- Its own contiguous address space
- Independent from physical memory organization

The OS and HW cooperate to translate

Virtual Address to Physical Address

The program works with virtual addresses

The DRAM works with physical addresses

Translation is mandatory

Virtual vs Physical Address

Virtual Address (VA)	Physical Address (PA)
Generated by CPU	Used by memory hardware
Process-specific	Global system-wide
Abstraction	Actual DRAM location

Example: two processes may use the same virtual address:

Process A: VA 0x1000

Process B: VA 0x1000

but map to different physical frames.

Process Isolation

Without virtual memory:

- One process could overwrite another process memory
- Security would collapse
- Multitasking would be unsafe

With virtual memory:

- Each process has isolated mappings
- Illegal accesses generate exceptions

Page table entries contain permission bits:

- Read
- Write
- Execute

Paging

Concept: memory is divided into fixed-size blocks

Virtual pages are blocks of virtual address space

Physical frames are blocks of physical memory

Typical page sizes:

- 4 KB
- 2 MB (huge page)
- 1 GB (very large page)

Why Fixed-Size Pages?

Compared to variable-size allocation:

Advantages

- Simpler allocation
- Easier replacement
- Simpler hardware
- No external fragmentation

Disadvantages

- Internal fragmentation

Internal Fragmentation

Example: suppose

- page size = 4 KB
- Application requests 6 KB

Allocation requires 2 pages; thus, the total allocated memory is 8 KB

Unused memory:

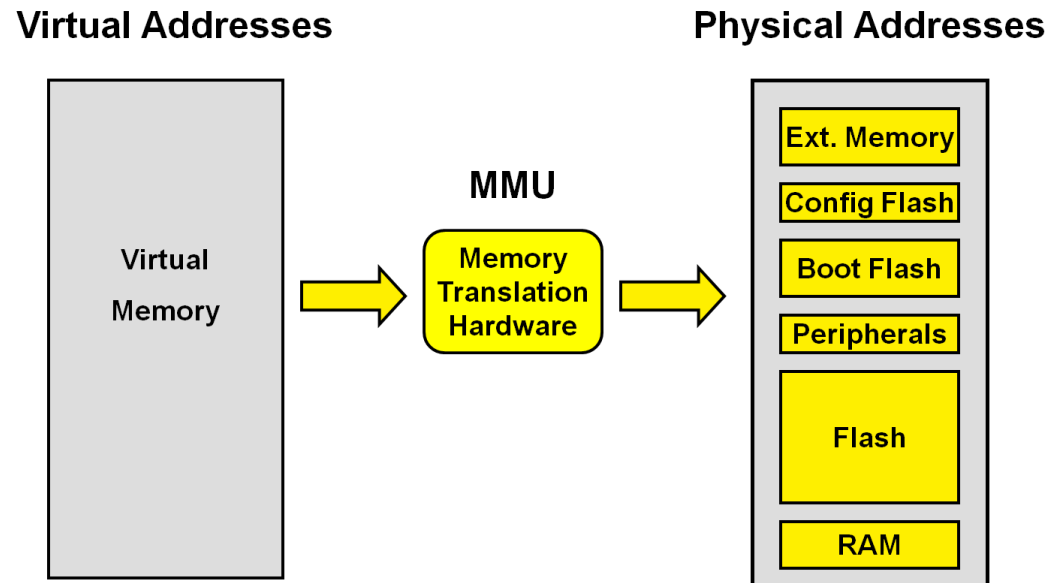
$$8\text{KB} - 6\text{KB} = 2\text{KB}$$

This unused space is called internal fragmentation

Address Translation

Translation mechanism:

- CPU generates a virtual address
- The MMU (Memory Management Unit):
 - Translates the address
 - Checks permissions
 - Produces the physical address



Structure of a Virtual Address

A virtual address is divided into:

- Virtual Page Number (VPN)
- Offset (byte inside page)

The offset does not change during translation, only the page number changes

Example: consider a 32-bit virtual address and a page size of 4KB

Since $4\text{KB} = 2^{12}$ the offset is 12 bits. The remaining bits are $32 - 12 = 20$

The VPN is 20 bits and the offset is 12 bits

Translation Procedure

The translation procedure

1. Extract offset
2. Extract VPN
3. Use VPN to index page table
4. Obtain PFN
5. Combine PFN and offset

Page Tables

The main idea is that page tables store mappings:

VPN $\rightarrow$ PFN

Each entry is called Page Table Entry (PTE)

PTE contains:

- Physical frame number
- Valid bit
- Dirty bit
- Access permission
- Accessed/reference bit

Single-Level Page Table

Simplest organization

One large table indexed directly by the VPN

Problem: large virtual address spaces require enormous tables

Example: consider 32-bit VA and 4KB pages

The number of pages is $2^{32}/2^{12}=2^{20}$

Page Table Size Example

Suppose:

- 1 million entries
- Each PTE is 4 bytes

The total size is $2^{20} \times 4\text{B} = 4\text{ MB}$ per process

This is a problem in modern systems since they may execute

- Hundreds of processes
- Sparse address spaces

Huge memory overhead

Multi-Level Page Tables

Solution

Use hierarchical page tables

Only allocate portions actually used

Typical structure:

- Directory
- Table
- Offset

Modern x86-64 systems use 4-level paging (sometimes 5-level paging)

Multi-Level Translation

Translation Procedure:

1. Use top-level index
2. Find next-level table
3. Repeat until leaf entry
4. Obtain PFN

Benefit

Sparse allocation

Only required page tables are allocated

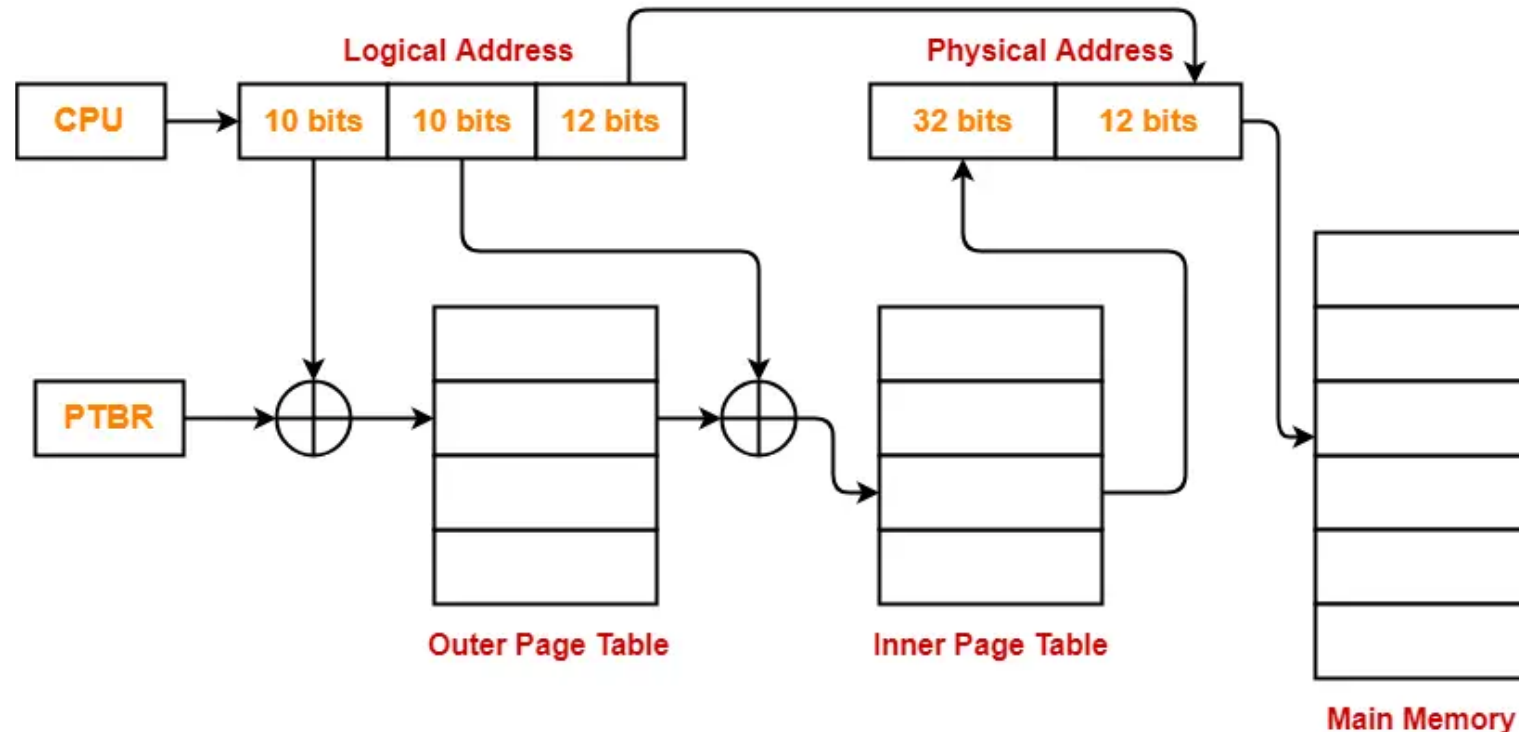
Visualization of Multi-Level Paging

Key idea:

Virtual address bits are used progressively

Each level indexes another table

This creates a hierarchical tree structure



Why Translation is Expensive

Without optimization:

Every memory access requires

1. Page table access
2. Actual memory access

Potentially:

$2 \times$ memory latency

Or worse in multi-level paging

Translation Lookaside Buffer (TLB)

TLB definition: a small associative cache storing recent translations

It stores VPN $\rightarrow$ PFN

The goal is to avoid repeated page table walks

TLB hit: translation found immediately, fast access

TLB Miss: hardware or OS perform page table walk, much slower

Effective Memory Access Time

EMAT is defined as

$$EMAT = TLB_{hit\ time} + MR_{TLB} \times Penalty$$

Where MR is the miss rate and $Penalty$ is the page walk cost

Example: $TLB_{hit\ time} = 1$ cycle, $MR_{TLB} = 1\%$ and $Penalty = 100$ cycles

$$EMAT = 1 + 0.01 \times 100 = 2$$

TLB Reach

Definition: Amount of memory addressable by TLB without misses

$$TLB\ Reach = Entries \times Page\ Size$$

Example: with 4 KB and 512-entry TLB

$$TLB\ Reach = 512 \times 4KB = 2MB$$

Huge Pages

Motivation: increase TLB reach

Benefits:

- Fewer TLB misses
- Smaller page tables

Costs:

- Internal fragmentation
- Allocation complexity

TLB and Multicore Systems

Problem: if page mappings change the TLB entries in multiple cores may become invalid

Solution: TLB shutdown

- OS sends invalidation requests to cores

It is expensive because it requires:

- Inter-processor interrupts
- Synchronization
- Invalidation coordination

Can become a scalability bottleneck

Page Faults

Definition: a page fault occurs when the requested page is not currently mapped in physical memory

The CPU traps into the operating system

OS actions:

1. Identify missing page
2. Locate page on disk
3. Allocate frame
4. Load page
5. Update page table
6. Resume execution

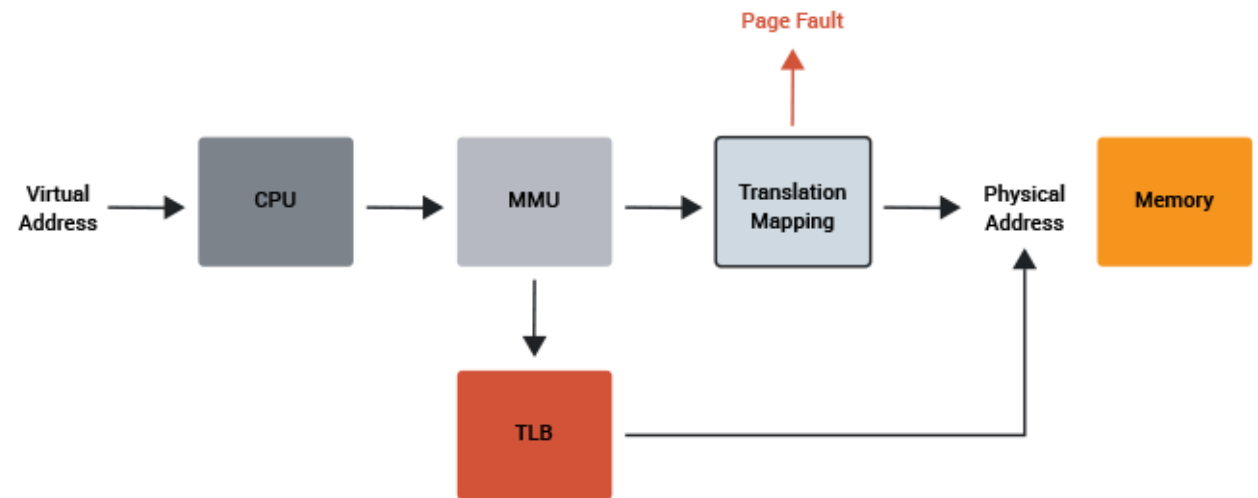
Major vs Minority

Minor fault

- Page is already available in RAM
- No disk access

Major fault

- Disk access required
- Very expensive



Demand Paging

Main idea: page are loaded only when actually accessed

Advantages:

- Lower memory usage
- Faster program startup

Drawback:

- Initial page faults

Page Replacement

Problem: Physical memory is finite and when memory is full the OS must evict pages

Common replacement policies:

- FIFO
- LRU
- Clock
- Random

Goal: minimize future page faults

Page Replacement

FIFO: evict oldest page

Advantages

- Simple
- Low overhead

Problem

- May evict frequently used pages

Last Recently Used (LRU): evict page not used recently. It is based on the temporal locality principle (recently used pages likely reused soon)

Problem: Tracking exact access order is expensive (real systems often use approximations, reference bits, aging techniques)

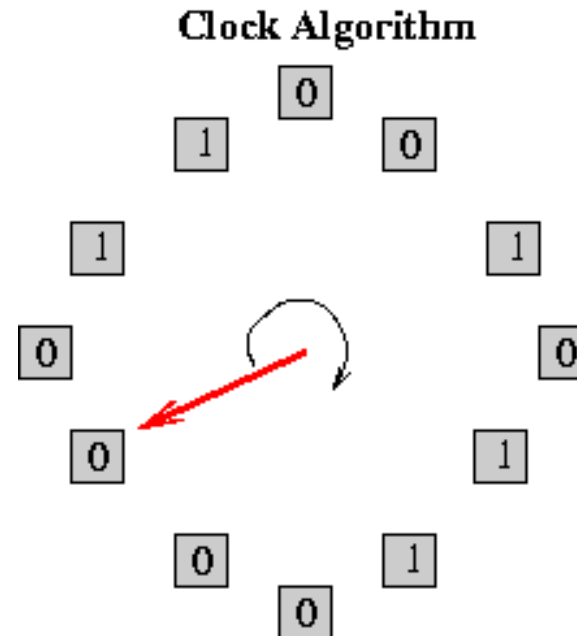
Page Replacement

Clock policy: pages arranged circularly. Reach page has a reference bit.

Algorithm:

- If bit = 0 → evict
- If bit = 1 → clear bit and continue

Cheap approximation of LRU



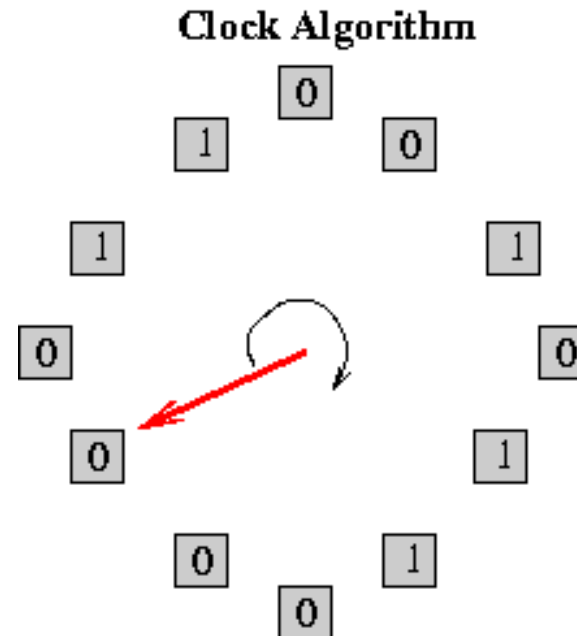
Page Replacement

Clock policy: pages arranged circularly. Reach page has a reference bit.

Algorithm:

- If bit = 0 $\rightarrow$ evict
- If bit = 1 $\rightarrow$ clear bit and continue

Cheap approximation of LRU



Thrashing

Definition: system spends more time paging than computing

Symptoms:

- Massive page faults
- Low CPU utilization
- High disk activity

Main cause:

- Working set exceeds physical memory
- System continuously swaps pages

Working Set Model

The working set is the set of actively used pages

If working set fits in memory → low page fault rate
Otherwise → thrashing likely

Virtual memory interacts with:

- L1/L2/L3 caches
- TLB
- Coherence
- Memory controllers

NUMA systems

Non-Uniform Memory Access

Modern systems have:

- Multiple sockets
- Multiple memory controllers
- Local and remote memory

Latency depends on memory location

Remote accesses feature higher latency and lower bandwidth

OS and applications must preserve locality

Virtualization

Virtual machines introduce:

- Guest virtual addresses
- Guest physical addresses
- Host physical addresses

Additional translation layer required

Two level translation

- Guest OS performs one translation
- Hypervisor performs another
- Potentially large overhead

Modern CPUs provide hardware acceleration

Modern Perspective

Virtual memory is not isolated

It interacts with:

- OS
- Compilers
- Caches
- TLBs
- Multicore systems
- Virtualization

Summary

Virtual memory is

- A protection mechanism
- An abstraction mechanism
- A performance problem
- A HW/SW co-design challenge

It is one of the most important abstractions in modern computing systems.

Next episode: DRAM architecture and memory controllers

DRAM ARCHITECTURE AND MEMORY CONTROLLERS

DRAM Architecture and Memory Controllers

Topics:

- Why DRAM matters
- DRAM cell organization
- Banks, rows and columns
- DRAM timing parameters
- Row buffer locality
- DRAM commands
- Memory controllers
- Scheduling policies
- Bandwidth vs latency
- Modern DRAM technologies
- DDR, LPDDR, HBM

Motivation

Modern processors are often limited by:

- Memory latency
- Memory bandwidth
- Memory contention

Even with large caches many accesses eventually reach DRAM

Thus DRAM architecture strongly affects system performance

Memory Hierarchy Recap

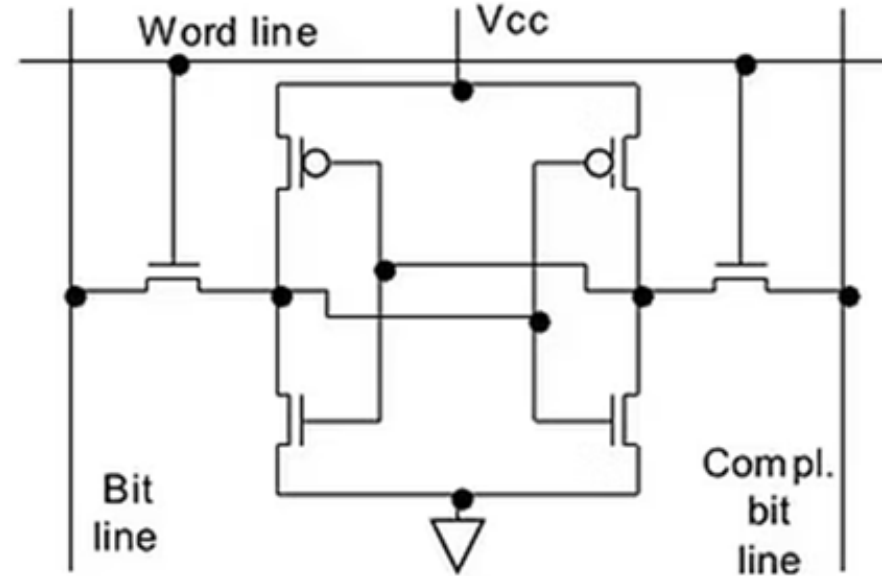
Level	Typical latency
Registers	~1 cycle
L1 cache	~1-4 cycles
L2 cache	~10 cycles
L3 cache	~30-50 cycles
DRAM	~100-300 cycles

DRAM is order of magnitude slower than caches!

DRAM vs SRAM

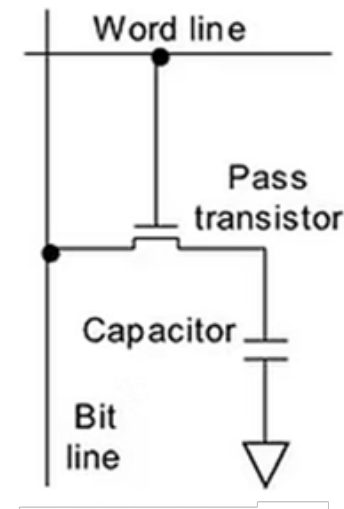
SRAM (used in cache)

- Fast
- Expensive
- Low density
- Uses flip-flops



DRAM

- Slower
- Very dense
- Cheap per bit
- Uses capacitors



DRAM Cell

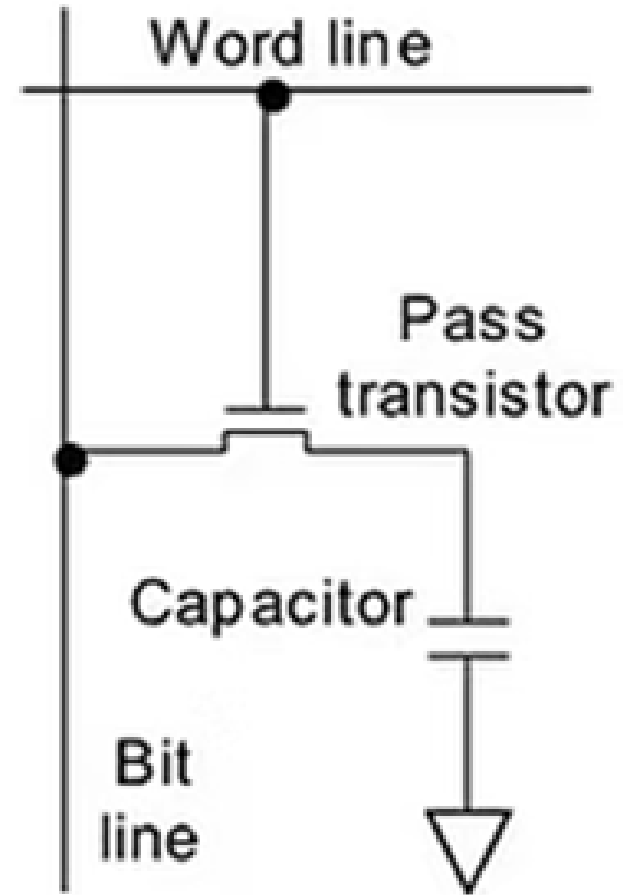
DRAM Storage

Each DRAM cell contains

- One transistor
- One capacitor

The capacitor stores charge representing

- 1 = charge
- 0 = discharged

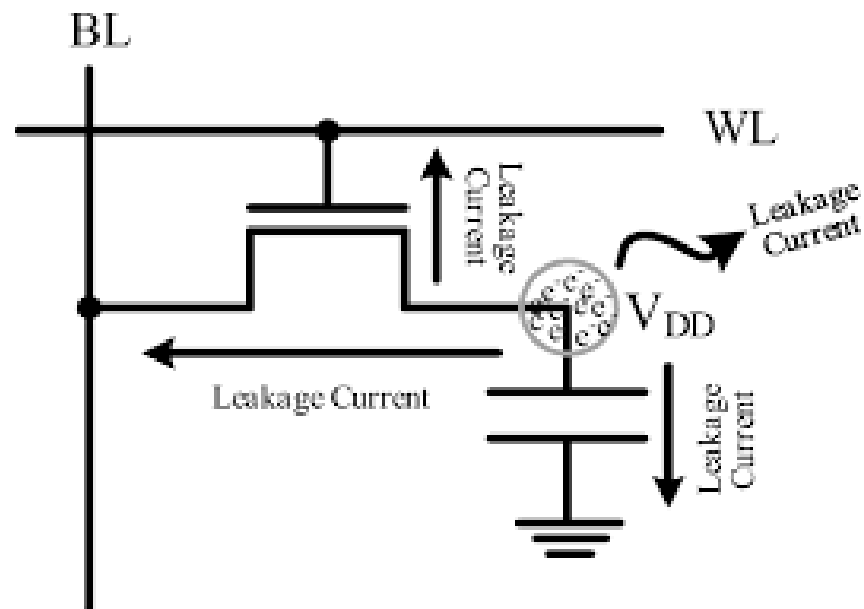


Problem: Charge Leakage

DRAM capacitors lose charge over time

Therefore data must be periodically refreshed

Typical refresh interval: 64 ms



Consequence of Refresh

Refresh operations:

- Consume bandwidth
- Increase latency
- Temporarily block accesses

Refresh becomes a majority issue in

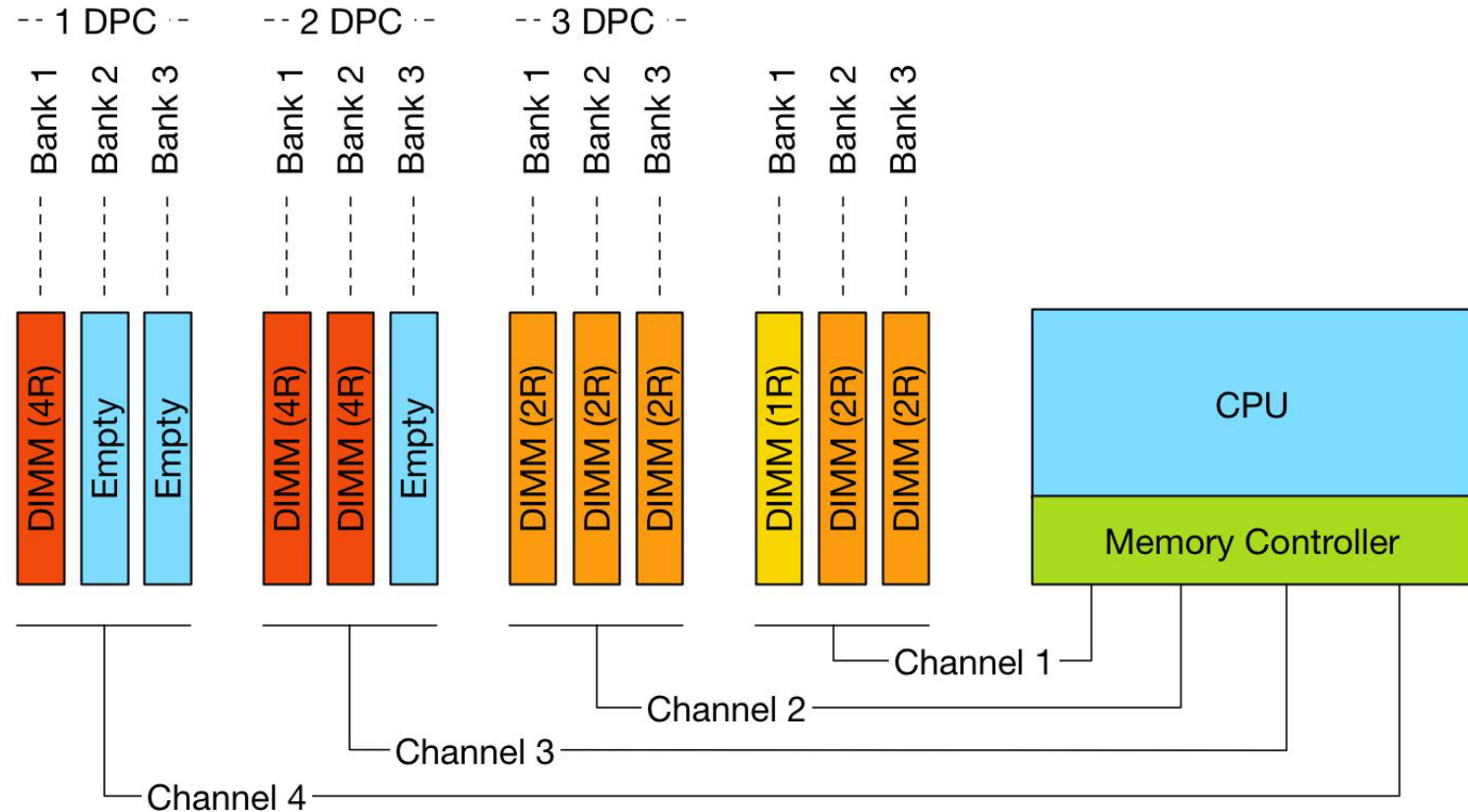
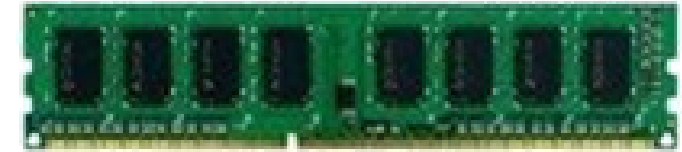
- Large memories
- High-density DRAM

DRAM Organization

DRAM is hierarchical
Structure:

- Channel
- DIMM
- Rank
- Chip
- Bank
- Row
- Column

DIMM (Dual in-line memory module)



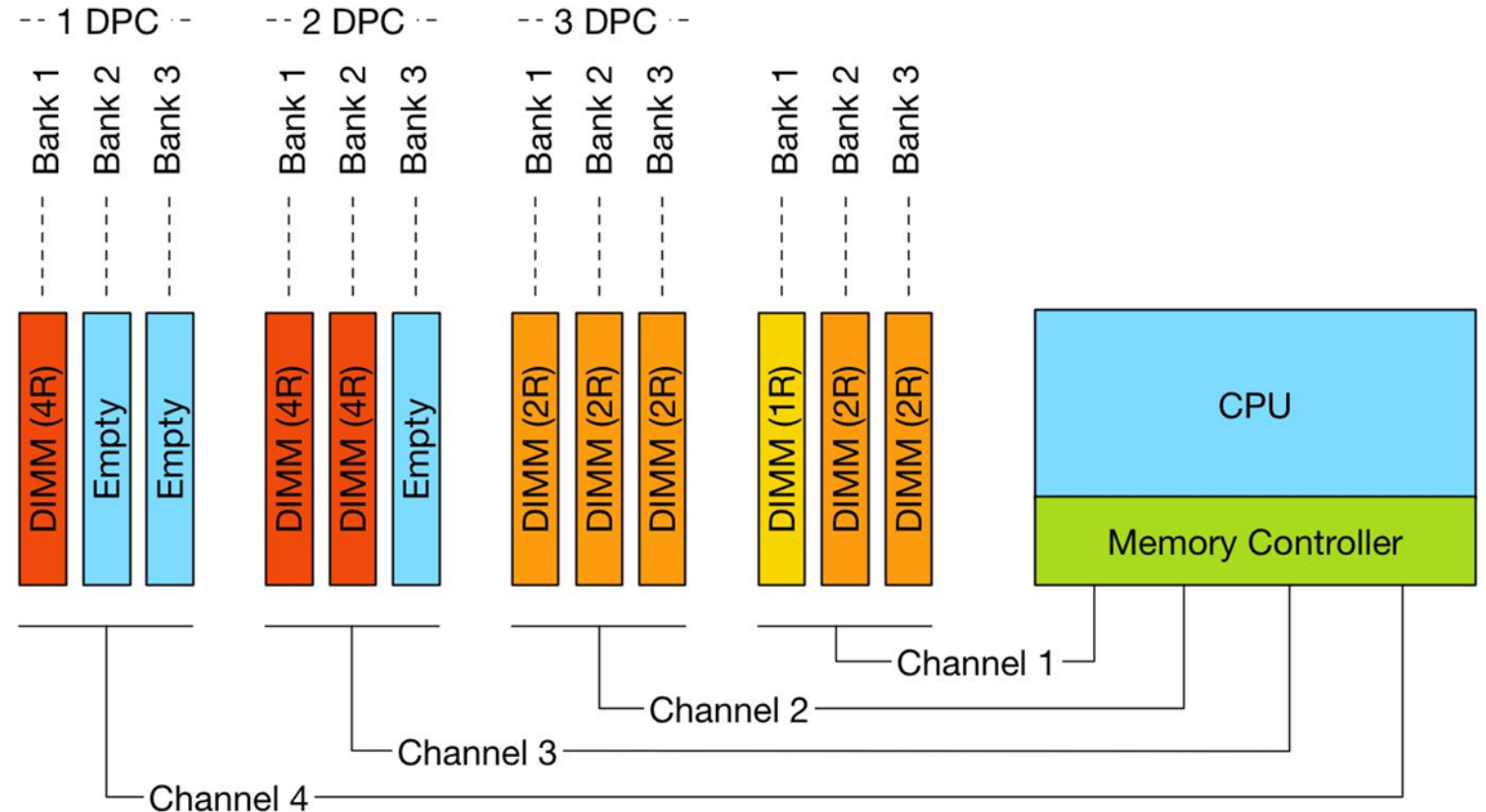
Channels

A channel is an independent communication path between:

- Memory controller
- DRAM modules

More channels:

- Higher bandwidth
- Parallel accesses



DIMMs and Ranks

DIMM is a physical memory module

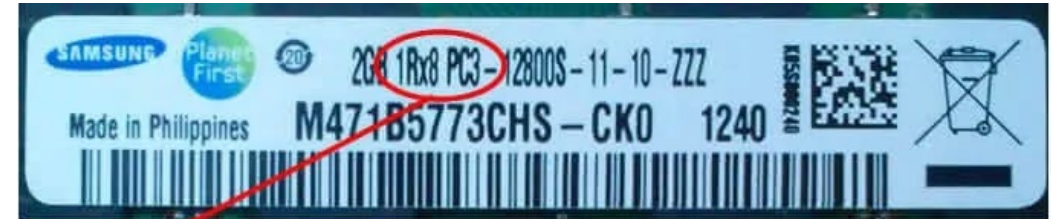
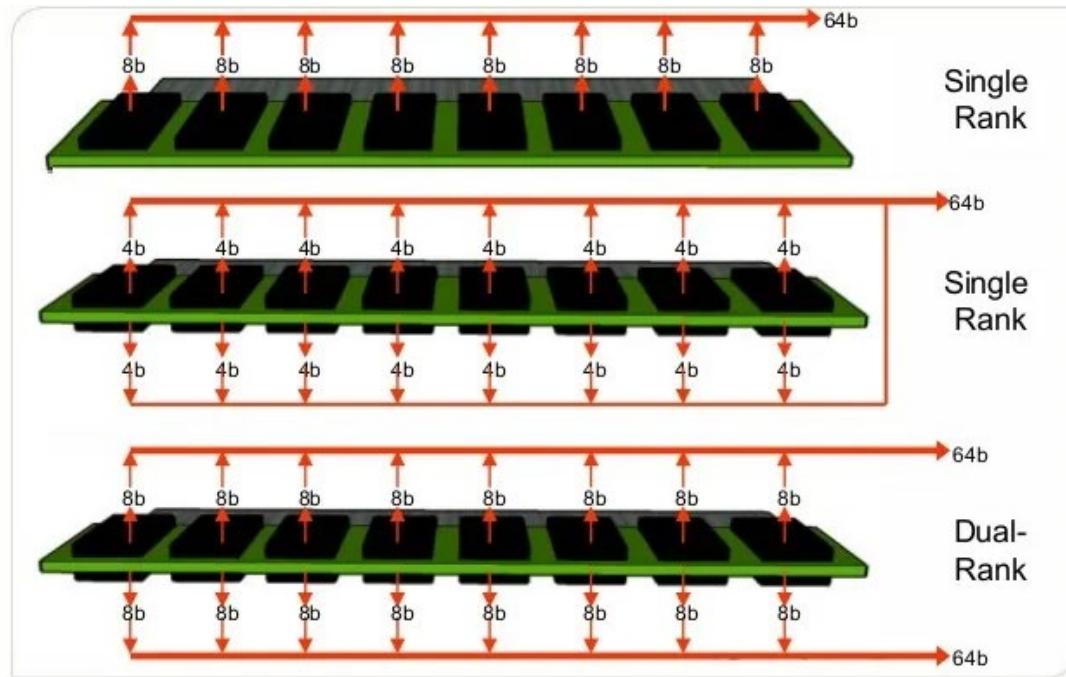
Rank is a group of DRAM chips operating together

Ranks share:

- Command bus
- Address bus

But have separate chip-select signals

DIMMs and Ranks



1Rx8 Single Rank



2Rx8
Dual
Rank



4Rx4 Quad Rank

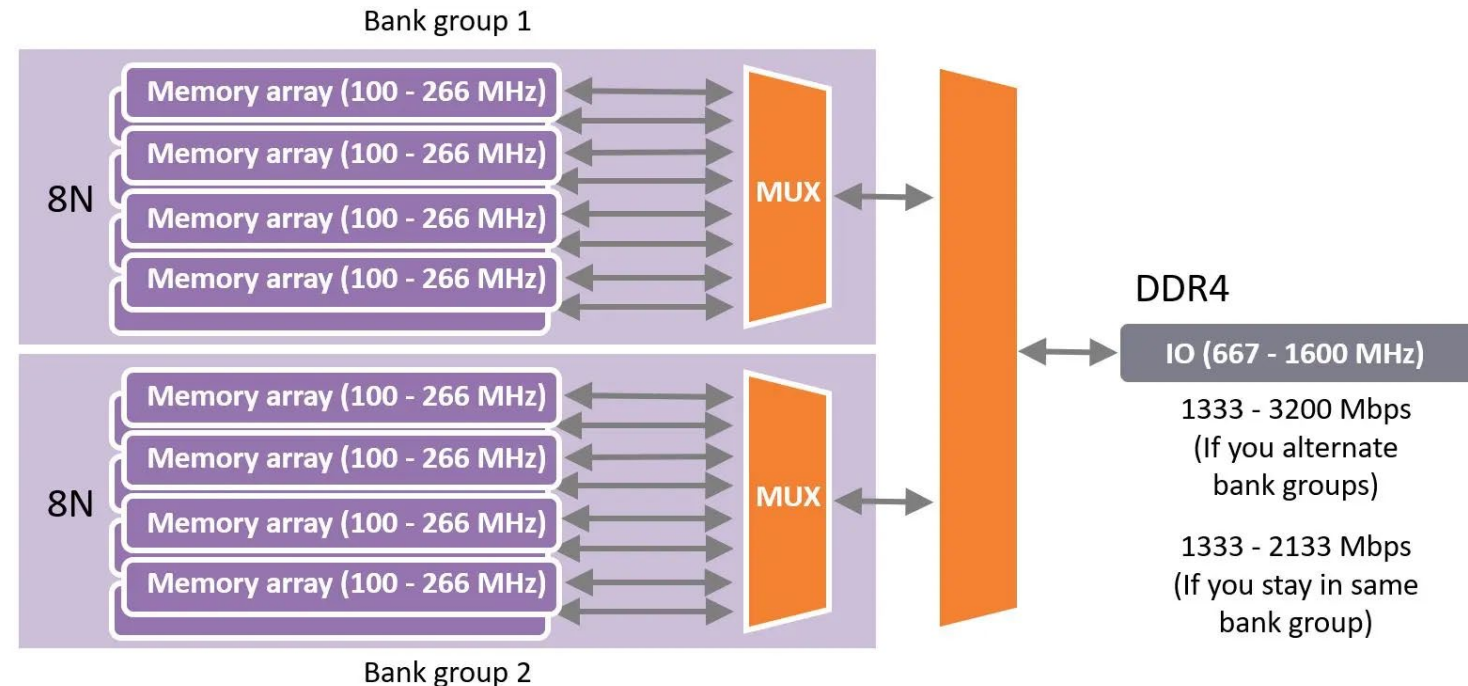
Banks

Each bank behaves approximatively like an independent memory array

Modern DRAM may contain:

- 8 banks
- 16 banks
- Bank groups

Banks allow parallelism



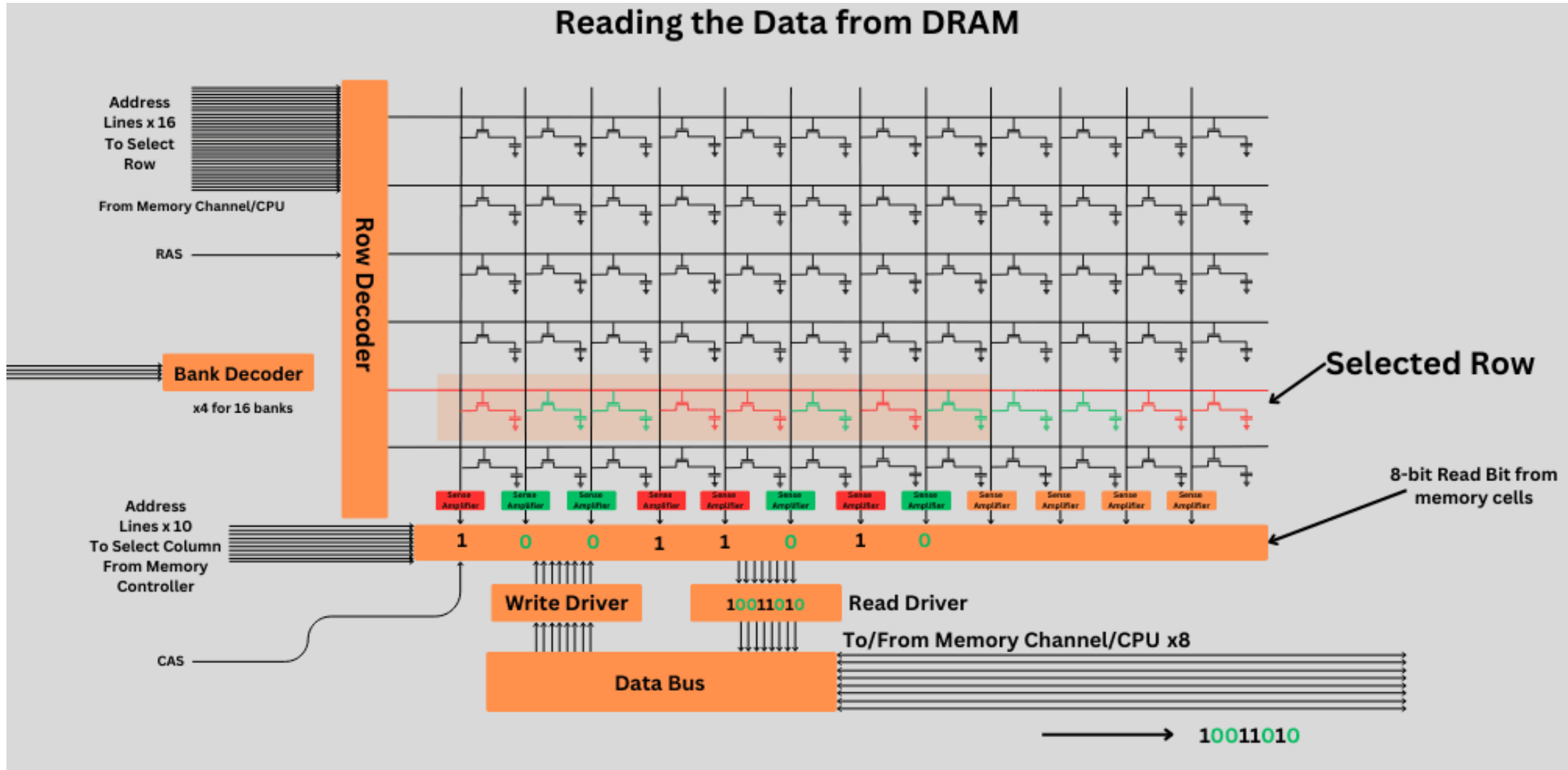
Rows and Columns

Inside each bank data are organized in rows and columns

Typical row size is several KB

It is important that the entire row is activated at once

Rows and Columns



Row buffer

Hey concept -> when a row is activated it is copied into the row buffer

The row buffer behaves similarly to a cache

If next access targets same row we have a row hit and a fast access

If access targets different row, the current row is closed and a new row is activated

Higher latency

DRAM Commands

Main DRAM commands:

- ACTIVATE -> opens a row
 - Entire row loaded into row buffer. Costly operation
- READ -> selects columns from row buffer
 - Access is relatively fast because row already active
- WRITE -> writes data into DRAM
- PRECHARGE -> before operating another row, the current row must be closed
- REFRESH -> refresh the DRAM cells

DRAM Access Sequence

Typical access sequence:

PRECHARGE
ACTIVATE
READ/WRITE
PRECHARGE

Most latency comes from row management

DRAM Timing Parameters

Important timings:

- t_{RCD} -> row to column delay
 - Time between ACTIVATE and READ/WRITE
- t_{CAS} -> column access strobe latency
 - Time between READ command and data availability
- t_{RP} -> row precharge time
 - Time required to close a row before opening another
- t_{RAS} -> row active time
 - Minimum time a row must remain active

These define minimum delays between commands

Full Timing Example

Example timing

ACTIVATE	-> wait t_{RCD}
READ	-> wait t_{CAS}
PRECHARGE	-> wait t_{RP}

Total latency accumulates quickly

Row Buffer Hit vs Miss

Row Buffer Hit

Same row already open

Fast

Row Buffer Miss

Different row needed

Must:

- precharge
- activate new row

Much slower

Row Buffer Conflict

Worst case -> repeated accesses alternate between rows

Example:

Row A -> Row B -> Row A -> Row B

Causes repeated:

- Precharge
- Activate

Large latency increase

Memory Controllers

Hardware unit managing DRAM accesses

Responsibilities

- Scheduling
- Command generation
- Refresh handling
- Arbitration

Memory Controller Queues and Scheduling Problem

Request arrive from:

- Cores
- DMA
- I/O devices

Controller maintains request queues

Controller must decide which request to serve next

Objectives:

- Maximize throughput
- Minimize latency
- Preserve fairness

FCFS Scheduling

First-Come First Served

Serve oldest request first

Advantages:

- Simple
- Fair

Disadvantages

- Ignores row locality

FR-FCFS Scheduling

First Ready First-Come First Served

Prioritize:

1. Row buffer hits
2. Oldest requests

Widely used

Works because row hits avoid activate and precharge
Higher bandwidth and lower average latency

Downside of FR-FCFS Scheduling

May starve some request

Especially requests targeting closed rows

Trade-off:

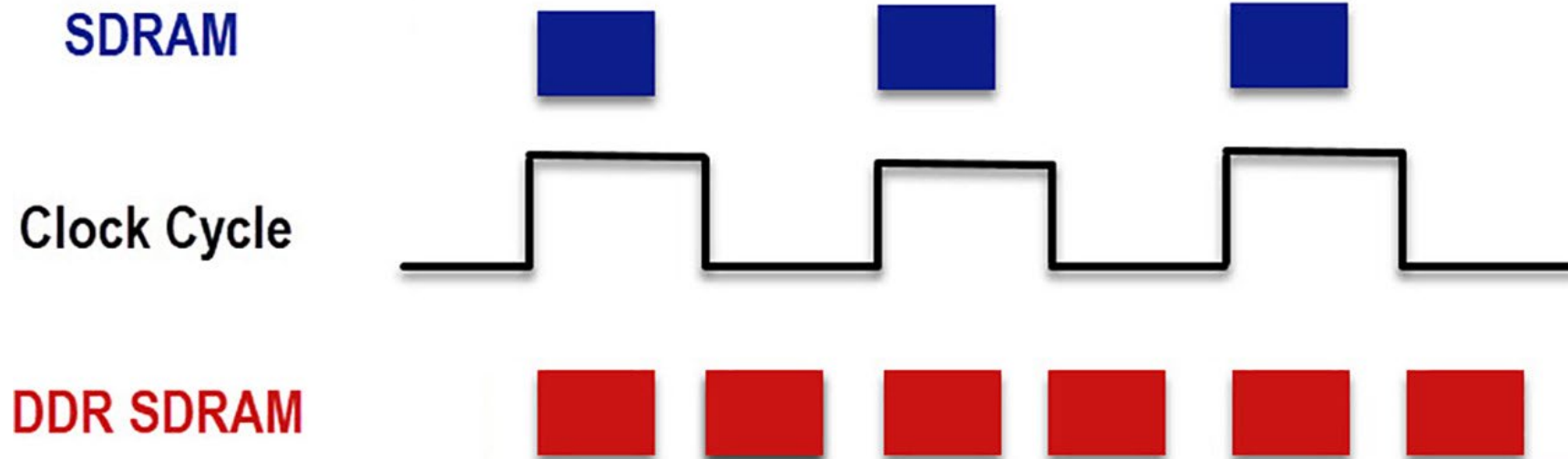
- Performance
- Fairness

DDR SDRAM

Double Data Rate

Transfers data on rising edge and falling edge

Effectively doubles bandwidth



DDR Generations

Technology	Bandwidth Increase
DDR	Baseline
DDR2	Higher frequency
DDR3	Improved efficiency
DDR4	Bank groups
DDR5	More parallelism

Low Power DDR

LPDDR are designed for mobile devices and energy efficiency

Optimizations are lower voltage and deep power states



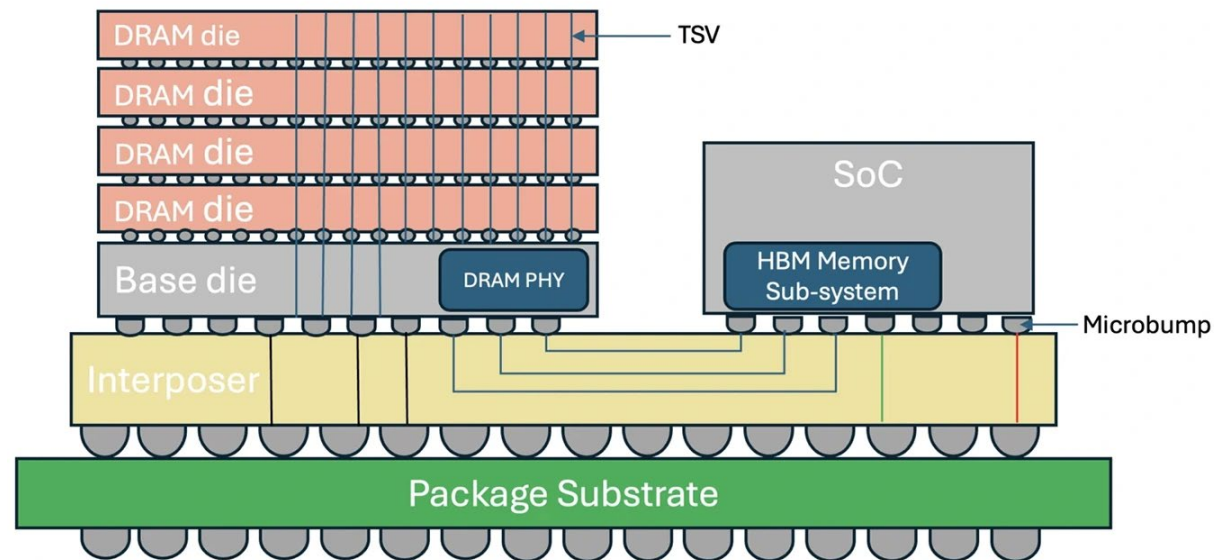
High Bandwidth Memory

HBM are made up by 3D-stacked DRAM

Uses wide interfaces and TSVs (Through-Silicon Vias)

Provides enormous bandwidth and low energy per bit

Used in GPUs, AI accelerators



Closing

Key concepts:

- DRAM organization
- Banks and row buffers
- Timing parameters
- Memory controllers
- Scheduling policies
- DDR technologies
- HBM

MEMORY PERFORMANCE MODELING AND PARALLEL MEMORY SYSTEMS

Memory Performance Modeling

Topics:

- The Memory Wall
- Latency vs Bandwidth
- Performance Modeling
- Roofline Model
- Memory-Level Parallelism (MLP)
- Non-Blocking Caches and MSHRs
- Prefetching
- NUMA Architectures
- Multicore Memory Scaling
- Real-World
- Case Studies

Motivation

So far we studied cache hierarchies, coherence, consistency, virtual memory and DRAM

However, understanding individual components does not automatically explain system performance.

This lesson focuses on **System-Level Behavior**

How memory subsystems affect:

- execution time
- scalability
- throughput
- energy consumption

Memory Wall Recap

Since the 1980s processor performance improved much faster than memory latency.

Year	CPU performance	DRAM latency
1980	baseline	Baseline
2000	x1000	X10
Today	Much higher	Modest improvement

Processors spend increasing amounts of time waiting for memory.

This phenomenon is called the Memory Wall

Why the Memory Wall Exists

Improvements in CPUs come from:

- deeper pipelines
- superscalar execution
- out-of-order execution
- higher frequencies

DRAM improvements are limited by:

- capacitor physics
- signal integrity
- power constraints

CPU Performance Equation

Recall:

$$CPU\ time = IC \times CPI \times Clock\ Cycle$$

Where IC is the instruction count and CPI are the cycles per instruction

In a real system the CPI includes also the memory contribution

$$CPI = CPI_{base} + Memory\ Stall\ Cycles$$

Example

Suppose $CPI_{base} = 1$ and *Memory Stall Cycles* = 4

The total CPI is 5

Interpretation:

- only 20% of execution is useful work
- the remaining 80% is waiting for memory

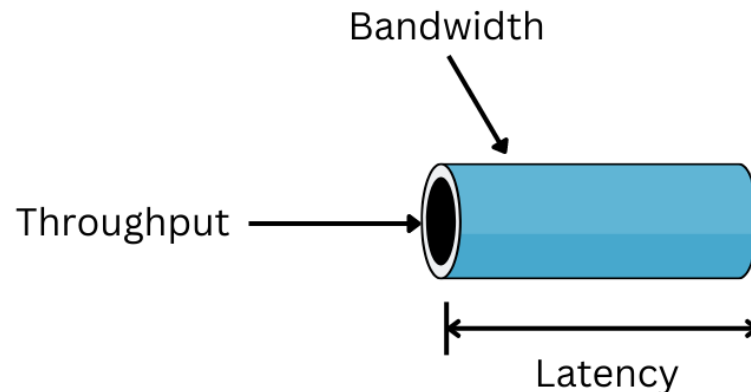
Latency vs Bandwidth

Latency is the time required to complete one request

example: latency of 100 ns

Bandwidth is the amount of data transferred per second

example: bandwidth of 50 GB/s



Why Both Matter

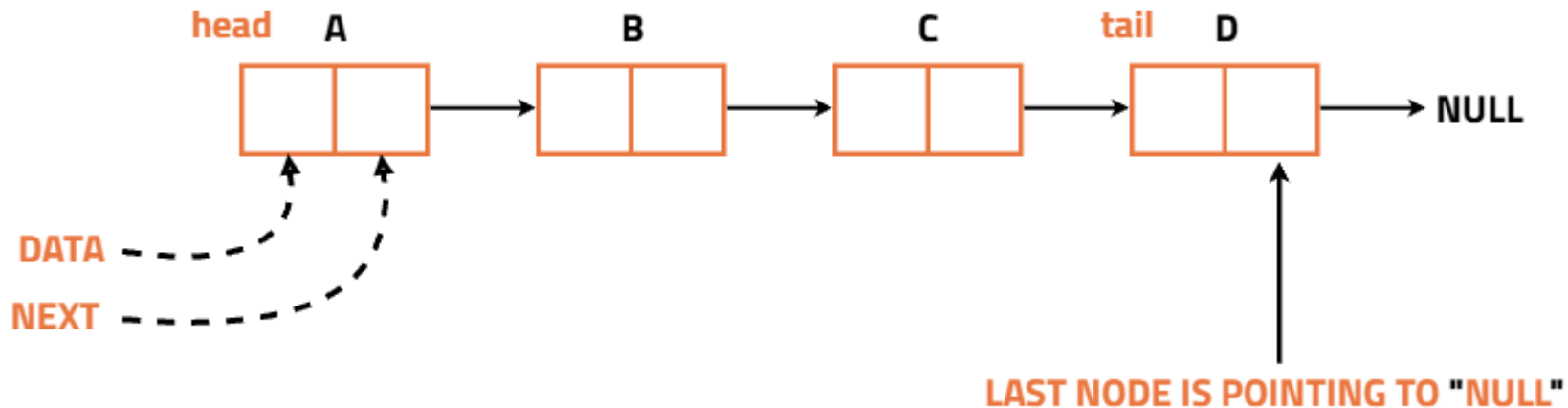
Example: linked list traversal

```
p = head;
```

```
while(p)
```

```
    p = p->next;
```

Performance depends mainly on latency

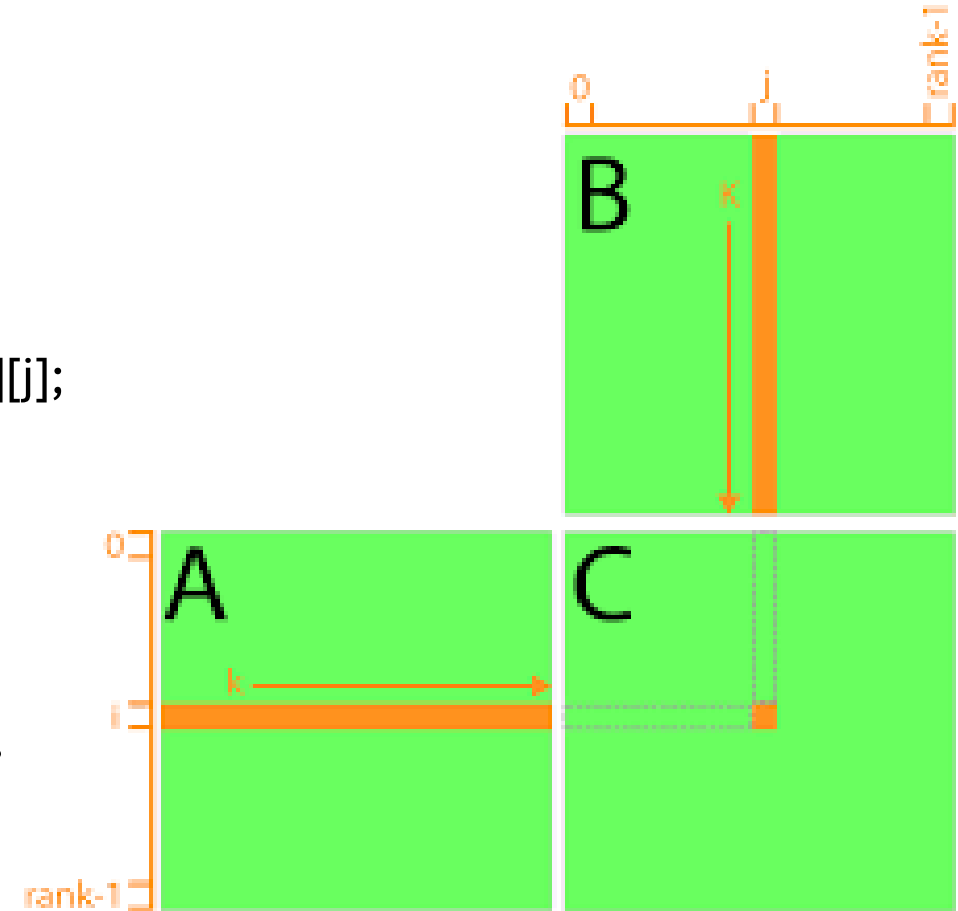


Why Both Matter

Example: matrix multiplication

```
for (int i = 0; i < R1; i++) {  
    for (int j = 0; j < C2; j++) {  
        result[i][j] = 0;  
        for (int k = 0; k < R2; k++) {  
            result[i][j] += m1[i][k] * m2[k][j];  
        }  
    }  
}
```

Performance depends mainly on bandwidth.



Memory Bandwidth

$$\textit{Bandwidth} = \textit{BusWidth} \times \textit{Transfer Rate}$$

Example: consider a memory with a 64-b bus and a transfer rate of 6400 MT/s

The Bus Width is 64-b = 8B

The bandwidth is $8 \times 6.4 \times 10^9 = 51.2$ GB/s

Memory Bandwidth

Modern processors may have multiple channels:

- 4 channels
- 6 channels
- 8 channels
- 12 channels

Thus, if the memory of the previous example has 8 channels, the bandwidth is

$$8 \times 51.2 = 409.6 \text{ GB/s}$$

Compute-Bound vs Memory-Bound

Compute-Bound means that the processor limits performance

Examples:

- Dense matrix multiplication
- Tensor operations

Memory-Bound means that the memory systems limits performance

Examples:

- Graph processing
- Database scans
- Sparse matrices

Introduction to the Roofline Model

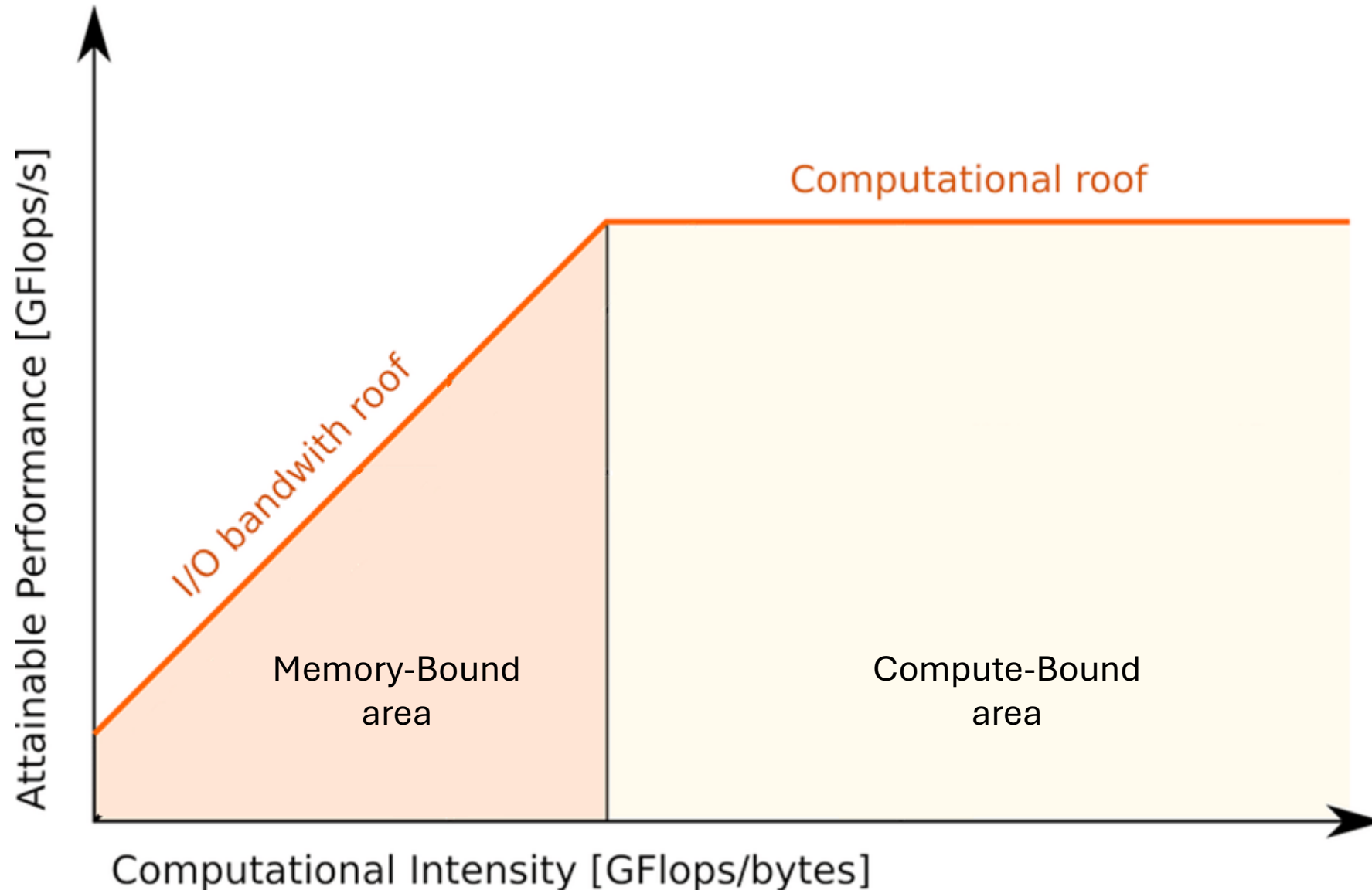
The Roofline model provides a visual performance bound

It combines:

- Peak compute performance
- Peak memory bandwidth

Into one graph

Introduction to the Roofline Model



Computational Intensity

Definition:

$$CI = \frac{\textit{Operations}}{\textit{Bytes transferred}}$$

Units of measure: FLOP/Bytes

Low CI -> many memory accesses and few computations

High CI -> many computations and relatively few memory accesses

Roofline Equation

Performance is bounded by:

$$Performance = \min(PeakCompute, CI \times Bandwidth)$$

Example: consider a simple addition

$$C = A + B$$

Operations: 1 addition

Data: 3 values

Very low CI, thus memory bound

Roofline Equation

Consider now the matrix-matrix multiplication

$$C = A \times B \text{ (triple for loop)}$$

Data reused many times!

High CI, thus compute bound

Memory Level Parallelism

Definition: the ability to overlap multiple memory accesses.

or

The processor's ability to handle multiple pending memory requests (like cache or TLB misses) at the same time.

Instead of:

Miss 1 -> Wait -> Miss 2 -> Wait -> Miss 3 -> Wait

We have:

Miss 1 -> Miss 2 -> Miss 3
simultaneously

Memory Level Parallelism

Suppose that a DRAM had a latency of 100 ns

Without memory level parallelism 10 accesses take 1000 ns

With perfect memory level parallelism 10 accesses take 100 ns

Sources of Memory Level Parallelism:

- Out-of-order Execution
 - Independent instructions continue execution
- Non-Blocking Caches
 - Allow multiple outstanding misses
- Bank-Level parallelism
 - Different DRAM banks operate simultaneously

Non-blocking cache

Traditional caches stop when a miss occurs until it is resolved

Non-blocking caches continue executing and issues more misses

The Miss Status Holding Registers (MSHRs) track outstanding misses.

Store:

- Missing block address
- Waiting instructions
- State information

Limitation of Memory Level Parallelism

Consider $p = p \rightarrow \text{next}$;

Each access depends on previous access,

This situation is known as pointer chasing. Very little memory level parallelism possible.

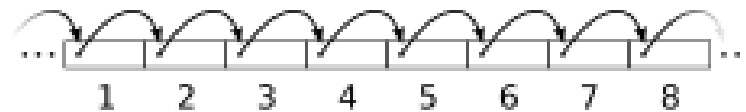


Memory Access Patterns

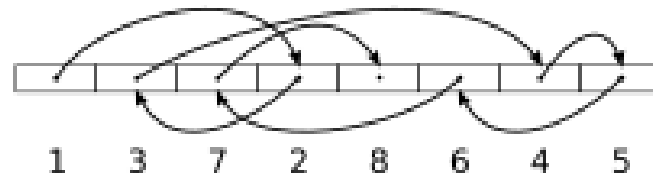
Three important patterns to access the memory:

- Sequential -> excellent locality
0,1,2,3,4
- Strided -> optimal for some DMAs and linear algebra
0,4,8,12
- Random

Sequential access



Random access



Hardware Prefetching

Goal: predict future accesses

Load data before it is requested

Benefits: hide latency

Different strategy for hardware prefetching:

- Next-line prefetcher
- Stride prefetcher

Hardware Prefetching

Next-Line Prefetcher:

Access: block i $\rightarrow$ automatically fetch: block $i+1$

A very simple strategy that works well for sequential streams

Stride Prefetcher:

Detects: 0, 4, 8, 12, 16 $\rightarrow$ stride of +4 $\rightarrow$ predicts next accesses

Useful for scientific workloads

Risks of Hardware Prefetching

Incorrect predictions:

- Waste bandwidth
- Pollute caches
- Increase energy

Good prefetchers balance:

- Coverage
- accuracy

Non-Uniform Memory Access

Modern servers have:

- Multiple CPUs
- Multiple memory controllers
- Local DRAM

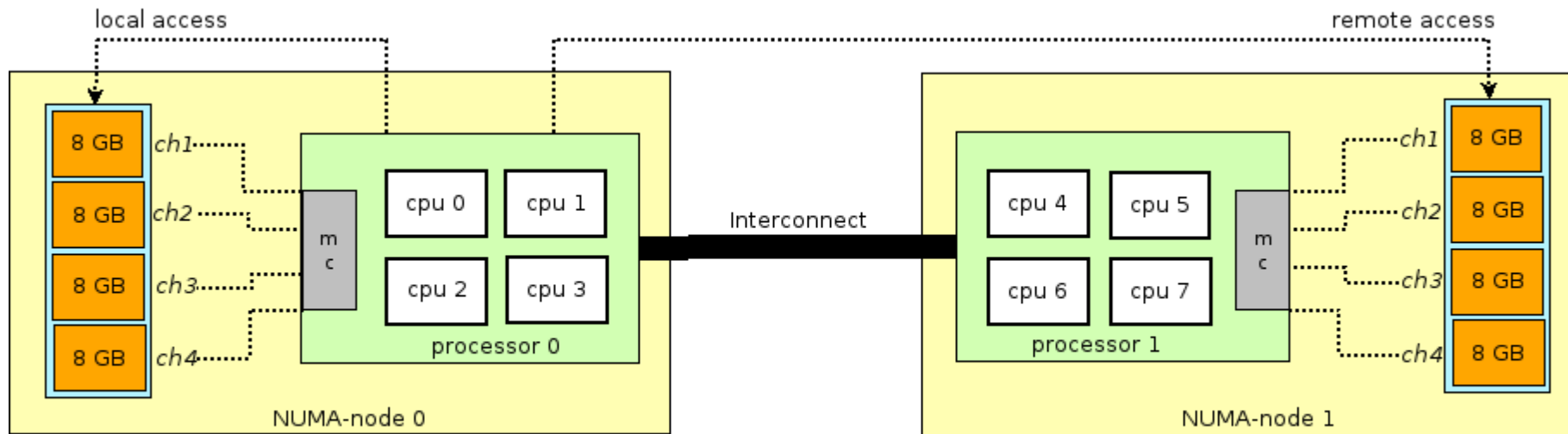
Memory latency depends on location

Example: CPU 0 has a local DRAM with 80 ns of latency and a remote DRAM of 140 ns

Remote accesses can be nearly twice as expensive

First-Touch Policy

Operating systems often allocate memory to the NUMA node of the thread that first accesses it.

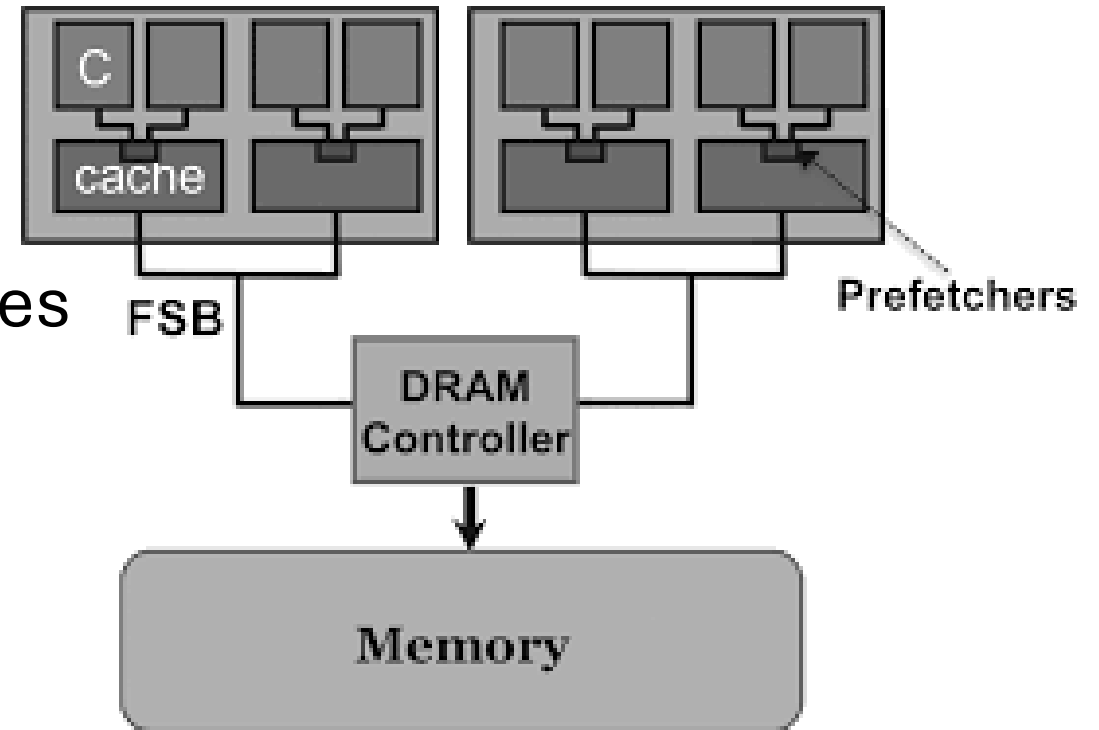


Multicore Memory Contention

All core share:

- Memory channels
- Memory controllers
- DRAM bandwidth

As core increase the contention increases

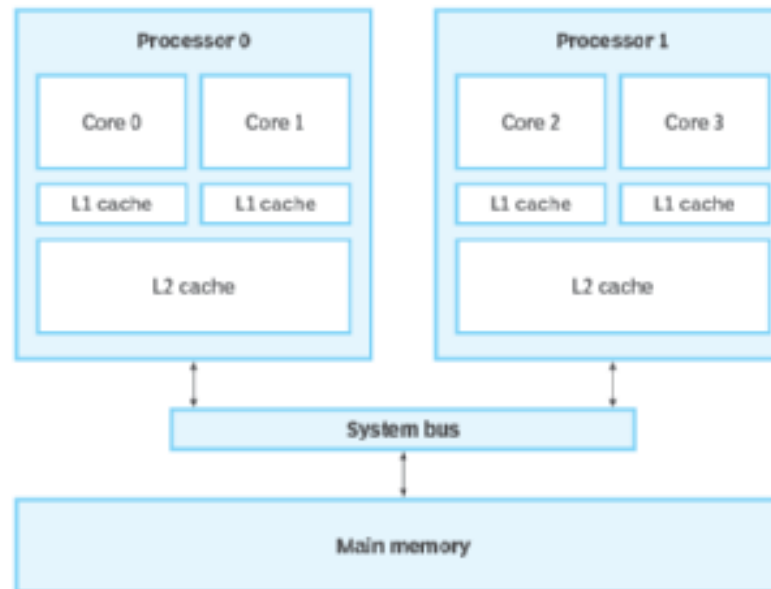


Bandwidth per Core

Example: 64-core processor with a total bandwidth of 256 GB/s

The bandwidth per core is 4 GB/s

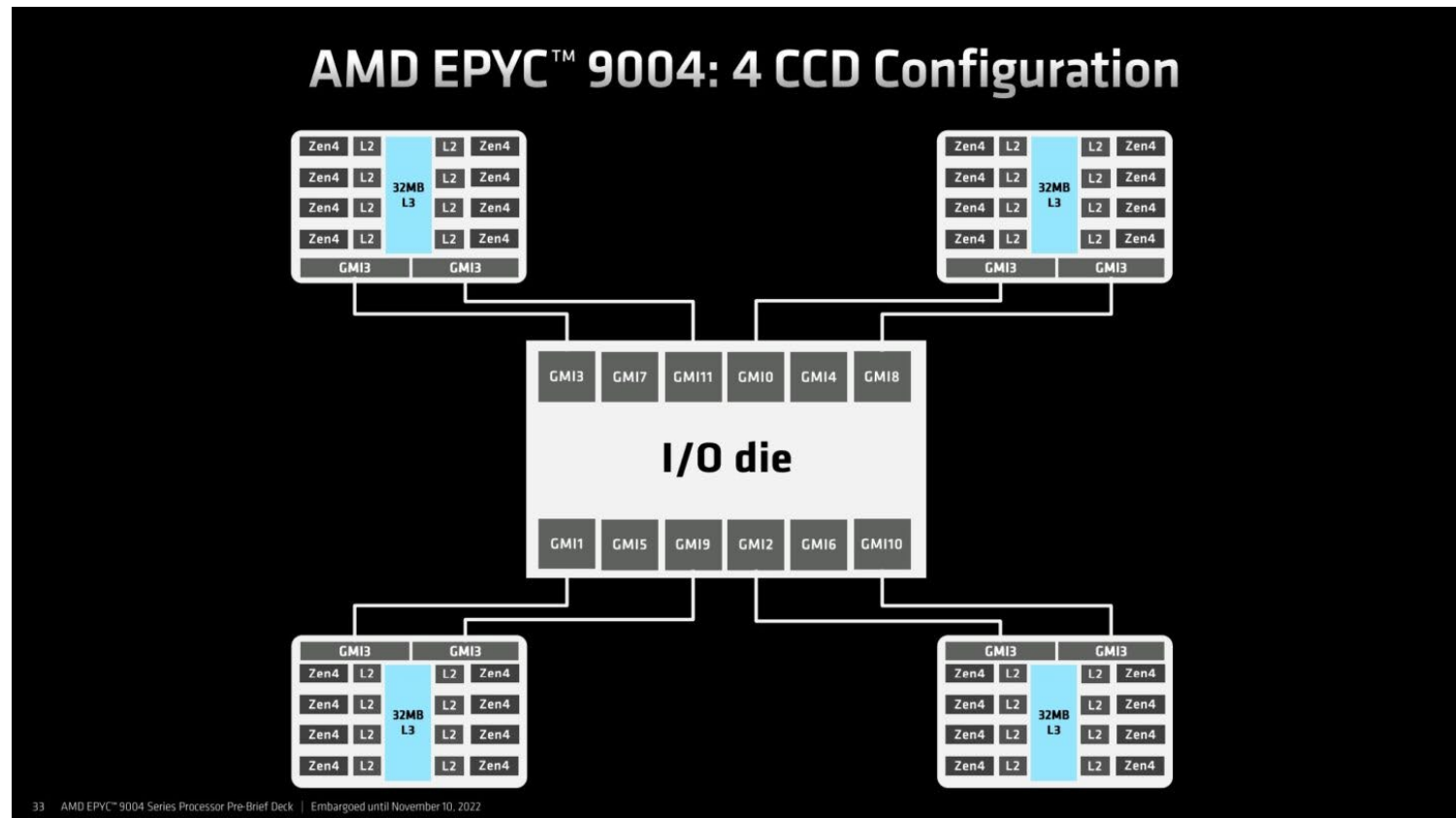
Adding cores does not automatically increase bandwidth



Case Study: AMD EPYC

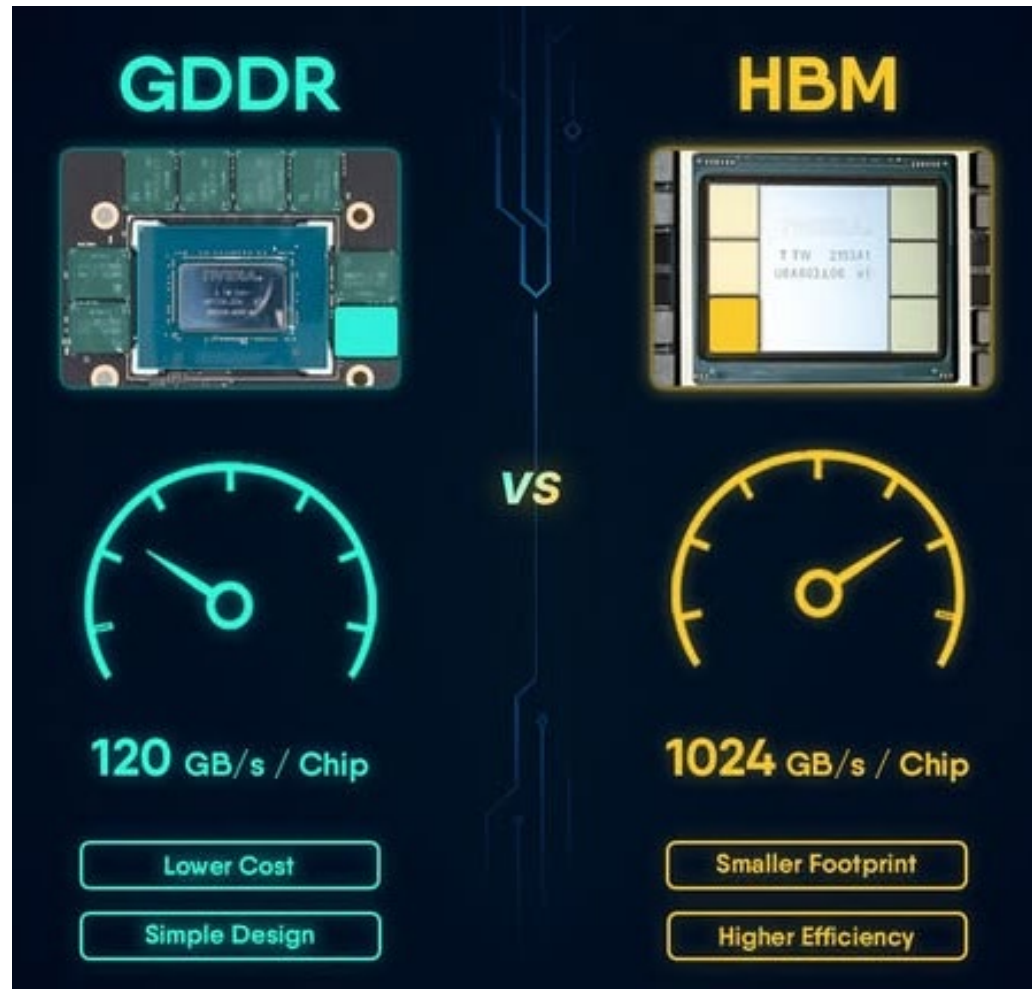
Many cores and many memory channels

Goal: increase memory bandwidth scalability



Case Study: AI Accelerators

GPUs use HBM memory, thus the bandwidth is often $> 1\text{TB/s}$

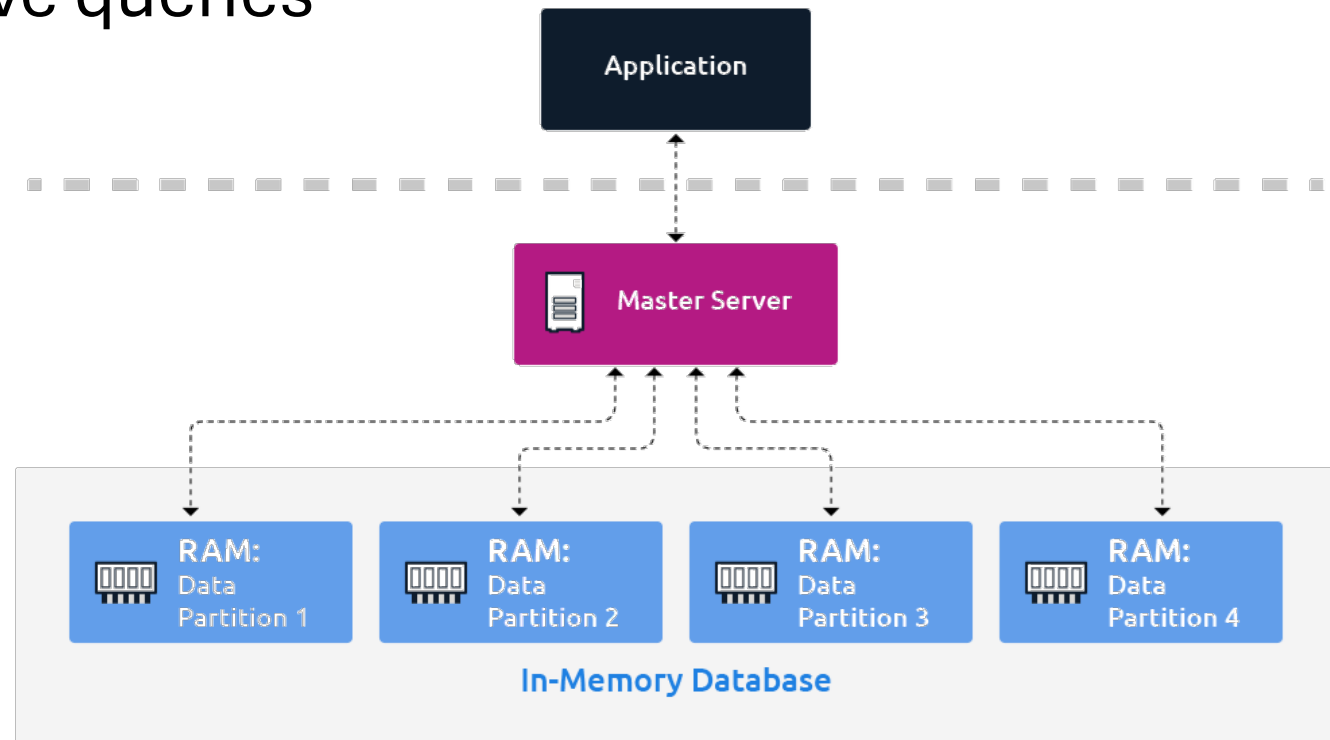


Case Study: Databases

Characteristics:

- Huge datasets
- Random accesses
- Memory –intensive queries

Often memory-bound.



Design Guidelines

To improve performance:

1. Increase locality
2. Increase computational intensity
3. Exploit memory level parallelism
4. Reduce memory traffic

Hardware and software must co-operate.

Closing

Key messages:

- Memory wall remains a major challenge
- Latency and bandwidth are different
- Roofline explains performance limits
- MLP hides latency
- NUMA affects scalability
- Modern systems are often memory-bound

Exercises on cache misses

Exercise 1

A processor has a fully associative L1 cache of 64 B with a block size of 16 B. The replacement policy is RLU. The address and data busses are 32 bits. With respect to the following assembly find:

- a) How many memory blocks are occupied by array?
- b) How many blocks are inside the cache memory?
- c) How many misses occur in the first loop?
- d) How many misses occur in the second loop?
- e) Calculate AMAT assuming a hit time of 1 clock cycle and a miss penalty of 5 clock cycles.

```
.data 0xACDEFF00
array: .space 128

.text

la    $t0, array
li    $t1, 32

loop1:
    lw    $t2, 0($t0)
    addi  $t0, $t0, 4
    addi  $t1, $t1, -1
    bne   $t1, $zero, loop1

la    $t0, array
li    $t1, 32

loop2:
    lw    $t2, 0($t0)
    addi  $t0, $t0, 4
    addi  $t1, $t1, -1
    bne   $t1, $zero, loop2
```

a)
occupied memory blocks -> $128/16 = 8$ blocks

b)
number of blocks $64/16 = 4$ blocks

c)
before loop1
address: 0xACDEFF00
block size 16 B -> 4 bits offset
fully associative cache -> tag = $32 - 4 = 28$ bits offset

Validity bit	Tag	Block
0		
0		
0		
0		

Iteration 1 of loop1

Load the content of 0xACDEFF00 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
0		
0		
0		

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 5 of loop1:

Load the content of 0xACDEFF10 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
0		
0		

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 9 of loop1

Load the content of 0xACDEFF20 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF2	Word 8 -> 11
0		

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 13 of loop1

Load the content of 0xACDEFF30 into cache memory (compulsory miss).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF2	Word 8 -> 11
1	0xACDEFF3	Word 12 -> 15

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Iteration 17 of loop1

Load the content of 0xACDEFF40 into cache memory (compulsory miss).

Since we are using LRU, the first cache block is evicted.

Validity bit	Tag	Block
1	0xACDEFF4	Word 16 -> 19
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF2	Word 8 -> 11
1	0xACDEFF3	Word 12 -> 15

In the next 3 loop1 iterations we have 3 hits (one hit per iteration).

Of course, this behaviour is the same for the next iterations and the pattern 1 miss and 3 hit is always the same. At the end of loop1, the cache has the following content.

Validity bit	Tag	Block
1	0xACDEFF4	Word 16 -> 19
1	0xACDEFF5	Word 20 -> 23
1	0xACDEFF6	Word 24 -> 27
1	0xACDEFF7	Word 28 -> 31

For loop1 we have 32 load instructions. We have 8 compulsory misses and 24 hits.

d)

before loop2 the cache stores the same data as at the end of loop1.

Iteration 1 of loop2

Load the content of 0xACDEFF00 into cache memory (capacity miss, since this address has already been cached in the past). Since the replacement policy is LRU, the content of the first cache line is evicted).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF5	Word 20 -> 23
1	0xACDEFF6	Word 24 -> 27
1	0xACDEFF7	Word 28 -> 31

In the next 3 loop2 iterations we have 3 hits (one hit per iteration).

Iteration 5 of loop2:

Load the content of 0xACDEFF10 into cache memory (capacity miss, since this address has already been cached in the past). Since the replacement policy is LRU, the content of the second cache line is evicted).

Validity bit	Tag	Block
1	0xACDEFF0	Word 0 -> 3
1	0xACDEFF1	Word 4 -> 7
1	0xACDEFF6	Word 24 -> 27
1	0xACDEFF7	Word 28 -> 31

As can be easily understood, we have the same access patterns of loop1, but all the misses (8 also in loop2) are capacity misses.

e)

$$AMAT = HitTime + MissRate \times MissPenalty$$

Number of memory accesses = 64 (32 in loop1 and 32 in loop2)

Number of misses = 16 (8 in loop1 and 8 in loop2)

$$MissRate = \frac{16}{64} = 0.25$$

$$AMAT = 1 + 0.25 \times 5 = 2.25$$

Exercise 2

A processor has a 2-way set associative L1 cache of 128 B with a block size of 16 B. The replacement policy is RLU. There is also a victim cache of one block. The address and data busses are 32 bits. With respect to the following assembly find:

- How are the address split?
- How many misses occur?
- Compute the AMAT assuming a hit time of 1 clock cycle and a miss penalty of 10 clock cycles.

```
.data 0x10001000
```

```
A: .space 144
```

```
.text
```

```
la $t0, A
```

```
la $t1, B
```

```
lw $t2, 0($t0)
```

```
lw $t2, 64($t0)
```

```
lw $t2, 128($t0)
```

```
lw $t2, 0($t0)
```

```
lw $t2, 64($t0)
```

```
lw $t2, 128($t0)
```

a)

number of cache blocks = 128/16= 8

number of lines in each way = 8/2 = 4

block size 16 B -> 4 bits offset

number of lines 4 -> index 2 bit

tag = 32 - 4 - 2 = 26

Way 0		
Validity bit	Tag	Block
0		
0		
0		
0		

Way 1		
Validity bit	Tag	Block
0		
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
0		

lw \$t2, 0(\$t0) -> the referenced address is 0x10001000

To better split the bits, we use the binary format instead of hexadecimal.

bx00010000000000000000100000000000

The offset is highlighted in red.

The index is highlighted in blue.

The tag is highlighted in green.

We have a compulsory miss.

Way 0		
Validity bit	Tag	Block
1	000100000000000000001000000	Word 0 -> 3
0		
0		
0		

Way 1		
Validity bit	Tag	Block
0		
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
0		

lw \$t2, 64(\$t0) -> the referenced address is 0x10001040

bx00010000000000000000100000100000

We have a compulsory miss. The referenced cache index is again 0, so the block is stored in Way1

Way 0		
V bit	Tag	Block
1	000100000000000000001000000	Word 0 -> 3
0		
0		
0		

Way 1		
V bit	Tag	Block
1	000100000000000000001000001	Word 64 -> 67
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
0		

lw \$t2, 128(\$t0) -> the referenced address is 0x10001080

bx00010000000000000000100001000000

We have a compulsory miss. Again, the referenced cache index is 0, the evicted block is the one in Way 0 (LRU). It is copied to the victim cache (with two more bits in the tag!)

Way 0		
V bit	Tag	Block
1	00010000000000000001000010	Word 128 -> 131
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000001	Word 64 -> 67
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100000000	Word 0 -> 3

lw \$t2, 0(\$t0) -> the referenced address is again 0x10001000

It is stored in the victim cache -> we have a victim hit! The evicted block is the one in Way1 (LRU).

Way 0		
V bit	Tag	Block
1	00010000000000000001000010	Word 128 -> 131
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000000	Word 0 -> 3
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100000100	Word 64 -> 67

lw \$t2, 64(\$t0) -> the referenced address is again 0x10001040

It is stored in the victim cache -> we have a victim hit! The evicted block is the one in Way0.

Way 0		
V bit	Tag	Block
1	00010000000000000001000001	Word 64 -> 67
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000000	Word 0 -> 3
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100001000	Word 128 -> 131

lw \$t2, 128(\$t0) -> the referenced address is again 0x10001080

It is stored in the victim cache -> we have a victim hit! The evicted block is the one in Way1.

Way 0		
V bit	Tag	Block
1	00010000000000000001000001	Word 64 -> 67
0		
0		
0		

Way 1		
V bit	Tag	Block
1	00010000000000000001000010	Word 128 -> 131
0		
0		
0		

Victim cache		
Validity bit	Tag	Block
1	0001000000000000000100000000	Word 0 -> 3

c)

$$AMAT = HitTime + MissRate \times MissPenalty$$

Number of memory accesses = 6

Number of misses = 3

$$MissRate = \frac{3}{6} = 0.5$$

$$AMAT = 1 + 0.5 \times 10 = 6$$

Exercise 3

A processor has a 2-way set associative L1 cache with a block size of 16 B and 16 lines. The replacement policy is RLU. The processor has a victim cache of 2 lines. The address and data busses are 32 bits. With respect to the following assembly code, compute the real size of the 2-way set associative cache memory and the AMAT (hit time and miss penalty as in Exercise 2).

```
.data 0xFFAB00A0
A: .space 16
.data 0xFFAB01A0
B: .space 16
.data 0xFFAB02A0
C: .space 16
.data 0xFFAB03A0
D: .space 16
```

```
.text
la $t0, A
la $t1, B
la $t2, C
la $t3, D
```

```
lw $s0, 0($t0)
lw $s0, 0($t1)
lw $s0, 0($t2)
lw $s0, 0($t3)
lw $s0, 0($t0)
lw $s0, 0($t1)
lw $s0, 0($t2)
lw $s0, 0($t3)
```

a)

block size 16 B -> 4 bits offset

number of lines 16 -> 4 bits for the cache index

tag = 32-4-4 = 24 bits

The, each way has 1 validity bit, 24 tag bits and 16 B (128 bits) block for each line.

Real cache size for each way = (1+24+128) * 16 = 2448 bits = 306 B

Since the cache has 2 ways the total real size is 612 B.

lw \$s0, 0(\$t0) -> The referenced address is 0xFFAB00A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFABB00.

The cache is empty, thus we have a compulsory miss.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
0		
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
0		
0		

lw \$s0, 0(\$t1) -> the referenced address is 0xFFAB01A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFAB01

This load causes a compulsory miss. Since the reference cache line is again 0xA, the block is cached in way1.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3 of A
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
0		
0		

lw \$s0, 0(\$t2) -> the referenced address is 0xFFAB02A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFAB02

This load causes a compulsory miss. Since the reference cache line is again 0xA, the block is cached in way0.

The evicted block goes in the victim cache.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
0		

lw \$s0, 0(\$t3) -> the referenced address is 0xFFAB03A0, the offset is 0x0, the cache line is 0xA and the tag is 0xFFAB03

This load causes a compulsory miss. Since the reference cache line is again 0xA, the block is cached in way1. The evicted block goes in the victim cache.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB03	Word 0->3 of D
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
1	FFAB01A	Word 0->3 of B

lw \$s0, 0(\$t0) -> The referenced address is 0xFFAB00A0, the block is stored in the victim cache. This block is swapped with the one in Way0.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3 of A
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB03	Word 0->3 of D
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB02A	Word 0->3 of C
1	FFAB01A	Word 0->3 of B

lw \$s0, 0(\$t1) -> the referenced address is 0xFFAB01A0, the block is stored in the victim cache. This block is swapped with the one in Way1.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB00	Word 0->3 of A
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB02A	Word 0->3 of C
1	FFAB03A	Word 0->3 of D

lw \$s0, 0(\$t2) -> the referenced address is 0xFFAB02A0, the block is stored in the victim cache. This block is swapped with the one in Way0.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB01	Word 0->3 of B
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
1	FFAB03A	Word 0->3 of D

lw \$s0, 0(\$t3) -> the referenced address is 0xFFAB03A0, the block is stored in the victim cache. This block is swapped with the one in Way1.

Index	Way 0		
	V bit	Tag	Block
0	0		
1	0		
...	...	...	...
10	1	FFAB02	Word 0->3 of C
11	0		
...	...	...	...
15	0		

Way 1		
V bit	Tag	Block
0		
0		
...	...	...
1	FFAB03	Word 0->3 of D
0		
...	...	...
0		

Victim cache		
Validity bit	Tag	Block
1	FFAB00A	Word 0->3 of A
1	FFAB01A	Word 0->3 of B

Number of memory accesses = 8

Number of misses = 4

$$MissRate = \frac{4}{8} = 0.5$$

$$AMAT = 1 + 0.5 \times 10 = 6$$

Exercise 4

A processor has a direct mapped L1 cache with a block size of 16 B and a size of 32 B and a L2 direct mapped cache with a block size of 16 B and a size of 64 B. The address and data busses are 8 bits. With respect to the following assembly code, compute the AMAT (Hit Time L1=1, Hit Time L2=5, Miss Penalty L2 =50).

```
.data 0x00
A: .space 16
.data 0x20
B: .space 16
.data 0x40
C: .space 16
```

```
.text
la $t0, A
la $t1, B
la $t2, C
```

```
lw $s0, 0($t0)
lw $s0, 0($t1)
lw $s0, 0($t0)
lw $s0, 0($t2)
lw $s0, 0($t1)
lw $s0, 0($t0)
```

L1 cache

Block size 16 B -> 4 bit offset

Number of lines -> 32B/16B = 2 -> 1 cache index bit

Tag -> 8-4-1= 3 bits

L2 cache

Block size 16 B -> 4 bit offset

Number of lines -> 64B/16B = 4 -> 2 cache index bits

Tag -> 8-4-2=2 bits

L1			
index	V bit	Tag	Block
0	0		
1	0		

L2			
index	V bit	Tag	Block
0	0		
1	0		
2	0		
3	0		

lw \$s0, 0(\$t0) -> The referenced address is 0x00 -> Compulsory miss and caching in L1 and L2.

Address split in L1 -> bx000 0 0000

Address split in L2 -> bx00 00 0000

L1			
index	V bit	Tag	Block
0	1	000	Array A
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	0		
3	0		

lw \$s0, 0(\$t1) -> The referenced address is 0x20 -> Compulsory miss and caching in L1 and L2.

Address split in L1 -> bx001 0 0000

Address split in L2 -> bx00 10 0000

L1			
index	V bit	Tag	Block
0	1	001	Array B
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t0) -> The referenced address is 0x00 -> miss in L1 and hit L2.

Address split in L1 -> bx000 0 0000

Address split in L2 -> bx00 00 0000

L1			
index	V bit	Tag	Block
0	1	000	Array A
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t2) -> The referenced address is 0x40 -> Compulsory miss and caching in L1 and L2.

Address split in L1 -> bx010 0 0000

Address split in L2 -> bx01 00 0000

L1			
index	V bit	Tag	Block
0	1	010	Array C
1	0		

L2			
index	V bit	Tag	Block
0	1	01	Array C
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t1) The referenced address is 0x20 -> miss in L1 and hit in L2

Address split in L1 -> bx001 0 0000

Address split in L2 -> bx00 10 0000

L1			
index	V bit	Tag	Block
0	1	001	Array B
1	0		

L2			
index	V bit	Tag	Block
0	1	01	Array C
1	0		
2	1	00	Array B
3	0		

lw \$s0, 0(\$t0) The referenced address is 0x00 -> miss in L1 and L2 and caching.

Address split in L1 -> bx000 0 0000

Address split in L2 -> bx00 00 0000

L1			
index	V bit	Tag	Block
0	1	000	Array A
1	0		

L2			
index	V bit	Tag	Block
0	1	00	Array A
1	0		
2	1	00	Array B
3	0		

To summarize:

Instruction	L1	L2
lw \$s0, 0(\$t0)	Miss	Miss
lw \$s0, 0(\$t1)	Miss	Miss
lw \$s0, 0(\$t0)	Miss	Hit
lw \$s0, 0(\$t2)	Miss	Miss
lw \$s0, 0(\$t1)	Miss	Hit
lw \$s0, 0(\$t0)	Miss	Miss

Number of memory accesses = 6

Number of misses in L1 = 6

Number of misses in L2 = 4

$$MR_{L1} = \frac{6}{6} = 1$$

$$MR_{L2} = \frac{4}{6} = 0.\bar{6}$$

$$AMAT = HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2} \times MP_{L2}) = 1 + 1 \times (5 + 0.\bar{6} \times 50) = 39.\bar{3}$$

Exercise on MESI protocol and consistency

A processor has two cores (P0 and P1), each one with a L1 direct-mapped private cache with 8 lines and a block size of 16 B. The coherence protocol is MESI. Show the evolution of the MESI states and of the caches.

```
.data 0x0120
X: .word 10

.text
# P0
lw $t0, 0x0120($zero)
li $t1, 20
sw $t1, 0x0120($zero)
lw $t2, 0x0120($zero)

# P1
lw $s0, 0x0120($zero)
lw $s1, 0x0120($zero)
```

Assume that the order of execution is interleaved, in terms of memory operations, between P0 and P1.

Partial Solution (only MESI states evolution is shown)

4 bits offset, 3 bits tag and 9 bits tag.

```
P0: lw $t0, 0x0120($zero) -> E
P1: lw $s0, 0x0120($zero) -> S
P0: sw $t1, 0x0120($zero) -> M e I
P1: lw $s1, 0x0120($zero) -> S
P0: lw $t2, 0x0120($zero) -> local hit
```

Exercise 1

A processor has 16-bit virtual addresses and a page size of 256 B. The system has a single level page table. The processor is equipped with a 2-entry fully associative TLB with LRU replacement policy. Moreover, it has a direct mapped L1 cache of 256 B with a block of 16 B. Consider the following assembly code and compute EMAT and AMAT. Assume that $T_{L1HIT}=1$ cycle, Miss Penalty = 50 cycles, $T_{TLBHIT} = 1$ cycle, $T_{PT}= 20$ cycles (time taken by the page walk in case of TLB miss).

```
lw $t0, 0x1000($zero)
lw $t1, 0x1004($zero)
lw $t2, 0x1100($zero)
lw $t3, 0x1000($zero)
```

Consider that the page table has been initialized by the OS with the following content:

VPN	PFN	Valid
0x10	0x20	1
0x11	0x21	1

Solution

Virtual address split: 8 bit VPN and 8 bit offset

Physical address split: 8 bit tag, 4 bit cache index, 4 bit offset

TLB and cache initially empty.

Valid	VPN	PFN
0		
0		

Index	Validity bit	Tag	Block
0	0		
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1000(\$zero)

The virtual address 0x1000 is translated into the physical address 0x2000. There is a TLB miss and a L1 cache miss. After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x10	0x20
0		

Index	Validity bit	Tag	Block
0	1	0x20	16 B
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1004(\$zero)

The virtual address 0x1004 is translated into the physical address 0x2004. There is a TLB hit and a L1 cache hit. After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x10	0x20
0		

Index	Validity bit	Tag	Block
0	1	0x20	16 B
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1100(\$zero)

The virtual address 0x1100 is translated into the physical address 0x2100. There is a TLB miss and a L1 cache miss (the block on line 0 is evicted). After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x10	0x20
1	0x11	0x21

Index	Validity bit	Tag	Block
0	1	0x21	16 B
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1000(\$zero)

The virtual address 0x1000 is translated into the physical address 0x2000. There is a TLB hit and a L1 cache miss (the block on line 0 is evicted). After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x10	0x20
1	0x11	0x21

Index	Validity bit	Tag	Block
0	1	0x20	16 B
1	0		
...	...	...	...
15	0		

In summary we have: 2 TLB misses and 2 TLB hits, 1 L1 hit and 3 L1 misses.

$$AMAT = T_{L1HIT} + MR_{L1} \times \text{Miss Penalty} = 1 + \frac{3}{4} \times 50 = 38.5$$

$$EMAT = AMAT + T_{TLBHIT} + MR_{TLB} \times T_{PT} = 38.5 + 1 + \frac{1}{2} \times 20 = 49.5$$

Exercise 2

A processor has 16-bit virtual addresses and a page size of 256 B. The systems have a two-levels page table. The processor is equipped with a 2-entry fully associative TLB with LRU replacement policy. Moreover, it has a direct mapped L1 cache of 256 B with a block of 16 B. Consider the following assembly code and compute EMAT and AMAT. Assume that $T_{L1HIT} = 1$ cycle, Miss Penalty = 50 cycles, $T_{TLBHIT} = 1$ cycle, $T_{PT} = 20$ cycles (time taken by the page walk in case of TLB miss).

```
lw $t0, 0x1000($zero)
lw $t1, 0x1004($zero)
lw $t2, 0x1100($zero)
lw $t3, 0x2000($zero)
lw $t4, 0x1000($zero)
```

Consider that the page tables have been initialized by the OS with the following content:

Page Directory (level 1)

L1 index	Second-level page table
0x1	PT1
0x2	PT2

PT 1

L2 index	PFN	Valid
0x0	0x20	1
0x1	0x21	1

PT 2

L2 index	PFN	Valid
0x0	0x30	1

Solution

Virtual address split: 8 bit VPN (divided into 4 bit for L1 and 4 bit for L2) and 8 bit offset

Physical address split: 8 bit tag, 4 bit cache index, 4 bit offset

TLB and cache initially empty.

Valid	VPN	PFN
0		
0		

Index	Validity bit	Tag	Block
0	0		
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1000(\$zero)

The virtual address 0x1000 is translated into the physical address 0x2000. There is a TLB miss and a L1 cache miss. After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x10	0x20
0		

Index	Validity bit	Tag	Block
0	1	0x20	16 B
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1004(\$zero)

The virtual address 0x1004 is translated into the physical address 0x2004. There is a TLB hit a L1 cache hit. After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x10	0x20
0		

Index	Validity bit	Tag	Block
0	1	0x20	16 B
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1100(\$zero)

The virtual address 0x1100 is translated into the physical address 0x2100. There is a TLB miss and a L1 cache miss. After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x10	0x20
1	0x11	0x21

Index	Validity bit	Tag	Block
0	1	0x21	16 B
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x2000(\$zero)

The virtual address 0x2000 is translated into the physical address 0x3000. There is a TLB miss and a L1 cache miss. After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x20	0x30
1	0x11	0x21

Index	Validity bit	Tag	Block
0	1	0x30	16 B
1	0		
...	...	...	...
15	0		

- lw \$t0, 0x1000(\$zero)

The virtual address 0x1000 is translated into the physical address 0x2000. There is a TLB miss and a L1 cache miss. After the lw instruction the status of the TLB and of the L1 cache is:

Valid	VPN	PFN
1	0x20	0x30
1	0x10	0x20

Index	Validity bit	Tag	Block
0	1	0x20	16 B
1	0		
...	...	...	...
15	0		

$$AMAT = T_{L1HIT} + MR_{L1} \times \text{Miss Penalty} = 1 + 4/5 \times 50 = 41$$

$$EMAT = AMAT + T_{TLBHIT} + MR_{TLB} \times T_{PT} = 41 + 1 + 4/5 \times 20 = 58$$

Exercise 1 – Address Decomposition

A system uses 32-bit virtual addresses and a page size of 4 KB.

Answer the following questions:

1. How many bits are used for the page offset?
2. How many bits are used for the Virtual Page Number (VPN)?
3. Decompose the virtual address 0x12345678 into VPN and offset.

Solution

A page size of 4 KB corresponds to:

4 KB = 4096 bytes = 2^{12} bytes

Therefore, 12 bits are required for the page offset.

Since the virtual address is 32 bits wide:

VPN bits = $32 - 12 = 20$ bits

To decompose 0x12345678:

Offset = lower 12 bits = 0x678

VPN = upper 20 bits = 0x12345

Final answer:

VPN = 0x12345

Offset = 0x678

Exercise 2 – Physical Address Translation

A system uses:

- 32-bit virtual addresses
- 4 KB pages
- the following page table mapping:

VPN 0x12345 -> PFN 0xABCDE

Translate the virtual address 0x12345678 into a physical address.

Solution

The offset is determined by the page size.

4 KB = 2^{12} bytes, therefore the offset occupies 12 bits.

Virtual address: 0x12345678

VPN = 0x12345

Offset = 0x678

The page table maps:

VPN 0x12345 -> PFN 0xABCDE

The physical address is obtained by concatenating PFN and offset.

Physical address = 0xABCDE678

Exercise 3 – Single-Level Page Table Size

Consider a system with:

- 48-bit virtual addresses
- page size = 4 KB
- each Page Table Entry (PTE) = 8 bytes

Compute:

1. Number of virtual pages
2. Total size of a single-level page table

Solution

Page size = 4 KB = 2^{12} bytes.

Number of virtual pages:

$$2^{48} / 2^{12} = 2^{36} \text{ pages}$$

Each PTE requires 8 bytes.

Total page table size:

$$2^{36} \times 8 \text{ bytes} = 2^{39} \text{ bytes}$$

$$2^{39} \text{ bytes} = 512 \text{ GB}$$

Exercise 4 – TLB Reach

A processor contains a TLB with 512 entries.

Compute the TLB reach for:

1. 4 KB pages
2. 2 MB pages

Solution

TLB reach is defined as:

$$\text{TLB Reach} = \text{Number of Entries} \times \text{Page Size}$$

Case 1: 4 KB pages

$$512 \times 4 \text{ KB} = 2048 \text{ KB} = 2 \text{ MB}$$

Case 2: 2 MB pages

$$512 \times 2 \text{ MB} = 1024 \text{ MB} = 1 \text{ GB}$$

Exercise 5 – Effective Memory Access Time (EMAT)

A processor has:

- TLB hit time = 1 cycle
- TLB miss rate = 2%
- Page walk penalty = 120 cycles

Solution

EMAT formula:

$$\text{EMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Substituting the values:

$$\text{EMAT} = 1 + 0.02 \times 120$$

$$\text{EMAT} = 1 + 2.4$$

$$\text{EMAT} = 3.4 \text{ cycles}$$

Exercise 6 – TLB Miss Penalty

A system uses 4-level paging.

Assume:

- each memory access requires 100 cycles
- all page table entries are already in DRAM

Compute the latency of a TLB miss.

Solution

With 4-level paging, translation requires:

- 4 accesses for page table traversal
- 1 additional access for actual data

Total accesses = 5

Latency = 5×100 cycles = 500 cycles

Exercise 7 – FIFO Page Replacement

A system has 3 physical frames.

Reference string:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Simulate FIFO page replacement and compute the number of page faults.

Solution

Frames initially empty.

1 -> fault
2 -> fault
3 -> fault
4 -> replace 1 -> fault
1 -> replace 2 -> fault
2 -> replace 3 -> fault
5 -> replace 4 -> fault
1 -> hit
2 -> hit
3 -> replace 1 -> fault
4 -> replace 2 -> fault
5 -> hit

Total page faults = 9

Exercise 8 – LRU Page Replacement

Using the same reference string from Exercise 9 and 3 frames, simulate LRU replacement.

Solution

Simulation:

1 -> fault
2 -> fault
3 -> fault
4 -> replace 1 -> fault
1 -> replace 2 -> fault
2 -> replace 3 -> fault
5 -> replace 4 -> fault
1 -> hit
2 -> hit
3 -> replace 5 -> fault
4 -> replace 1 -> fault
5 -> replace 2 -> fault

Total page faults = 10

Exercise 9 – Thrashing Analysis

A system has:

- physical memory = 8 GB
- application working set = 16 GB

Explain what is likely to happen.

Solution

The working set exceeds available physical memory.

The operating system continuously swaps pages in and out of memory.

This produces:

- high page fault rate
- heavy disk activity
- low CPU utilization

The system enters a condition called thrashing.

Performance collapses because the CPU spends more time waiting for paging operations than executing instructions.

Exercise 10 – Combined Cache and TLB Miss

A system has:

- TLB hit rate = 99%
- TLB miss penalty = 100 cycles
- L1 cache hit time = 1 cycle
- L1 miss rate = 5%

- L1 miss penalty = 40 cycles

Compute:

1. Effective translation overhead
2. Effective cache access time

Solution

Step 1 – Translation overhead

$$\text{EMAT} = 1 + 0.01 \times 100$$

$$\text{EMAT} = 2 \text{ cycles}$$

Step 2 – Cache AMAT

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

$$\text{AMAT} = 1 + 0.05 \times 40$$

$$\text{AMAT} = 1 + 2$$

$$\text{AMAT} = 3 \text{ cycles}$$

Exercise 11 – Page Fault Impact on Performance

A page fault requires 8 ms to load a page from disk.

Assume the processor clock frequency is 2 GHz.

Estimate how many processor cycles are lost during a page fault.

Solution

$$\text{Clock frequency} = 2 \text{ GHz} = 2 \times 10^9 \text{ cycles/s.}$$

$$\text{Page fault latency} = 8 \text{ ms} = 8 \times 10^{-3} \text{ s.}$$

Lost cycles:

$$(2 \times 10^9) \times (8 \times 10^{-3})$$

$$= 16 \times 10^6 \text{ cycles}$$

$$= 16 \text{ million cycles.}$$